Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.DOI

A Survey on Spiking Neural Network Foundation And Recent Progress

NHAN TRONG LUU[1, 3] (MEMBER, IEEE), DUONG TRUNG LUU[2], PHAM NGOC NAM[3] (MEMBER, IEEE), AND TRUONG CONG THANG[4] (SENIOR MEMBER, IEEE)

[1]College of Information and Communication Technology, Can Tho University, Can Tho, Vietnam
[2]Center for Digital Transformation and Communication, Can Tho University, Can Tho, Vietnam
[3]College of Engineering and Computer Science, VinUniversity, Hanoi, Vietnam
[4]Department of Computer Science and Engineering, The University of Aizu, Aizuwakamatsu, Japan

Email: luutn@ctu.edu.vn, luutd@ctu.edu.vn, nam.pn@vinuni.edu.vn, thang@u-aizu.ac.jp.
Corresponding to: Luu Trong Nhan.

ABSTRACT Over the past decade, deep neural networks (DNNs) have achieved remarkable success across numerous fields. Despite this progress, DNNs remain highly demanding in terms of computational power, energy consumption, and data requirements. As the demand for autonomous systems such as self-driving cars, drones, collaborative robots, etc. continues to rise, the deployment of DNNs in these real-world scenarios has gained significant attention. However, these applications require energy-efficient and computationally lightweight models to ensure real-time performance under limited power resources. A promising alternative has emerged in the form of biologically inspired spiking neural networks (SNNs), which offer a more energy-efficient and temporally dynamic computing paradigm. By mimicking the behavior of biological neurons, SNNs provide a bridge between neuroscience and machine learning, leveraging neural sparsity and temporal coding mechanisms inherent to biological systems. In this work, we make the following contributions: provide a comprehensive overview of biological neuron theories; summarize existing spike-based neuron models from neuroscience; describe synaptic modeling approaches; review conventional artificial neural networks; outline several methodologies for preprocessing, training and conversion approaches for spike-based models; explore SNN applications in various fields; discuss major frameworks supporting SNN implementation.

INDEX TERMS Spiking neural network, computer vision, toolkits, survey, review, natural language processing, reinforcement learning, graph neural network

I. INTRODUCTION

OVER the past decade, artificial neural networks (ANNs) have evolved dramatically, from the early multi-layer perceptrons (MLPs) to the advanced deep neural networks (DNNs) that dominate modern machine learning [1], [2]. This progress has been fueled by the availability of large-scale labeled datasets, powerful computing resources and graphics processing units (GPUs) [3]. Despite their impressive achievements, developing a high performance and energy efficient model remains an attention-drawing topic in ANN research [4], [5].

To address these limitations, researchers have proposed numerous strategies to reduce the computational and energy demands of traditional deep learning models [4], [5]. Efforts such as quantization [4], pruning [5], and knowledge distillation [6] aim to develop more compact networks that preserve performance while minimizing complexity and parameter count. Quantization simplifies computation by converting network weights and activations into low-precision integer formats [4]. Pruning iteratively removes redundant connections during or after training, reducing network size [5]. Knowledge distillation, on the other hand, transfers the knowledge of a large "teacher" model into a smaller "student" model to achieve comparable performance with lower cost [6].

Although ANNs and DNNs were originally inspired by the brain, their architectures, computational principles, and learning mechanisms differ fundamentally from biological neural systems [7]. This realization has driven the emergence of spiking neural networks (SNNs), which aim to overcome the limitations of conventional ANNs [8]. SNNs communicate using discrete electrical impulses, or spikes, and operate in continuous time, closely mimicking biological neurons [9]. When implemented on neuromorphic hardware such

as IBM's TrueNorth [10], Intel's Loihi [11], SpiNNaker [12], or NeuroGrid [13], SNNs offer a highly promising approach to achieving energy-efficient computation.

Thanks to their biological plausibility, SNNs naturally exploit sparsity and are inherently compatible with temporal coding mechanisms [7], [14], [15]. While their performance still lags behind that of DNNs in many tasks, this gap is steadily narrowing [16], [17]. Moreover, SNNs typically consume far less energy, making them highly attractive for real-time and low-power applications [18], [19], [17], [20]. By consolidating the theoretical underpinnings and recent breakthroughs in SNN research, this paper aims to clarify the current landscape and catalyze new research directions within the neuromorphic community. Particularly, we make the following contributions:

- Establish the foundations of neural computation, synthesizing biological neuron theories, conventional ANN knowledge as the basis for understanding SNNs.
- Provide a treatment of spike representation and encoding schemes that govern how real-valued data is transformed into spike-based formats.
- Categorize spiking neuron models into a principled taxonomy (conductance-based, integrate-and-fire family, and phenomenological models) alongside synaptic transmission models, enabling readers to select appropriate models for their applications.
- Organize training and optimization paradigms into gradient-based strategies (surrogate gradients and ANN-to-SNN conversion, local learning, hybrid training) and biologically plausible learning rules (STDP variants, reward-modulated plasticity, and homeostatic mechanisms).
- Systematically investigate the expanding range of SNN applications across diverse real-world domains, including foundational knowledge of these problems and detailed explanations on how current state-of-the-art (SOTA) SNNs are designed to tackle given issues. Topics covered include: computer vision, graph learning, natural language processing, hardware implementations, and emerging topics beyond the usage of neurons and synapses. To the best of our knowledge, our survey is the only existing SNN survey that aims to cover these problems in such broad scope, with relatively detailed explanations on how these SOTA methods work.
- Finally, we conclude with an analysis of the core open-source software frameworks currently facilitating the development and deployment of SNNs.

The paper is organized in the following order: Section II and III introduces the biological and artificial neural network foundations. Section IV presents spiking neuron and synapse models organized by computational class. Section V details training and optimization paradigms. Section VI covers spike encoding schemes. The rest of the content then survey SNN applications in computer vision, graph learning, natural language processing, reinforcement learning, neuromorphic hardware, and emerging research beyond neurons and synapses, followed by available tools and frameworks, and concluding remarks.

II. BIOLOGICAL NEURONS

The foundational material on biological neurons and spiking neuron models has been thoroughly covered in several prior surveys [22], [7], [23], [24], [25]. More recent treatments have expanded these foundations to address large-scale SNN architectures [26], hardware-software co-design considerations [27], [28], and energy-efficiency analysis [19], [29]. In this section, we provide a concise but comprehensive summary of these topics to establish the necessary formalism; readers are referred to the mentioned works for extended treatments.

At the most basic level, a neuron is an electrochemical device. Its computational behavior emerges from the controlled movement of ions (primarily Na^+, K^+, and Cl^-) across a lipid membrane whose permeability is gated by voltage- and ligand-sensitive channels [30], [31]. Three anatomical compartments give rise to three distinct computational roles (Figure 1): the dendritic tree integrates incoming signals, the soma applies a nonlinear thresholding operation, and the axon propagates the resulting all-or-none action potential to downstream targets through chemical or electrical synapses [32]. This compartmentalization is not merely descriptive; it is the structural reason that point-neuron models (which collapse all three compartments into a single state variable) remain a useful but lossy abstraction.

A. MEMBRANE POTENTIAL

The instantaneous membrane potential V_m reflects the equilibrium between the electrochemical gradients of the permeant ion species and is described by the Goldman-Hodgkin-Katz (GHK) voltage equation [30]:

$$V_m = \frac{RT}{F} \cdot \ln \left(\frac{\begin{array}{l} P_K[K^+]_{\text{out}} \\ + P_{Na}[Na^+]_{\text{out}} + P_{Cl}[Cl^-]_{\text{in}} \end{array}}{\begin{array}{l} P_K[K^+]_{\text{in}} \\ + P_{Na}[Na^+]_{\text{in}} + P_{Cl}[Cl^-]_{\text{out}} \end{array}} \right) \quad (1)$$

with R as the universal gas constant, the absolute temperature T, the Faraday constant F, and the membrane permeability P_X to ion species X. Equation 1 is important less for its precise numerical predictions than for what it implies about neural computation: the membrane potential is a ratio of weighted ion concentrations, which means that even modest, transient changes in relative permeability can drive large excursions in V_m. This sensitivity is precisely what voltage-gated channels exploit to generate spikes.

Two regimes illustrate the dynamic range of this mechanism. At rest, the relative permeabilities $P_{\mathrm{K}} : P_{\mathrm{Na}} : P_{\mathrm{Cl}}$ follow approximately $1 : 0.04 : 0.45$, placing V_m near -70 mV (the well-known resting potential of mammalian neurons) [33]. When a sufficiently strong depolarizing input opens voltage-gated sodium channels, $P_{\mathrm{Na}}/P_{\mathrm{K}}$ rises by more than two orders of magnitude, the equilibrium swings toward the sodium

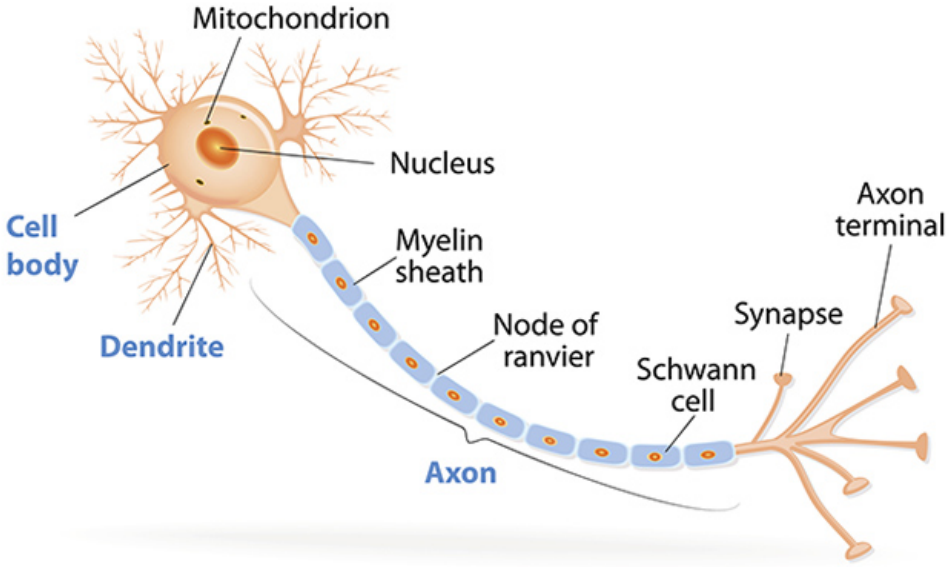


FIGURE 1: Neuron components and structure, illustrated in [21].

reversal potential, and the membrane briefly approaches $+40$ mV before delayed potassium efflux restores the resting state. The entire excursion lasts on the order of a millisecond and is, to a very good approximation, stereotyped: its shape carries no information, only its occurrence in time does [32].

B. COMPUTATIONAL DENDRITES

Recent neuroscience research has increasingly recognized that dendrites are not passive cables but active computational elements in their own right [34], [35]. Individual dendritic branches can perform nonlinear operations such as supralinear summation mediated by NMDA receptor-dependent plateau potentials, enabling single biological neuron to act as a multilayer computing unit [35], [36]. This perspective has motivated the development of frameworks such as Dendrify [37], which provides reduced compartmental neuron models that incorporate biologically relevant dendritic and synaptic integration properties while remaining computationally tractable for large-network simulations. Furthermore, studies on human cortical pyramidal neurons have revealed that human layer 2/3 pyramidal neurons exhibit enhanced dendritic signaling, faster somatic kinetics, and a greater number of functional dendritic compartments compared to their rodent counterparts [38], [39], suggesting that dendritic complexity may contribute to species-specific cognitive capabilities.

The computational relevance of dendritic structure extends beyond biology into machine learning. Dendritic integration has been shown to follow a quadratic rule for synaptic inputs, which enables quadratic neurons to inherently capture correlation within structured data [40]. This finding has inspired architectures such as Dendritic integration-inspired CNNs (Dit-CNNs), which achieve competitive performance on classification benchmarks while retaining the simplicity of traditional architectures. Moreover, distinct interneuron subtypes in the neocortex employ different dendritic integration strategies (somatostatin-expressing interneurons use NMDA-dependent supralinear integration for sustained activity, while parvalbumin-expressing interneurons rely on sublinear integration for precise tracking of fast-changing signals [41]) highlighting the diversity of computational primitives available within a single neural circuit.

III. ANN MODELS

In recent years, ANNs have been widely adopted across domains addressing scientific, engineering, and industrial challenges [43], [44]. Their key advantage over traditional analytical methods lies in adaptability, enabling effective data-driven analysis and optimization in diverse fields, including business, social sciences, and the arts [45]. Applications span industrial manufacturing, petroleum exploration [46], and business operations [47].

ANNs are powerful computational models inspired by the human brain, capable of handling complex systems with many inputs while maintaining high accuracy and implementation flexibility [48]. As interest in brain function has increased [49], neural-inspired computation has advanced rapidly. An ANN can be viewed as a machine that emulates cognitive processes of the brain [50].

The biological brain is a highly efficient information-processing system, where interconnected neurons collaboratively perform tasks such as perception, reasoning, and decision-making [51]. ANNs mirror this structure through networks of interconnected nodes that process and transmit signals. Similarly, ANNs have achieved strong performance in tasks such as medical diagnosis [52], speech and facial

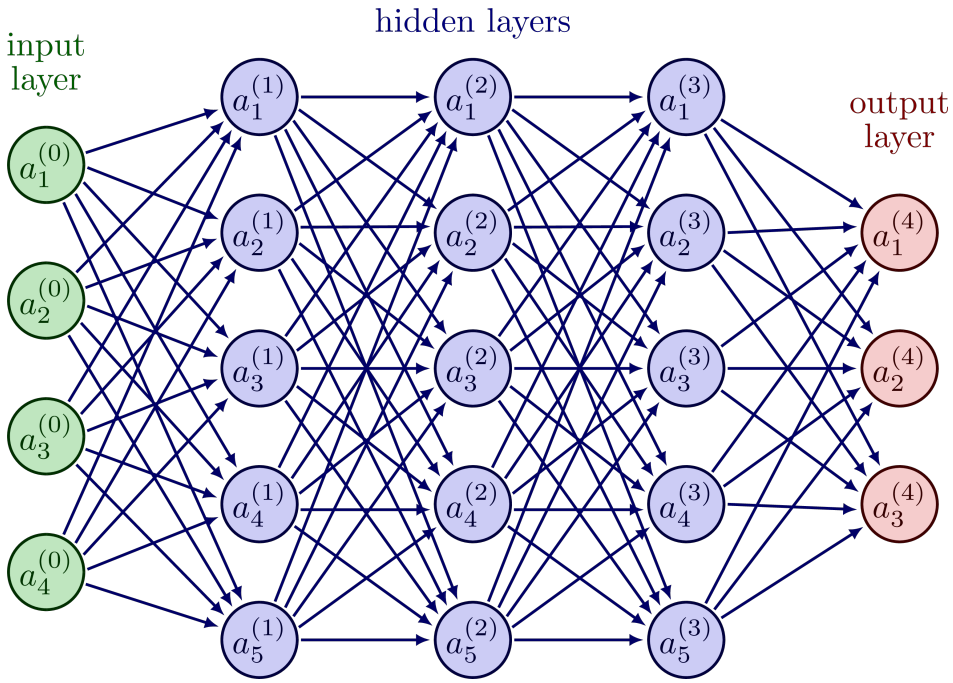


FIGURE 2: Typical multi-layer neural network architecture, illustrated by [42].

recognition [53], [54], telecommunications [55], multilingual processing [56], 3D object recognition [57], and handwritten recognition [15]. A defining property of ANNs is their ability to learn from data by adjusting connection weights, analogous to synaptic adaptation in biological systems [58]. This enables their use in classification, pattern recognition, and function approximation tasks.

Figure 2 presents a standard multi-layer ANN architecture. Each layer consists of interconnected nodes and includes a bias term, analogous to the intercept in linear models, which improves model flexibility and training efficiency [1]. For classification, input and output sizes correspond to features and classes, while function approximation typically uses fewer input/output nodes but more hidden units to capture complex relationships.

The evolution of ANNs is commonly described through a generational taxonomy introduced in [59]. The first generation, originating with the McCulloch-Pitts formal neuron [60] and later formalized as the perceptron by [61], comprises networks of threshold logic units that produce binary outputs. These networks compute Boolean functions and their computational power is limited to linearly separable problems, a constraint famously highlighted in [62].

The second generation overcame this limitation by introducing continuous activation functions (sigmoid [63], hyperbolic tangent [64], and more recently the rectified linear unit (ReLU) [65]) which enabled the application of gradient-based optimization through backpropagation [66]. These deep ANNs have achieved transformative results across numerous domains [1], from image recognition [67] and natural language processing [68] to scientific discovery [69]. However, their computational units (static neurons that apply a continuous activation function to a weighted sum of inputs) abstract away virtually all temporal dynamics present in biological neural computation.

The third generation, represented by SNNs, incorporates the temporal dimension of neural processing by communicating through discrete spike events rather than continuous values [59], [70]. [59] demonstrated that networks of spiking neurons with temporal coding are computationally more powerful than equivalent second-generation networks: a single spiking neuron with k inputs can compute functions that require a conventional sigmoidal network with an exponential (in k) number of hidden units. This theoretical advantage, combined with the promise of event-driven, energy-efficient computation on neuromorphic hardware, has driven the rapid growth of SNN research in recent years.

Despite their remarkable success, second-generation ANNs face fundamental scaling challenges that motivate the exploration of alternative paradigms. The computational cost of training deep networks has been growing at a rate that outpaces Moore's Law: training state-of-the-art models requires billions of floating-point operations per inference, with energy costs that raise significant sustainability concerns [71], [72]. Scaling laws for neural language models suggest that achieving further performance gains will require ever-larger models and datasets, creating an unsustainable trajectory in terms of both energy consumption and financial cost [72].

Furthermore, the fundamental computational paradigm of ANNs (dense, synchronous, multiply-accumulate (MAC) operations on continuous-valued activations) is inherently mis-

matched with the sparse, asynchronous, event-driven properties of sensory data in real-world scenarios. This mismatch is particularly acute in edge computing scenarios (autonomous vehicles, IoT devices, wearable health monitors) where power budgets are severely constrained and real-time processing is essential [29], [18]. Hardware analyses have shown that memory access, rather than arithmetic, dominates the energy budget of ANN accelerators, accounting for up to 60-90% of total energy consumption in practical deployments [19].

These limitations have fueled growing interest in SNNs as a complementary paradigm. When deployed on neuromorphic hardware, SNNs can replace energy-intensive MAC operations with accumulate (AC) operations, since the binary nature of spikes eliminates the need for multiplication [19]. On 45nm CMOS technology, a single MAC operation consumes approximately 4.6 pJ compared to 0.9 pJ for an AC operation [73]. However, realizing this theoretical energy advantage in practice requires that the average spike rate remain sufficiently sparse, where recent hardware-aware studies have shown that SNNs must maintain fewer than approximately 0.4 spikes per synapse per inference to outperform optimized ANN accelerators such as Eyeriss [19]. This finding suggests that hybrid ANN-SNN architectures, which leverage event-driven processing in high-sparsity layers and conventional parallel processing in dense layers, may represent the most promising path toward practical energy-efficient neural computation [74], [19].

IV. SNN MODELS

The choice of neuron model is the single most consequential design decision in constructing a spiking neural network. It determines what temporal phenomena the network can represent, how expensive each simulation step will be, whether gradient-based training is tractable, and ultimately whether the resulting system can be deployed on neuromorphic hardware [9], [75], [23]. Unlike artificial neurons, which collapse a weighted sum into a single nonlinear activation (Figure 3), spiking neurons carry internal state that evolves continuously in time and emits discrete events when that state crosses a threshold. This temporal state is both the source of an SNN's computational appeal (energy efficiency, event-driven processing, biological plausibility) and the source of its principal engineering difficulties.

The literature offers a remarkably broad spectrum of spiking neuron models, ranging from biophysically detailed conductance-based formulations that faithfully reproduce ion channel kinetics, to highly abstract point models designed for the sole purpose of being differentiable and GPU-friendly [75], [9], [76]. Rather than enumerate these models in isolation, we organize them along two axes that practitioners actually care about: biological expressivity (the range of firing patterns and temporal phenomena the model can reproduce) and computational cost (the per-step floating-point burden). Table 1 summarizes this trade-off space, and the subsections below walk through it from the biophysical extreme to the modern reduced models that underpin trainable deep SNNs. We close with a brief discussion of synaptic transmission and recent extensions that move beyond the point-neuron abstraction.

A. HODGKIN-HUXLEY (HH) MODEL

Any discussion of spiking neuron models must begin with the formalism that established the field. In their landmark 1952 study of the squid giant axon, Hodgkin and Huxley showed that the action potential could be reproduced from first principles by treating the membrane as an electrical circuit whose conductances depend on voltage-gated ion channels [33]. The resulting model expresses the membrane current balance as

$$C_m \frac{dv_m}{dt} = -I_{\text{ion}}(v_m, t) + I_{\text{syn}}(t), \tag{2}$$

where the ionic current decomposes into sodium, potassium, and leak components,

$$\begin{aligned} I_{\text{ion}} &= g_{\text{Na}}\, m^3 h\, (v_m - E_{\text{Na}}) \\ &\quad + g_{\text{K}}\, n^4\, (v_m - E_{\text{K}}) + g_L\, (v_m - E_L), \end{aligned} \tag{3}$$

and each gating variable $x \in \{m, h, n\} \subset [0, 1]$ obeys a first-order kinetic equation

$$\frac{dx}{dt} = \alpha_x(v_m)\,(1 - x) - \beta_x(v_m)\, x, \tag{4}$$

with empirically fitted voltage-dependent rate functions α_x, β_x [33]. The coupled nonlinear ODEs in Equations 2, 3 and 4 reproduce the action potential waveform with remarkable fidelity, predict the existence of refractory periods, and account for a wide range of cortical firing patterns [119], [111].

The price of this fidelity is steep. Each simulation step requires approximately 1200 floating-point operations (FLOPs) [75], the gating equations are stiff and demand small integration steps, and the model's sensitivity to parameter choices makes it ill-suited to backpropagation-based training [120], [119]. Consequently, HH neurons remain the gold standard for computational neuroscience and detailed biophysical simulations [110], [113], [112] but appear only rarely in deep SNN pipelines, where they have been used primarily in retina-inspired sensory front-ends [117], [118] and analog VLSI prototypes [115], [116]. The persistent gap between HH realism and the simplifying abstractions used in trainable SNNs motivates the entire family of reduced models that follows.

B. INTEGRATE-AND-FIRE VARIANTS

The integrate-and-fire (IF) family arose from the observation that, for many purposes, the precise shape of the action potential is irrelevant: what matters is when the spike occurs. Lapicque's 1907 model already captured this insight more than four decades before Hodgkin and Huxley's biophysical account [77]. Modern IF models retain Lapicque's threshold-and-reset abstraction but vary in how they treat subthreshold dynamics, adaptation, and spike onset. They form the

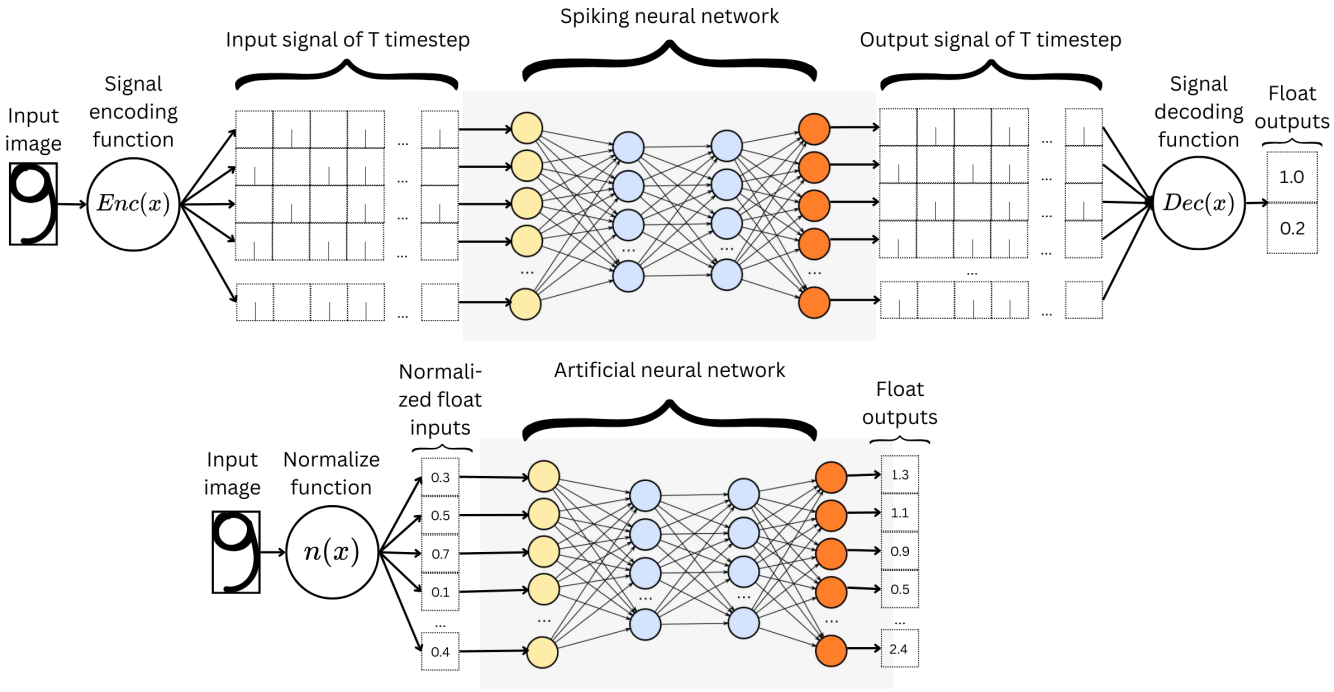


FIGURE 3: Comparison between artificial and spiking neurons. The ANN unit applies a static nonlinearity to a weighted sum of real-valued inputs, whereas the spiking unit maintains an internal membrane potential that integrates incoming spike trains over time and emits a binary event whenever a threshold is crossed.

computational backbone of nearly every contemporary deep SNN [23], [84].

1) Integrate-and-Fire (IF)

In its purest form, the IF neuron simply accumulates incoming current until a threshold is reached:

$$C_m \frac{dv_m}{dt} = I_{\text{syn}}(t), \quad \text{if } v_m \geq v_\theta : \; v_m \leftarrow v_{\text{reset}}. \tag{5}$$

Lacking a leak term, the IF neuron is a perfect integrator: any nonzero input will eventually drive it to fire, regardless of the input's temporal structure. This simplicity is both a virtue and a limitation. On the positive side, the IF model is essentially free to simulate (roughly 5 FLOPs per step as estimated by [75]), admits straightforward analytical treatment, and turns out to be the natural target of ANN-to-SNN conversion: when an artificial network's ReLU activations are interpreted as firing rates, the corresponding spiking network can be built from IF neurons with minimal accuracy loss [121], [122], [123]. On the negative side, the absence of leakage means the model has no intrinsic notion of temporal proximity (two spikes arriving milliseconds apart are treated identically to two spikes arriving seconds apart) which severely limits its ability to encode time.

2) Leaky Integrate-and-Fire (LIF)

Adding a passive leak conductance recovers a finite memory time and yields the workhorse of modern SNN research:

$$\tau_m \frac{dv_m}{dt} = -(v_m - E_L) + R_m I_{\text{syn}}(t), \quad \text{if } v_m \geq v_\theta : \; v_m \leftarrow v_{\text{reset}}, \tag{6}$$

where $\tau_m = R_m C_m$ is the membrane time constant. The leak term causes the membrane potential to decay exponentially toward E_L in the absence of input, so that the neuron acts as a low-pass filter on its input current with cutoff frequency $1/(2\pi\tau_m)$. For numerical implementation, the continuous equation is discretized by the forward-Euler scheme [124] to give the familiar update:

$$v_m[t+1] = \beta\, v_m[t] + (1-\beta)\, R_m\, I_{\text{syn}}[t] - S[t]\, v_\theta, \tag{7}$$

with decay factor $\beta = e^{-\Delta t/\tau_m}$ and binary spike output $S[t] \in \{0, 1\}$.

The LIF model occupies a sweet spot in the expressivity-cost trade-off. It is cheap enough to scale to networks of millions of neurons, smooth enough that surrogate-gradient methods can train it end-to-end [8], [84], [125], and supported by virtually every major neuromorphic platform, including Loihi [126], TrueNorth [127], and SpiNNaker [128]. Its limitations are the mirror image of its strengths: a single time constant and a hard reset cannot reproduce spike-frequency adaptation, bursting, bistability, or resonance, and the resulting firing-rate nonlinearity is essentially a noisy,

TABLE 1: Comparison of spiking neuron models in terms of electrophysiological expressivity and computational cost. FLOP counts refer to a single Euler step at typical simulation resolution.

Model	FLOPs	Characteristic Behaviors / Expressivity	Advantages	Disadvantages / Limitations	Utilization Examples
IF [77]	5†	Basic tonic spiking; no adaptation, bursting, bistability, or resonance; purely integrative dynamics with hard threshold-reset.	Minimal computational cost; analytically tractable; highly efficient for large-scale simulations and neuromorphic deployment; easy ANN-to-SNN conversion.	Biologically oversimplified; cannot reproduce adaptation, bursting, or subthreshold dynamics; poor modeling of temporal neural phenomena.	Supervised and unsupervised STDP learning [78], [79], neuromorphic VLSI design [80], reproduces cortical conditioning [81], spike coding benchmarks [14], [15].
LIF [82]	8††	Tonic spiking with passive membrane leakage; limited temporal filtering; no intrinsic bursting or resonance; approximates a ReLU-like firing-rate nonlinearity.	Excellent efficiency-accuracy trade-off; widely supported by neuromorphic hardware; stable for surrogate-gradient training; suitable for deep SNNs.	Cannot model spike-frequency adaptation, bursting, bistability, or oscillatory selectivity; weak biological fidelity.	ANN-to-SNN conversion [23], [83], spike-based backpropagation [84], neuromorphic VLSI design [85].
SRM [86]	15-40††	Kernel-based formulation of LIF dynamics; explicit modeling of post-synaptic potentials and refractoriness; supports arbitrary response kernels.	Closed-form expressions for spike response; well-suited to event-driven simulation and analytical work; flexible kernel design.	Computationally heavier than LIF when many input spikes are active; less natural for GPU batching; rarely used in deep SNNs.	Theoretical analysis of spiking codes [9], supervised learning rules such as SpikeProp [87], tempotron-style classifiers [88].
IFB [89]	13†	Switches between tonic spiking and burst firing; models low-threshold calcium dynamics; captures thalamic relay behavior.	Supports burst coding and saliency detection; richer temporal expressivity than LIF; useful for attention and event-driven processing.	Higher computational cost; burst dynamics may destabilize gradient-based training; limited support in neuromorphic toolchains.	Neural synchronization research [90], sensory information representation [91], periodic forcing studies [92].
RaF [93]	10†	Subthreshold oscillations and frequency-selective resonance; band-pass filtering of temporal input; phase-sensitive spiking.	Suitable for rhythm detection, temporal coding, and oscillatory networks; captures frequency preference absent in IF/LIF.	More complex dynamics; harder to tune and train; niche applicability in deep SNN pipelines.	Neuromorphic VLSI design [94], [95], neural oscillation research [96], gradient-based deep SNNs [97], [98].
QIF/EIF [9]	10-15††	Smooth nonlinear spike initiation; better approximation of cortical neuron f-I curves than LIF; no adaptation by default.	Continuously differentiable spike onset; analytically tractable mean-field reductions; bridge between LIF and AdEx.	Slightly more expensive than LIF; still lacks adaptation and bursting without extension; less hardware support.	Mean-field analysis [99], theoretical neuroscience studies [9].
Izhikevich [75]	13†	Wide repertoire: tonic spiking, phasic spiking, bursting, spike-frequency adaptation, bistability, resonance, and chaos.	Excellent expressivity-efficiency trade-off; reproduces most cortical neuron classes at low FLOP cost; scalable to large networks.	Parameter tuning nontrivial; reset-based spike generation is heuristic; less compatible with ANN-to-SNN conversion pipelines.	Temporal coding research [100], large-scale brain simulation [101], cognitive modeling [102], neuromorphic VLSI [103], reservoir computing [104], mean-field derivations [105].
AdEx [106]	25-35††^{a}	Spike-frequency adaptation; variable spike onset sharpness; bursting and irregular firing under appropriate parameters.	Smooth nonlinear dynamics; strong realism-efficiency balance; compatible with surrogate-gradient learning.	Higher cost than LIF; sensitive to hyperparameters; not fully biophysically grounded.	Neuromorphic VLSI [107], firing-regime analysis [108], mean-field derivations [99], conductance-based extensions [109].
HH [33]	1200†	Full repertoire of biophysical behaviors: adaptation, bursting, bistability, resonance, and chaotic dynamics; realistic spike waveforms.	Gold-standard biological realism; accurate membrane and channel dynamics; ideal for neuroscientific modeling.	Extremely high computational cost; stiff ODEs; unsuitable for large-scale SNN training and GPU batching.	Analysis and extensions [110], [111], [112], [113], [114], neuromorphic VLSI [115], [116], retina-like image coding [117], motion detection [118].

†: Measurements provided by [75]. ††: Independently approximated. ^{a}: Varies with exponential function implementation.

time-quantized version of a ReLU. For tasks where these phenomena are essential, richer models are required.

3) Quadratic and Exponential Integrate-and-Fire

A pair of refinements addresses the most glaring artifact of the LIF model: its sharp, discontinuous spike-initiation rule. The quadratic integrate-and-fire (QIF) and exponential integrate-and-fire (EIF) neurons replace the linear leak term with a nonlinearity that smoothly accelerates the membrane potential toward threshold once a critical voltage is crossed [9]. The EIF formulation,

$$\begin{aligned} \tau_m \frac{dv_m}{dt} &= -(v_m - E_L) \\ &\quad + \Delta_T\, e^{(v_m - V_T)/\Delta_T} + R_m\, I_{\text{syn}}(t), \end{aligned} \tag{8}$$

introduces a slope factor Δ_T controlling spike-onset sharpness and a soft threshold V_T. As $\Delta_T \to 0$, the EIF reduces to a LIF with a hard threshold; for moderate Δ_T, the model reproduces the smooth spike-initiation curves observed in cortical pyramidal cells more faithfully than LIF, while remaining cheap enough for large-scale simulation [9].

4) Adaptive Exponential Integrate-and-Fire (AdEx)

Building on EIF, the adaptive exponential integrate-and-fire (AdEx) model [106] introduces an auxiliary adaptation variable w that captures the slow currents responsible for spike-frequency adaptation:

$$\begin{aligned} C_m \frac{dv_m}{dt} &= -g_L(v_m - E_L) \\ &\quad + g_L\, \Delta_T\, e^{(v_m - V_T)/\Delta_T} - w + I_{\text{syn}}, \\ \tau_w \frac{dw}{dt} &= a\,(v_m - E_L) - w, \end{aligned} \tag{9}$$

with the discrete reset $v_m \leftarrow v_{\text{reset}}$ and $w \leftarrow w + b$ upon firing. The four free parameters (a, b, τ_w, Δ_T) together control adaptation strength, spike-triggered increment, adaptation timescale, and onset sharpness, allowing AdEx to reproduce tonic spiking, adaptation, initial bursting, and irregular firing within a single formalism [108]. AdEx has been implemented in analog VLSI as part of the BrainScaleS system [107], [129] and serves as the basis for several mean-field reductions used in large-scale brain modeling [99], [109].

5) Integrate-and-Fire-or-Burst (IFB)

Thalamic relay neurons exhibit a striking dual mode of operation: at depolarized potentials they fire tonic spike trains, while at hyperpolarized potentials they generate brief, high-frequency bursts mediated by low-threshold Ca^{2+} currents. Casti and colleagues [89] captured this behavior with the integrate-and-fire-or-burst (IFB) model, which augments LIF dynamics with a slow calcium current I_T:

$$\begin{aligned} C_m \frac{dv_m}{dt} &= -g_L(v_m - E_L) - I_T + I_{\text{syn}}, \\ I_T &= g_T\, m_\infty^2(v_m)\, h\,(v_m - E_T), \end{aligned} \tag{10}$$

where the instantaneous activation $m_\infty(v_m) = 1/\big(1 + e^{-(v_m+59)/6.2}\big)$ and the inactivation variable h follows $\tau_h\, dh/dt = h_\infty(v_m) - h$. IFB neurons have been used to model thalamic relay function [91], [90] and as building blocks for networks that exploit burst coding for saliency detection [92]. Their richer dynamics come at modest computational cost (∼13 FLOPs per step [75]) but introduce non-smooth transitions that complicate gradient-based training.

C. PHENOMENOLOGICAL MODELS

A separate branch of the literature pursues a different goal: rather than deriving spiking dynamics from membrane biophysics, phenomenological models start from a list of firing patterns that the model must reproduce and then construct the simplest dynamical system capable of doing so. The two canonical examples are the Izhikevich model and the resonate-and-fire model.

1) Izhikevich Model

Izhikevich's celebrated two-variable model [75] couples a quadratic membrane equation with a slow recovery variable:

$$\begin{aligned} C_m \frac{dv_m}{dt} &= k\,(v_m - E_L)(v_m - v_t) - u + I_{\text{syn}}, \\ \frac{du}{dt} &= a\,\big(b\,(v_m - E_L) - u\big), \end{aligned} \tag{11}$$

with the discrete reset $v_m \leftarrow c,\ u \leftarrow u{+}d$ upon $v_m \geq v_\theta$. By tuning the four parameters (a, b, c, d), this compact system reproduces substantially all known cortical firing patterns (tonic and phasic spiking, chattering, intrinsic bursting, low-threshold spiking, resonator and integrator behavior, rebound bursts, bistability, and even chaotic dynamics [75], [130]) at a cost of roughly 13 FLOPs per step. This combination of expressivity and efficiency has made the Izhikevich neuron the model of choice for large-scale brain simulation [101], [100], [102], neuromorphic VLSI design [103], and reservoir computing studies [104]. Its principal weakness is that the parameters lack direct biophysical interpretation, so fitting them to experimental data typically requires numerical optimization rather than direct measurement.

2) Resonate-and-Fire Model

Most spiking neurons in the IF family act as integrators: they accumulate input and fire in response to depolarization, irrespective of the input's temporal structure. A subset of biological neurons, however, behave as resonators, preferentially responding to inputs whose frequency content matches an intrinsic membrane oscillation frequency. Resonate-and-fire (RaF) model [93] captures this behavior with a damped harmonic oscillator:

$$\begin{aligned} \frac{dv_m}{dt} &= u, \\ \frac{du}{dt} &= -2\lambda\, u - \omega_0^2\,(v_m - E_L) + \frac{I_{\text{syn}}}{C_m}, \end{aligned} \tag{12}$$

where λ is the damping coefficient and ω_0 the natural oscillation frequency. The neuron fires when v_m crosses threshold and is reset to $v_m \leftarrow v_{\text{reset}},\ u \leftarrow u + d$. Because the response is sharply tuned around ω_0, RaF neurons act as biological band-pass filters and are well suited to tasks involving rhythm detection, phase coding, and oscillatory processing [96], [97], [98]. Recent analog CMOS implementations have demonstrated linear-time RaF dynamics for low-power audio processing [94], [95].

D. THE SPIKE RESPONSE MODEL

A complementary view of LIF dynamics [86], [9] abandons the differential-equation formulation altogether in favor of a kernel-based description of how a neuron responds to incoming spikes. The membrane potential at time t in the spike response model (SRM) is written as a sum of contributions from past presynaptic spikes and from the neuron's own previous firings:

$$v_m(t) = \eta(t{-}\hat{t}) + \sum_j w_j \sum_{t_j^{(f)}} \kappa\Big(t - t_j^{(f)}\Big) + v_{\text{rest}}, \tag{13}$$

where $\eta(\cdot)$ is a refractory kernel describing the after-effect of the last output spike at time $\hat{t}$, $\kappa(\cdot)$ is a post-synaptic potential kernel, and the inner sum runs over all presynaptic spikes from neuron j. By choosing the kernels appropriately, SRM can reproduce LIF dynamics exactly while making explicit the relationship between input timing and output response. This explicit form has made SRM the natural starting point for several seminal supervised learning rules, including SpikeProp [87] and the tempotron [88], and remains the preferred framework for analytical work on temporal coding [9].

E. DENDRITIC AND GLIAL EXTENSIONS

All of the models discussed so far collapse the entire dendritic tree, soma, and axon into a single state variable. Mounting evidence from electrophysiology suggests that this point-neuron abstraction discards computationally relevant structure [30], and a small but rapidly growing literature has begun to incorporate dendritic and glial mechanisms into trainable SNNs.

1) Dendritic models

Active dendrites can implement local nonlinear operations (coincidence detection, gain modulation, multi-timescale filtering) that effectively turn each neuron into a small two-

layer network [131]. Recent work has shown that incorporating simple dendritic compartments into LIF-based SNNs improves performance on sequential tasks while remaining compatible with surrogate-gradient training [132], [133]. Hardware implementations of dendritic SNNs have demonstrated low-power online learning through dendritic event processing [134], [135]. These results suggest that the gap between point neurons and biophysically detailed models can be narrowed without paying the full cost of HH-style simulation.

2) Astrocyte-modulated networks

A second strand of work moves beyond neurons entirely by modeling the astrocytes that surround synapses and modulate neurotransmitter dynamics on slow timescales. Spiking neuron-astrocyte networks have been used to study working memory, situational recall, and image recognition [136], [137], [138], [139], [140]. While still largely confined to small-scale demonstrations, these models point toward a richer computational vocabulary than the synapse-only abstraction inherited from classical neural network theory.

F. SYNAPTIC TRANSMISSION

Whichever neuron model is chosen, it must be paired with a description of how presynaptic spikes are converted into postsynaptic currents. For a neuron with synaptic weight matrix $W \in \mathbb{R}^{N_{\text{post}} \times N_{\text{pre}}}$, the synaptic input current is

$$I_{\text{syn}}(t) \;=\; W\, s_{\text{syn}}(t), \tag{14}$$

where $s_{\text{syn}}(t)$ is the vector of synaptic activations driven by presynaptic spike trains. Three formulations are in common use, ordered by biological fidelity [9], [32].

The simplest, the instantaneous or current-based model, treats each spike as a delta-function increment to the postsynaptic current. This is the default in most deep SNN implementations because it minimizes per-step overhead and integrates trivially with discrete-time simulation, at the cost of ignoring the finite time course of real synaptic events.

The single-exponential model captures the dominant feature of biological synapses (an exponential decay following each spike) through a single first-order ODE:

$$\frac{ds_{\text{syn}}}{dt} \;=\; -\frac{s_{\text{syn}}}{\tau_d} \;+\; \frac{1}{\tau_d} \sum_{t_k < t} \delta(t - t_k), \tag{15}$$

where the sum runs over all incoming presynaptic spike times t_k and τ_d is the synaptic decay time constant. This formulation is widely used in computational neuroscience and is implemented natively on most neuromorphic hardware.

The double-exponential (or alpha) model adds a finite rise time, producing the characteristic shape of an excitatory or inhibitory postsynaptic potential:

$$\begin{aligned} \frac{ds_{\text{syn}}}{dt} &= -\frac{s_{\text{syn}}}{\tau_d} + h, \\ \frac{dh}{dt} &= -\frac{h}{\tau_r} + \frac{1}{\tau_r \tau_d} \sum_{t_k < t} \delta(t - t_k), \end{aligned} \tag{16}$$

with rise and decay time constants τ_r and τ_d. When $\tau_r = \tau_d$, this reduces to the symmetric alpha function. The double-exponential form is preferred when realistic synaptic temporal profiles matter. For example, in models of dendritic integration [133] or in event-based audio processing where synaptic time constants shape the system's temporal receptive field [141].

V. SNN LEARNING MECHANISMS

If the choice of neuron model defines what an SNN can compute, the choice of training algorithm defines what it will compute in practice. This gives rise to the central tension in the field. The very property that makes spiking neurons appealing (the binary, event-driven nature of their output) is also the property that breaks the standard backpropagation machinery that has powered the deep learning revolution. The Heaviside step function used to generate spikes has a derivative that is zero almost everywhere and infinite at the threshold, blocking any naïve application of the chain rule [8], [23]. A second, less obvious difficulty is that information in an SNN is distributed across both layers and time steps, so credit assignment must propagate through a much larger computational graph than in a comparable ANN, creating memory and gradient-stability problems that grow with simulation length [142], [84].

Several broad strategies have emerged to navigate these obstacles, each embodying a different attitude toward the non-differentiability of the spike mechanism. The first, surrogate-gradient learning, accepts the spike function as it is and patches the chain rule with a smooth approximation during the backward pass. The second, biologically plausible learning, abandons backpropagation altogether in favor of local update rules (most prominently spike-timing-dependent plasticity) that more closely mirror the mechanisms observed in biological synapses. The third, ANN-to-SNN conversion, sidesteps the problem entirely by training a conventional ANN with standard backpropagation and then mapping its parameters into an equivalent spiking network. A fourth, increasingly popular strategy combines elements of the first three into hybrid pipelines that aim to inherit the strengths of each. Finally, we also briefly include other types of supervised online learning methodologies that focus more on spatial/temporal locality and energy efficiency than their STDP predecessors. Table 2 summarizes the landscape; the remainder of this section examines each paradigm in turn, emphasizing the technical choices that distinguish recent work from the classical formulations covered in earlier surveys [24], [23], [25].

A. SPIKE-BASED BACKPROPAGATION

1) The problem of credit assignment

Consider a generic discrete-time LIF neuron in layer ℓ with membrane potential $v_m^{(\ell)}[t]$, output spike $s^{(\ell)}[t] \in \{0, 1\}$, and

TABLE 2: Major learning paradigms for spiking neural networks. The table is organized by training strategy rather than by chronology, reflecting the practical decision a designer must make when choosing how to train an SNN.

Learning Mechanism	Type	Core Idea	Representative Methods
SpikeProp / temporal BP [87]	Supervised	Defines error gradients only at spike times by linearizing the membrane potential around threshold crossings.	SpikeProp [87], multi-layer extensions [143]
SuperSpike / SLAYER [144], [145]	Supervised	Differentiate through the membrane trajectory using a smooth pseudo-derivative, enabling learning even when the postsynaptic neuron is silent.	SuperSpike [144], SLAYER [145], gradient descent for SNNs [146]
Surrogate-gradient BPTT [8]	Supervised	Replaces the spike derivative with a smooth surrogate (sigmoid, fast sigmoid, arctan, triangular) during backpropagation through time.	Direct training [147], [141], [84], learnable surrogates [148], gradient rescaling [149], membrane time-constant learning [125]
ANN-to-SNN conversion [121]	Indirect / Supervised	Trains a ReLU ANN conventionally, then maps activations to IF firing rates via threshold and weight rebalancing.	Threshold balancing [123], [122], BN folding and theory [150], optimal conversion [151], [152], [153], quantization-aware [154], multi-bit spikes [155], residual and deep networks [156], [83], [157], Q-network conversion [158], transformer conversion [159], [160]
Three-factor eligibility-trace rules [161]	Online supervised	Decompose the BPTT gradient into a forward-propagated eligibility trace (encoding pre/post-synaptic activity history) and an instantaneous learning signal, achieving temporal locality with constant memory in T.	e-prop [162], OSTL [163], OTTT [164], S-TLLR [165], DECOLLE [166]
Fully local learning rules [167]	Online supervised	Achieve both temporal and spatial locality by replacing inter-layer error backpropagation with locally generated learning signals (e.g., fixed random target projections or neural synchronization), enabling $\mathcal{O}(Ln)$ memory and $\mathcal{O}(LCn)$ time.	TESS [167], ETLP [168], OSTTP [169], DRTP [170]
Classical STDP [171]	Unsupervised	Local Hebbian rule: pre-before-post strengthens the synapse, post-before-pre weakens it, with exponentially decaying timing windows.	Foundational studies [172], [173], [174], unsupervised digit recognition [79], [78], visual feature learning [175]
Anti-Hebbian / Mirrored STDP [176]	Unsupervised	Sign-flipped or symmetrically paired STDP rules used for decorrelation, inhibition, or autoencoder-style reconstruction.	Cerebellar models [176], mirrored STDP for autoencoders [177]
Probabilistic / Stable STDP [175]	Unsupervised	Stochastic weight updates whose magnitude depends on current weight, providing implicit homeostasis.	Visual feature acquisition [178], [179], optical flow learning [180]
Reward-modulated STDP [181]	Reinforcement	Gates STDP eligibility traces with a global scalar reward, solving the distal credit assignment problem in spiking networks. The earliest instance of the three-factor learning framework.	Dopamine-modulated plasticity [182], [181], three-factor framework [161]
ReSuMe / supervised STDP [183]	Supervised local	Combines STDP and anti-STDP windows driven by a teacher spike train to push output spikes toward target times.	ReSuMe [183], BP-STDP [184], SSTDP [185]
Intrinsic plasticity [186]	Homeostatic	Adapts neuron-level parameters (threshold, leak, gain) to maintain firing rates near a target distribution.	Excitability homeostasis [187]
Hybrid pretrain + finetune [188]	Mixed	Uses STDP for unsupervised feature learning followed by gradient-based supervised fine-tuning.	STDP pretraining + BP [188], [189], joint ANN-SNN training [190], knowledge distillation [191], [192]

update equations

$$\begin{aligned} v_m^{(\ell)}[t] &= \beta\, v_m^{(\ell)}[t-1] \\ &\quad + W^{(\ell)} s^{(\ell-1)}[t] - s^{(\ell)}[t-1]\, V_{\mathrm{th}}, \\ s^{(\ell)}[t] &= \Theta\big(v_m^{(\ell)}[t] - V_{\mathrm{th}}\big), \end{aligned} \tag{17}$$

where $\Theta(\cdot)$ is the Heaviside step function. Computing the gradient of any loss $\mathcal{L}$ defined over the spike train $s^{(L)}[1{:}T]$ with respect to the weights $W^{(\ell)}$ requires evaluating $\partial s^{(\ell)}[t]/\partial v_m^{(\ell)}[t] = \delta(v_m^{(\ell)}[t] - V_{\mathrm{th}})$, which is zero everywhere except at the threshold, where it is unbounded [8], [23]. Standard backpropagation cannot proceed through this point. Worse, even if it could, the recurrent dependence of $v_m^{(\ell)}[t]$ on $s^{(\ell)}[t-1]$ means that gradients must be unrolled across both depth and time, exposing the network to the same vanishing- and exploding-gradient pathologies that plague training of deep recurrent networks [142], [149].

2) Early temporal backpropagation methods

The first attempts to train multilayer SNNs by gradient descent predate the surrogate-gradient era by more than a decade. SpikeProp [87] circumvented the non-differentiability problem by linearizing the membrane potential around each spike time, treating the threshold crossing as an implicit function of the input weights. The resulting gradient is well-defined whenever a spike actually occurs but provides no learning signal for silent neurons, which is a substantial limitation, as deep SNNs typically operate in a regime where many neurons remain inactive on any given input. Subsequent work extended SpikeProp to multilayer architectures and to networks with multiple spikes per neuron [143], but the silent-neuron problem persisted as a fundamental obstacle.

A second wave of methods, exemplified by SuperSpike [144] and SLAYER [145], resolved this issue by differ-

entiating through the entire membrane trajectory rather than only at spike times. SuperSpike defined a smooth pseudo-derivative $\sigma'(v_m) = \left(1 + \beta\,|v_m - V_{\text{th}}|\right)^{-2}$ that is nonzero everywhere, providing a learning signal even for neurons that fail to fire. SLAYER took a complementary approach, using a probabilistic spike model to define a temporal credit assignment scheme that propagates gradients backward through axonal delays as well as synaptic weights, jointly optimizing both. [147] independently demonstrated that backpropagation through time (BPTT) with a piecewise-linear spike approximation could train multi-layer SNNs to competitive accuracy on MNIST and N-MNIST, providing the first clear evidence that deep SNNs were compatible with end-to-end gradient learning.

3) The surrogate-gradient framework

[8] subsequently consolidated these approaches into a unifying framework. The surrogate-gradient (SG) recipe is deceptively simple: we use the true Heaviside function for spike modeling in the forward pass; in the backward pass, we replace its derivative with a smooth pseudo-derivative $\tilde{\sigma}'(v_m)$ of the designer's choice. Common choices include:

- Sigmoid derivative [23]:

$$\tilde{\sigma}'_{\text{sig}}(v_m) \;=\; \frac{e^{-(v_m - V_{\text{th}})/\tau}}{\tau\left(1 + e^{-(v_m - V_{\text{th}})/\tau}\right)^2}, \tag{18}$$

- Fast sigmoid surrogate [8]:

$$\tilde{\sigma}'_{\text{fast}}(v_m) = 1/\left(1 + |v_m - V_{\text{th}}|\right)^2, \tag{19}$$

- Arctangent surrogate [84]:

$$\tilde{\sigma}'_{\text{atan}}(v_m) = \frac{\alpha}{2\left(1 + \left(\frac{\pi}{2}\alpha v_m\right)^2\right)} \quad \text{where } \alpha = 2, \tag{20}$$

- Piecewise-linear "box" approximations centered on threshold [84], [23], [15], [74].

Despite the inconsistency between the forward and backward passes (the spiking network being trained is not, in a strict mathematical sense, the same network whose loss is being optimized) the procedure works remarkably well in practice [8], and it now underlies essentially every state-of-the-art directly-trained SNN, including deep residual SNNs [84], [83], [156], spiking transformers [193], [194], and event-based vision networks [195], [196].

4) Choosing and learning the surrogate

The choice of surrogate function turns out to be more consequential than its ad hoc origins might suggest. Functions whose support is too narrow yield sparse gradients that fail to propagate through deep networks; functions whose support is too wide allow neurons that are far from threshold to receive spurious credit, biasing the optimization. Learnable surrogate gradient (LSG) framework [148] addresses this by treating the surrogate width as a trainable parameter, coupled to the membrane time constant so that the gradient distribution adapts to the dynamic range of each layer. A complementary line of work has focused on stabilizing gradient flow across deep stacks of spiking layers: [149] identified a vanishing-gradient pathology specific to LIF networks and proposed an adaptive rescaling scheme that preserves gradient norms across many timesteps and layers, while [125] demonstrated that making the membrane time constant itself a learnable parameter substantially improves both convergence speed and final accuracy.

A more recent thread examines the interaction between surrogate-gradient sparsity and adversarial robustness. Sparse gradients reduce the attack surface available to gradient-based adversaries, and several works have exploited this property to argue that SNNs offer an intrinsic robustness advantage over equivalent ANNs [197], [198], [199], [200], [201]. The trade-off, however, is that aggressive sparsification degrades clean-input accuracy, leading to a robustness-accuracy frontier that mirrors the well-studied tension in adversarial training of conventional networks [17].

5) Memory cost and online alternatives

A practical drawback of BPTT-based SNN training is its memory footprint: storing activations across all T time steps for backward computation makes training a network with $T = 100$ roughly $100\times$ more memory-intensive than training the corresponding ANN. This cost has motivated research on online and forward-mode alternatives that propagate eligibility traces forward in time rather than gradients backward, trading some accuracy for a constant-memory training procedure. While the original online learning algorithms for spiking networks fall outside the scope of the references collected here, the broader trend toward memory-efficient SNN training (through gradient checkpointing, truncated BPTT, and surrogate-trace methods) has become an active area of work [25], [202].

B. SPIKE-TIMING-DEPENDENT PLASTICITY (STDP)

Spike-timing-dependent plasticity (STDP) is a refinement of Hebbian learning [203] discovered in hippocampal cultures by [171] and subsequently documented in cortex, cerebellum, and thalamus [172], [174]. In classical STDP rule, the synaptic weight change Δw is expressed as a function of the time difference Δt between paired pre-synaptic and post-synaptic spikes:

$$\Delta w \;=\; \begin{cases} A_+ \exp(-\Delta t/\tau_+) & \text{if } \Delta t > 0, \\ -A_- \exp(\Delta t/\tau_-) & \text{if } \Delta t < 0, \end{cases} \tag{21}$$

where $\Delta t = t_{\text{post}} - t_{\text{pre}}$, $A_\pm$ and $\tau_\pm$ control the magnitude and time scale of potentiation and depression respectively. The pre-before-post pairing strengthens the synapse (long-term potentiation), reflecting the inference that the presynaptic spike contributed to driving the postsynaptic neuron; the reverse pairing weakens it (long-term depression), removing acausal connections. Practical implementations replace the explicit pairing with exponentially decaying eligibility traces

$x_{\text{pre}}(t)$ and $x_{\text{post}}(t)$ updated on each spike, yielding an entirely local online rule [180], [175].

Pure STDP suffers from a well-known stability problem: the Hebbian "rich-get-richer" dynamic causes weights to drift toward their bounds, producing either runaway potentiation or complete depression. Two homeostatic mechanisms are commonly used to control this. Multiplicative STDP [204], [173] scales the learning rate by the current weight magnitude, slowing potentiation as weights approach their upper bound. Triplet and rate-based extensions introduce a slower depression term that counteracts the imbalance accumulated by the classical pairwise rule. [79] demonstrated that unsupervised STDP could achieve 95% accuracy on MNIST without any teacher signal, relied on exactly this kind of homeostatic control combined with adaptive thresholds. Subsequent extensions have used STDP-trained convolutional layers as feature extractors for handwritten digit and object recognition [78], [205], [206], demonstrating that local learning can produce competitive features even on natural images, albeit at substantially lower accuracy than gradient-based methods.

1) Anti-Hebbian, mirrored, and probabilistic STDP variants

Several variants of the basic STDP rule capture biological observations or serve specific computational goals. Anti-Hebbian STDP [176], originally documented in the cerebellar cortex, reverses the sign of the timing-dependent update, producing decorrelating rather than associative learning. Mirrored STDP [177] pairs ordinary STDP on feedforward synapses with anti-STDP on feedback synapses to implement a biologically plausible form of autoencoder learning, in which the network learns to reconstruct its own inputs. Probabilistic STDP [178], [179] replaces the deterministic update rule with a stochastic one whose magnitude depends on the current weight, providing a softer form of homeostasis that has proved effective for visual feature learning. [180] demonstrated that a stable variant of STDP (S-STDP) could learn hierarchical features for optical flow estimation from event-camera data without any supervision, offering a rare example of local learning matching gradient-based methods on a real-world task.

2) Supervised local learning

A separate line of work has tried to graft supervision onto STDP without sacrificing locality. ReSuMe rule [183] combines an STDP window driven by target spikes with an anti-STDP window driven by actual output spikes, pushing the postsynaptic spike train toward a desired template. BP-STDP [184] approximates the backpropagation update with a sequence of STDP-like local rules, achieving competitive accuracy on small-scale benchmarks while remaining compatible with neuromorphic hardware implementation. SSTDP [185] formalizes a similar idea by deriving STDP-style updates from a supervised loss function, providing a more principled bridge between gradient descent and local plasticity. These methods occupy an interesting middle ground: they retain locality, but at the cost of requiring teacher spike trains that must be designed or generated by a separate supervisory mechanism.

3) Reward-modulated plasticity

Pure unsupervised STDP cannot solve tasks with delayed scalar feedback, because the relevant pre/post-synaptic events typically occur long before the reward arrives. Reward-modulated STDP (R-STDP) [181], [182] addresses this by storing each STDP update in an exponentially decaying eligibility trace $e_{ij}(t)$ and gating the actual weight change with a global reward signal $r(t)$ released by a simulated dopaminergic system:

$$\frac{de_{ij}}{dt} = -\frac{e_{ij}}{\tau_e} + \text{STDP}(t), \quad \frac{dw_{ij}}{dt} = r(t)\, e_{ij}(t). \tag{22}$$

The eligibility trace solves the distal reward problem by holding plasticity events in suspended animation until the reward arrives, at which point they are converted into actual weight changes. R-STDP has been used to train SNN controllers for robotic grasping [207], simple navigation tasks, and reinforcement learning benchmarks, though scaling it to the complexity of modern deep RL benchmarks remains an open challenge.

4) Intrinsic and structural plasticity

Synaptic plasticity is only one of the mechanisms by which biological neurons adapt. Intrinsic plasticity [186], [187] adjusts neuron-level parameters such as the firing threshold, leak conductance, or input gain to maintain a target firing-rate distribution, acting as a homeostatic regulator that prevents runaway activity or silent neurons. Including intrinsic plasticity alongside STDP substantially improves the stability of unsupervised SNNs and is an effective substitute for the explicit batch normalization used in gradient-trained models.

C. ANN-TO-SNN CONVERSION

The conversion paradigm starts from a different observation: the average firing rate of an integrate-and-fire neuron driven by a constant input is, to a very good approximation, a piecewise-linear function of that input that saturates at $1/\Delta t$ spikes per second. This function looks remarkably like a clipped ReLU, suggesting that any ANN built from ReLU activations should be convertible into an equivalent spiking network simply by replacing each ReLU with an IF neuron and rescaling weights appropriately. [121] were the first to make this idea concrete, and subsequent work by [123] and [150], [122] placed it on a firm theoretical footing. The appeal of conversion is that it inherits the entire toolbox of conventional deep learning (data augmentation, batch normalization, modern optimizers, residual architectures) and pays only a deployment-time cost for the spiking conversion, making it the dominant strategy for high-accuracy SNNs on large-scale image classification [156], [83], [157].

1) Threshold and weight balancing

The accuracy of a converted SNN depends critically on choosing each layer's firing threshold so that the IF neuron's effective firing-rate range matches the dynamic range of the corresponding ANN activations. [123] proposed model-based normalization sets the threshold $V_{\mathrm{th}}^{(\ell)}$ proportional to the maximum activation observed in layer ℓ over a calibration set, ensuring that no neuron saturates at the maximum firing rate. Data-based normalization replaces the worst-case maximum with a high percentile (typically the 99.9th) to suppress the influence of activation outliers, yielding more balanced firing rates across layers. [122] subsequently showed that the accuracy loss from decrementing the membrane potential by V_{th} can be mitigated by a soft-reset mechanism rather than resetting to zero on each spike, preserving the residual information that hard reset discards and substantially reducing conversion error. This soft-reset variant has since become the de facto standard in modern conversion pipelines [152], [151], [153], [157].

2) Batch normalization absorption

Because batch normalization (BN) layers compute mini-batch statistics that have no direct equivalent in event-driven inference, BN must be absorbed into adjacent linear layers before conversion. For a convolution with weight w_{ij} and bias b_i followed by a BN layer with parameters $(\gamma_i, \beta_i, \mu_i, \sigma_i^2)$, the absorbed parameters are

$$\tilde{w}_{ij} = \frac{\gamma_i\, w_{ij}}{\sqrt{\sigma_i^2 + \epsilon}}, \qquad \tilde{b}_i = \frac{\gamma_i\,(b_i - \mu_i)}{\sqrt{\sigma_i^2 + \epsilon}} + \beta_i, \tag{23}$$

which yields an equivalent linear transformation that the spiking network can implement directly [150], [122], [124]. While BN folding is conceptually straightforward, its interaction with threshold balancing turns out to be nontrivial: the absorbed weights have a different statistical distribution from the original convolutional weights, and naively applying max-based threshold normalization to the absorbed layer can produce thresholds that are either too aggressive or too conservative. Modern pipelines typically recompute activation statistics after BN absorption to avoid this pitfall.

3) Pooling and architectural compatibility

Max-pooling is similarly awkward in spiking networks because it has no natural time-domain analogue: which spike is the "maximum" over a pooling window? Two practical workarounds dominate the literature. Under temporal coding, the earliest spike within the pooling window is forwarded, on the grounds that earlier spikes encode larger ANN activations under time-to-first-spike encoding [122]. Under rate coding, the spike train with the highest count over a sliding window is selected, often approximated in hardware by a logical OR across time steps [156]. Average-pooling, by contrast, transfers trivially because it commutes with the linear update rule of IF neurons, and modern conversion pipelines strongly favor architectures built around average-pooling for this reason [122], [157]. Residual connections [67] require similar care: naive shortcuts can cause membrane potentials to accumulate without bound, motivating the spike-aware residual blocks introduced by [83] and [156].

4) Conversion latency bottleneck

The principal weakness of conversion-based SNNs has always been simulation latency. Because rate-coded inference requires averaging spike counts over a window long enough to suppress quantization noise, classical conversion methods needed hundreds or thousands of time steps to match ANN accuracy [122], [156], eliminating the energy advantage that motivated the spiking formulation in the first place. In the last few years, closing this latency gap has been one of the most active area of SNN researches.

Several complementary strategies have proven effective. Optimal conversion methods [151], [152], [153] formulate the conversion error as an explicit optimization objective and derive closed-form expressions for the threshold and initial membrane potential that minimize it, reducing the required latency from hundreds of steps to fewer than ten while preserving accuracy. Quantization-aware training [154], [208], [209] prepares the source ANN by clipping and quantizing its activations during training, so that the activation distribution is already matched to what an IF neuron can represent at low latency. Multi-bit spikes [155], [210] relax the strictly binary spike assumption by allowing each event to carry a small integer value, recovering accuracy at very short time windows at the cost of a slight increase in event payload size. Burst coding [157] exploits the IFB and Izhikevich models' burst-firing capability to transmit multiple effective bits per neuron per window. Finally, calibration-based methods [153], [211] initialize the membrane potentials of the converted network from data-driven statistics rather than from zero, eliminating an entire class of transient errors that previously dominated short-latency conversion.

5) Beyond convolutional networks

Until recently, conversion methods were essentially confined to convolutional architectures because the ReLU $\rightarrow$ IF mapping does not extend naturally to operations like softmax attention, layer normalization, and GELU activations that dominate modern transformers. Recent work has addressed this gap, where SpikeZIP-TF [160], [212] proposes a spike-equivalent transformer block in which every nonlinearity has a faithful spiking counterpart, achieving 83.82% top-1 accuracy on ImageNet and 93.79% on SST-2 without any retraining of the source transformer. [159] extend this idea to a fully training-free conversion pipeline for transformer-based SNNs, while several directly-trained spiking transformers [193], [213], [194], [214] approach the same goal from the opposite direction. Conversion has also been extended to deep reinforcement learning, where spiking Q-networks are derived from pretrained DQNs [158], [215] for energy-efficient inference on neuromorphic hardware [216].

D. HYBRID TRAINING PIPELINES

The taxonomy above is conceptually clean but rarely respected in practice. Many of the most successful SNNs published in the last three years combine two or more paradigms in a deliberate attempt to inherit the strengths of each, and this hybridization deserves a section of its own.

The earliest hybrid approach uses STDP for unsupervised pretraining followed by supervised gradient-based fine-tuning. [188] showed that STDP-pretrained convolutional layers provide a substantially better initialization for backpropagation-based SNN training than random weights, particularly for shallow networks where unsupervised features cover most of the representational capacity. Panda and Roy's regenerative learning framework [189] builds on this idea with an autoencoder-like reconstruction objective that improves feature quality without any labeled data.

A second hybrid strategy fuses ANN and SNN training in a single pipeline. Joint A-SNN [190] trains an artificial and a spiking network simultaneously, with the ANN serving as a teacher for the SNN through self-distillation while the SNN's discrete nature regularizes the ANN. The result is a pair of models that share weights but differ in their inference mode, allowing the same parameters to be deployed either as a high-accuracy ANN or as an energy-efficient SNN depending on the target hardware.

A third approach uses knowledge distillation to transfer information from a pretrained large model to a spiking student. SpikingBERT [192] and SpikeBERT [191] distill BERT-scale language models into spiking transformers using implicit differentiation and intermediate-layer matching, demonstrating that high-quality spiking language models can be obtained without the prohibitive cost of training them from scratch. Similar distillation strategies have been used to train spiking image classifiers from large convolutional or transformer teachers [190], [160], and the approach has become a de facto standard whenever a strong ANN teacher is available.

E. OTHER ONLINE AND EDGE-FRIENDLY LEARNING RULES

The discussion in Section V-A concluded with a practical observation: BPTT-based surrogate-gradient training has a memory footprint that scales as $\mathcal{O}(TLn)$, where T is the simulation length, L the depth, and n the average layer width. For edge devices and on-chip learning scenarios, this scaling is fatal. A network trained with $T = 100$ time steps consumes roughly $100\times$ the activation memory of an equivalent ANN, which is incompatible with the milliwatt power and sub-megabyte memory budgets of neuromorphic chips such as Loihi [126], BrainScaleS-2 [129], and ReckOn-class processors. This mismatch has motivated a distinct family of training algorithms whose central goal is not raw accuracy but tractability under deployment constraints. We refer to these as online and edge-friendly learning rules, and we organize them along two largely independent axes that have crystallized in the recent literature: temporal locality and spatial locality.

1) Two axes of locality

A learning rule is temporally local if computing the weight update at time step t does not require access to future activations or to past activations beyond a constant-sized state. Temporal locality eliminates the T factor from the memory cost and is what makes a rule compatible with real-time, streaming operation. A learning rule is spatially local if each layer can compute its own weight update without backpropagating an error signal across layers; spatial locality eliminates the $\mathcal{O}(Ln^2)$ cost of layer-wise error propagation and removes the need for the symmetric backward weight matrices that are biologically implausible and hardware-hostile. BPTT is neither temporally nor spatially local. Most online SNN algorithms proposed since 2019 achieve at least one of the two; only a handful achieve both, and doing so without sacrificing accuracy has emerged as one of the field's clearest open challenges [167], [165].

2) Eligibility traces and three-factor learning

Nearly all online SNN learning rules build on the framework of three-factor plasticity [161]. The synaptic weight update is decomposed into three multiplicative factors: a presynaptic activity term, a postsynaptic activity term, and a top-down learning signal. The first two factors are accumulated locally at the synapse into an eligibility trace $e_{ij}[t]$ that records which connections recently participated in producing output, and the third factor (a reward, an error, or a target projection) gates how those eligible synapses are actually modified. In its most general form, the eligibility trace obeys the recurrence

$$e_{ij}^{(\ell)}[t] \;=\; \beta\, e_{ij}^{(\ell)}[t-1] \;+\; f\big(o_i^{(\ell)}[t]\big)\, g\big(o_j^{(\ell-1)}[t]\big), \qquad (24)$$

with decay $\beta \in [0,1]$ and elementwise functions f, g defined on post- and presynaptic spikes respectively. The actual weight update is then

$$\Delta W_{ij}^{(\ell)} \;=\; \sum_t m_i^{(\ell)}[t]\, e_{ij}^{(\ell)}[t], \qquad (25)$$

where $m_i^{(\ell)}[t]$ is the top-down learning signal. The various online algorithms differ almost entirely in (i) how the eligibility trace is computed and (ii) where the learning signal $m^{(\ell)}[t]$ comes from.

3) Temporally local rules with global error signals

The first wave of online SNN algorithms, exemplified by e-prop [162] and OSTL [163], introduced full eligibility traces for recurrent spiking networks but still relied on backpropagation across layers to generate the learning signal. e-prop, in particular, derives an approximation of BPTT in which the gradient is decomposed into a forward-propagated eligibility trace and an instantaneous learning signal computed from the loss; the result is mathematically tractable, biologically motivated, and constant-memory in T, but the per-step cost

remains $\mathcal{O}(Ln^2)$ because of the inter-layer error propagation, and the eligibility traces themselves consume $\mathcal{O}(Ln^2)$ memory because they are indexed by synapse rather than by neuron. e-prop has nevertheless become the canonical online training algorithm for neuromorphic hardware: a variant has been deployed on the SpiNNaker-2 platform [217], and the ReckOn [218] task-agnostic on-chip learning processor implements an e-prop-derived rule whose memory overhead is just 0.8% of the equivalent inference-only design.

OTTT (Online Training Through Time) [164] and S-TLLR [165] subsequently reduced the memory cost from $\mathcal{O}(Ln^2)$ to $\mathcal{O}(Ln)$ by restricting the eligibility formulation to instantaneous traces: by setting $\beta = 0$ in Equation 24 and tracking pre/postsynaptic activity in two separate $\mathcal{O}(n)$-sized buffers rather than as an $\mathcal{O}(n^2)$ synaptic tensor. S-TLLR additionally draws on STDP by incorporating both causal (pre-before-post) and non-causal (post-before-pre) traces, and demonstrates accuracy within $\pm 2\%$ of BPTT on event-based vision and audio benchmarks while reducing memory by $5 - 50\times$ and MAC operations by $1.3 - 6.6\times$. Both methods, however, remain spatially non-local: they still require an inter-layer backward pass to produce the learning signal.

4) Spatially local rules via target projection

A parallel line of work attacks the spatial credit assignment problem directly. Direct random target projection (DRTP) [170] replaces the chain-rule backward pass with a fixed random matrix that projects the target labels (rather than the propagated error) directly to each layer. Each layer then computes its own learning signal from these projected targets without any communication with downstream layers, reducing the per-step cost from $\mathcal{O}(Ln^2)$ to $\mathcal{O}(LCn)$ where $C \ll n$ is the number of classes. The price is a small accuracy gap relative to backpropagation, which has historically prevented DRTP-style methods from scaling to deep architectures.

ETLP [168], OSTTP [169], and DECOLLE [166] combine DRTP-style target projection with eligibility traces to achieve both temporal and spatial locality. ETLP, in particular, was designed explicitly for FPGA implementation, uses a projected-label third factor, and demonstrates accuracy comparable to e-prop on auditory and visual spike datasets while supporting fully on-chip training. DECOLLE pursues a complementary route by attaching an auxiliary classifier to every layer and training each one independently with a local loss derived from random target projections, thereby avoiding error propagation across layers entirely. Despite these successes, all of these fully-local methods have struggled to scale beyond two- or three-layer networks: their per-layer learning signals lack the cross-layer coordination that backpropagation provides, and the resulting accuracy degrades as depth increases.

5) Toward fully local rules with competitive accuracy

The most recent generation of online SNN training rules attempts to close this depth gap. The TESS rule [167] combines S-TLLR-style instantaneous eligibility traces (achieving temporal locality with $\mathcal{O}(Ln)$ memory) with a layer-wise learning signal generated from fixed binary square-wave projection matrices inspired by neural synchronization (achieving spatial locality with $\mathcal{O}(LCn)$ time). The learning signal for layer ℓ at time t takes the form

$$m^{(\ell)}[t] \;=\; B^{(\ell)\top}\Big(f\big(B^{(\ell)}o^{(\ell)}[t]\big) - y^*\Big), \qquad (26)$$

where $B^{(\ell)}$ is a fixed binary matrix whose columns are square-wave basis vectors that distribute task-relevant information across the layer's neurons, and $f(\cdot)$ is a softmax (for classification) or identity (for regression). The square-wave structure makes $B^{(\ell)}$ both quasi-orthogonal — minimizing interference between class projections — and hardware-trivial to store and multiply. On VGG-9 architectures trained on DVS Gesture, CIFAR10, and CIFAR100, TESS matches BPTT accuracy while using 3-10$\times$ less memory and 205-661$\times$ fewer MAC operations for learning-signal generation; on CIFAR10-DVS it lags BPTT by only 1.4 accuracy points. These numbers are particularly significant because they refute the long-standing belief that fully local rules cannot scale to deep convolutional architectures.

TABLE 3: Memory and time complexity of online and locality-constrained learning rules for SNNs. L is the number of layers, n the average neurons per layer, T the simulation length, and C the number of targets. Temporally local means the cost is independent of T; spatially local means no inter-layer error propagation is required.

Method	Memory	Time / step	Temp.	Spat.
BPTT [23]	$\mathcal{O}(TLn)$	$\mathcal{O}(TLn^2)$	✗	✗
e-prop [162]	$\mathcal{O}(Ln^2)$	$\mathcal{O}(Ln^2)$	✓	✗
OSTL [163]	$\mathcal{O}(Ln^2)$	$\mathcal{O}(Ln^2)$	✓	✗
DECOLLE [166]	$\mathcal{O}(Ln)$	$\mathcal{O}(LCn)$	✓	✓
ETLP [168]	$\mathcal{O}(Ln^2)$	$\mathcal{O}(LCn)$	✓	✓
OSTTP [169]	$\mathcal{O}(Ln^2)$	$\mathcal{O}(LCn)$	✓	✓
OTTT [164]	$\mathcal{O}(Ln)$	$\mathcal{O}(Ln^2)$	✓	✗
S-TLLR [165]	$\mathcal{O}(Ln)$	$\mathcal{O}(Ln^2)$	✓	✗
TESS [167]	$\mathcal{O}(Ln)$	$\mathcal{O}(LCn)$	✓	✓

The complexity landscape of these methods is summarized in Table 3, which extends the comparison framework of [167]. The pattern that emerges is straightforward: methods that fix one axis of locality typically pay for it on the other, and only the most recent fully local rules achieve linear memory and linear-in-C time complexity simultaneously. Whether this pattern persists as the field tackles larger datasets and deeper architectures is one of the most pressing open questions in SNN training research.

6) Hardware implications

The motivation for this entire research thread is hardware deployment, and the architectural implications are concrete. A learning rule with $\mathcal{O}(Ln^2)$ memory cost cannot fit on a chip whose synaptic crossbar is the only large memory available; rules like e-prop, OSTL, ETLP, and OSTTP therefore require external DRAM during training, which dominates the overall

energy budget. Rules with $\mathcal{O}(Ln)$ memory cost (OTTT, S-TLLR, DECOLLE, and TESS) fit comfortably alongside the inference state on most neuromorphic processors. Among these, the rules that additionally achieve $\mathcal{O}(LCn)$ time cost (DECOLLE, TESS) avoid the need for inter-core spike routing of error signals, which on chips like Loihi has been shown to consume more energy than the actual learning computation [126]. The convergence of low memory, low inter-core communication, and competitive accuracy represented by TESS and its successors suggests that on-device SNN training at the milliwatt scale is no longer a distant goal but a near-term engineering target.

VI. SPIKE ENCODING

SNNs communicate using discrete spikes and spike trains; therefore, transforming real-valued input data into spike-based representations is a crucial step in constructing effective SNN models. While the exact biological mechanisms of neural information encoding remain an open question in neuroscience, three primary computational encoding strategies have been widely adopted in the literature: *direct encoding*, *rate encoding* and *temporal encoding* [219]. Additionally, certain neuromorphic sensors such as Dynamic Vision Sensors (DVS) [220] inherently output spike events directly, eliminating the need for artificial encoding.

A. DIRECT ENCODING

In the context of SNN, direct encoding [221] refers to an input representation method where continuous-valued data (such as image pixels) are injected directly into the network without conversion into spike trains.

Unlike biologically inspired encodings such as rate or temporal coding, direct coding bypasses stochastic spike generation. The input value is directly applied as a constant current or voltage to the membrane potential of the input neurons. In this scheme, the network itself determines the spiking behavior through its internal dynamics, rather than through an external encoding mechanism:

$$I_i(t) = x_i, \quad \forall t \in [0, T]$$

where $x_i \in [0, 1]$ is the normalized input signal (e.g., pixel intensity), and $I_i(t)$ is the input current applied to neuron i across all simulation timesteps. For a LIF neuron, the membrane potential $V_i(t)$ evolves as:

$$V_i(t+1) = V_i(t) + I_i(t) - S_i(t) \cdot V_{\text{reset}},$$

where $S_i(t) = \Theta(V_i(t) - V_{\text{th}})$ is the spike output, the Heaviside function $\Theta(\cdot)$, the firing threshold V_{th}, and the reset potential V_{reset}.

The existence of direct code remove randomness introduced through spike sampling, eliminates the need for Poisson or rate-based spike generation and facilitates gradient-based optimization in surrogate gradient SNNs. However, direct coding is less biologically plausible and energy efficient compared to rate or temporal coding [221] as it omits explicit temporal spike representations in the input layer.

B. RATE CODING

Rate coding is the oldest and most widely adopted neural coding hypothesis, tracing back to early recordings of sensory nerve fibers [222], [223]. Under this scheme, information is carried by the mean firing frequency of a neuron over a given observation window, rather than by the precise timing of individual spikes [219], [224]. For a spike train containing n spikes within a window of duration T, the firing rate is simply:

$$r = \frac{n}{T} \tag{27}$$

This quantity serves as a direct analogue to the continuous activation value in conventional ANNs, linking the SNN's spiking behavior to the frequency-current (f-I) response curve of the neuron model [82]. Rate codes can be further categorize into count rate codes (spike count of a single neuron), density rate codes (spike density estimated over a continuous time kernel), and population rate codes (firing statistics aggregated across a group of neurons) [219].

In practice, rate-coded inputs are most commonly generated through a Poisson process, which provides a stochastic mapping from real-valued data to spike trains [22], [123]. Given an analog input value mapped to a rate r, a spike at each discrete timestep Δt is emitted according to:

$$s[t] = \begin{cases} 1, & \text{if } \xi < r \cdot \Delta t \\ 0, & \text{otherwise} \end{cases} \tag{28}$$

where ξ is typically drawn from a uniform distribution $\xi \sim \mathcal{U}(0, 1)$ [22], though some implementations use a standard normal variate $\xi \sim \mathcal{N}(0, 1)$ instead [210], [124]. For image inputs, each pixel intensity is linearly scaled to a firing rate, producing an independent Poisson spike train per pixel per timestep [156], [122].

While Poisson rate coding has been the default choice in many foundational SNN works (particularly for ANN-to-SNN conversion pipelines [123], [156], [122]) it exhibits several practical limitations. First, faithfully representing an analog value through spike statistics requires a sufficiently long time window, often hundreds to thousands of timesteps, to suppress stochastic variance, which directly increases inference latency and energy cost [157], [209]. Second, the high volume of spikes generated under rate coding can negate the energy advantages that event-driven neuromorphic hardware is designed to exploit [225], [219]. Recent work on rate-synchrony encodings further demonstrates that controlling the temporal arrangement of rate-coded spikes-rather than relying on purely random Poisson generation can substantially improve adversarial resilience without sacrificing clean accuracy [197].

C. TEMPORAL ENCODING

In contrast to rate-based methods, temporal encoding [219] relies on the precise timing of individual spikes to convey information. In this scheme, more salient or stronger inputs are represented by earlier spike occurrences, while weaker

stimuli produce later spikes. Because information is carried by spike timing rather than frequency, temporal encoding often results in highly sparse spike patterns, potentially reducing energy consumption and computation.

1) Time-to-First-Spike

Time-to-First-Spike (TTFS) encoding [219], also known as latency coding in some literature [124], represents input intensity using the latency of the first emitted spike within a given encoding window of T discrete timesteps. Stronger stimuli are encoded with earlier spikes, whereas weaker stimuli generate later spikes. Each neuron fires at most one spike within the window $[0, T-1]$, leading to highly sparse and temporally precise spike representations.

For an input intensity $x \in [0,1]$, the spike latency $t_f(x)$ is defined according to the chosen encoding function. Two common formulations are used [124]:

- *Linear encoding:* In the linear case, the spike time decreases linearly with the input intensity:

$$t_f(x) = (T-1)(1-x) \tag{29}$$

  where T denotes the maximum (latest) firing time. Larger input values produce earlier spikes (smaller t_f).
- *Logarithmic encoding:* For a nonlinear mapping that emphasizes sensitivity to weaker inputs, a logarithmic version can be used:

$$t_f(x) = (T-1) - \ln(\alpha x + 1) \tag{30}$$

  where α is a scaling parameter determined by enforcing $t_f(1) = T-1$, that is:

$$\alpha = e^{T-1} - 1 \tag{31}$$

After computing $t_f(x)$, spike sequences $s(t)$ are generated such that a spike occurs exactly once at the firing time t_f:

$$s(t) = \begin{cases} 1, & \text{if } t = t_f(x) \\ 0, & \text{otherwise} \end{cases} \tag{32}$$

This encoding mechanism allows transforming continuous-valued inputs x (e.g., normalized pixel intensities) into temporally distributed binary spike trains suitable for spiking neural networks. TTFS encoding is efficient, interpretable, and compatible with both hardware and biologically inspired temporal learning rules.

2) Weighted Phase Encoding

Weighted phase encoding (WPE) proposed by [210], represents input intensities using a binary phase-based spike pattern. Unlike rate-based or latency-based schemes, WPE decomposes the normalized input $x \in [0, 1-2^{-K}]$ into a binary expansion across K discrete phases, each associated with a specific spike weight. This method allows each spike to carry a distinct amount of information according to its binary significance, leading to high representational efficiency.

Let K denote the total number of encoding phases, which typically matches the simulation time window in the spiking neural network. At each phase $t \in \{1, 2, \ldots, K\}$, the spike has an associated binary weight:

$$\omega(t) = 2^{-t} \tag{33}$$

The input x is iteratively decomposed according to these weights to determine the spike occurrence $s(t) \in \{0,1\}$:

$$s(t) = \begin{cases} 1, & \text{if } x_t \geq \omega(t) \\ 0, & \text{otherwise} \end{cases}, \quad x_{t+1} = x_t - \omega(t) \cdot s(t) \tag{34}$$

where $x_1 = x$. Each phase thus contributes to the total encoded value:

$$x = \sum_{t=1}^{K} \omega(t) \cdot s(t) \tag{35}$$

This binary-weighted encoding produces spike trains where earlier phases (larger $\omega(t)$) represent more significant bits of information, analogous to binary number representation.

3) Bit Planes Encoding

Bit planes encoding (BPE), recently explored by [15], [14], was inspired by traditional serial communication to efficiently convert spatial information from images into SNN spike signals with exactly 8 timestep. The process of converting a multi-channel image into its bit planes involves isolating each bit of the binary representation of the pixel values for each color channel. For an n_{bit} image, let $I_C(x,y)$ denote the intensity of a pixel at position (x,y) for a given color channel C, pixel intensity can be expressed in terms of its binary representation as:

$$I_C(x,y) = \sum_{k=0}^{n_{bit}} B_{k,C}(x,y) \cdot 2^k \tag{36}$$

where $B_{k,C}(x,y) \in \{0,1\}$ is the k-th bit of the binary representation of $I(x,y)$, and k represents the bit-plane index, with $k = 0$ corresponding to the least significant bit and $k = n_{bit}$ corresponding to the most significant bit.

To isolate a specific bit-plane, the k-th bit of the pixel can be extracted using the following equation:

$$B_{k,C}(x,y) = \left\lfloor \frac{I_C(x,y)}{2^k} \right\rfloor \mod 2. \tag{37}$$

VII. APPLICATIONS OF SNN IN COMPUTER VISION

A. WHAT IS COMPUTER VISION?

Computer vision (CV) is a field of study concerned with enabling machines to interpret and understand visual data [1]. Formally, it seeks to learn a mapping $f_\theta : \mathbb{R}^{C\times H\times W} \to \mathcal{Y}$, where $\mathbf{X} \in \mathbb{R}^{C\times H\times W}$ denotes an input image and $\mathcal{Y}$ represents the task-dependent output space. Unlike low-dimensional vector data, images exhibit strong spatial structure, where both pixel values and their spatial relationships are essential for extracting meaningful information.

CV encompasses several fundamental tasks, which differ primarily in the structure and dimensionality of their outputs, some well known problems include (but not limited to) [1]:

1) Image Classification

Image classification [226] assigns a semantic label to an entire image $y = f_\theta(\mathbf{X})$, $y \in \{1, \ldots, K\}$, where K is the number of predefined classes. The objective is to learn discriminative feature representations that capture semantic content while being invariant to irrelevant variations such as translation, illumination, and noise.

2) Object Detection

Object detection [227] extends classification by simultaneously identifying and localizing multiple objects within an image. The output consists of a set of object instances $\mathcal{O} = \{(c_i, \mathbf{b}_i)\}_{i=1}^{N}$, where c_i denotes the object class and $\mathbf{b}_i = (x_i, y_i, w_i, h_i)$ represents the bounding box coordinates. This task requires both semantic understanding and spatial localization.

3) Semantic Segmentation

Semantic segmentation [228] performs dense prediction by assigning a class label to every pixel $\mathbf{Y} = f_\theta(\mathbf{X})$, $\mathbf{Y} \in \{1, \ldots, K\}^{H \times W}$. This task requires precise spatial reasoning and preservation of fine-grained structural information.

4) Synthetic Image Generation

Synthetic image generation [229] aims to produce realistic images from latent representations or conditional inputs $\mathbf{X}_{\text{gen}} = f_\theta(\mathbf{z})$, where $\mathbf{z} \in \mathbb{R}^d$ is a latent variable sampled from a prior distribution. This task requires learning the underlying distribution of natural images $\mathbf{X}_{\text{gen}} \sim p_\theta(\mathbf{X})$. Such models must capture both local texture patterns and global structural consistency.

5) Motion Estimation

Motion estimation [230] involves predicting pixel-wise motion between consecutive frames. Given two frames $\mathbf{X}_t$ and $\mathbf{X}_{t+1}$, the objective is to estimate the motion field: $\mathbf{V} = f_\theta(\mathbf{X}_t, \mathbf{X}_{t+1})$, where $\mathbf{V} \in \mathbb{R}^{2 \times H \times W}$ represents horizontal and vertical motion components. This task requires modeling temporal correspondence and spatial continuity.

B. LIMITATIONS OF FULLY CONNECTED NETWORKS IN CV PROBLEMS

When working with CV related problems, a straightforward (and somewhat outdated) approach is to apply a fully connected network [1] to a flattened image $\mathbf{x} \in \mathbb{R}^D$, $D = C \times H \times W$ where a linear layer computes $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$. However, this formulation introduces several fundamental limitations.

First, the number of parameters grows rapidly with image resolution $\text{Params} = M \times C \times H \times W$, which becomes computationally prohibitive for high-resolution images.

Second, flattening destroys spatial structure. The original spatial relationships $\mathbf{X}_{i,j}$ and $\mathbf{X}_{i+1,j}$ are no longer explicitly represented, forcing the network to learn spatial dependencies inefficiently.

Third, fully connected layers lack spatial inductive bias. A translated input $\mathbf{X}' = \mathcal{T}_\Delta(\mathbf{X})$ does not produce a correspondingly translated output $f_\theta(\mathbf{X}') \neq \mathcal{T}_\Delta(f_\theta(\mathbf{X}))$ and this reduces generalization efficiency.

C. CONVOLUTION AS FEATURE EXTRACTION METHOD IN CV PROBLEMS

To address limitations of fully connected networks in CV tasks, convolution introduces structured connectivity and weight sharing [67]. The convolutional layer is the fundamental building block of vision-based ANNs. Unlike fully connected layers, convolutional layers exploit spatial locality and weight sharing to reduce the number of parameters and improve generalization. Given an input tensor $\mathbf{X} \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ and a convolutional kernel $\mathbf{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times K_h \times K_w}$, the output feature map $\mathbf{Y} \in \mathbb{R}^{C_{\text{out}} \times H' \times W'}$ is computed as:

$$\mathbf{Y}_{c,i,j} = \sum_{c'=1}^{C_{\text{in}}} \sum_{u=1}^{K_h} \sum_{v=1}^{K_w} \mathbf{W}_{c,c',u,v} \cdot \mathbf{X}_{c',\, i+u,\, j+v} + b_c, \quad (38)$$

where b_c is a learnable bias term. This operation can be interpreted as a sliding dot product between the kernel and local spatial regions of the input. Each kernel learns to detect specific visual patterns such as edges, textures, or object parts. The convolution operation is translation-equivariant:

$$\text{Conv}(\mathcal{T}_\Delta(\mathbf{X})) = \mathcal{T}_\Delta(\text{Conv}(\mathbf{X})), \quad (39)$$

where $\mathcal{T}_\Delta$ denotes a spatial translation operator, making this property essential for recognizing objects regardless of their position.

D. CONVOLUTION AND THEIR VARIANTS IN SNN

Similar to ANN, convolution is a fundamental operation in SNNs [156], [84], enabling hierarchical feature extraction from structured sensory inputs such as images, videos, and event streams. Early spiking convolutional models primarily reused convolutional operators developed for ANNs, applying them independently at each timestep while relying on membrane dynamics to integrate temporal information. This approach preserves the spatial feature extraction capabilities of convolution while enabling temporal processing through spike-based integration.

However, directly adopting ANN convolution does not fully exploit the unique computational properties of SNNs. In particular, spike-based computation introduces several distinct characteristics, including event-driven execution, temporal dynamics, sparse activation patterns, and compatibility with neuromorphic hardware constraints. These properties motivate the development of specialized convolution variants that extend or modify conventional convolution to better align with spike-based information processing.

Table 4 summarizes the major convolution variants in SNNs, highlighting their core design principles, mathematical characteristics, and primary advantages. This taxonomy provides a structured overview of how convolution has

TABLE 4: Summary of convolution variants in SNN.

Category	Core Idea	Primary Advantage
ANN convolution reutilization in SNN [156], [84]	Directly applies standard convolution to spike tensors at each timestep, with membrane integration accumulating temporal responses.	Preserves spatial feature extraction and enables reuse of ANN architectures and pretrained weights.
Event-driven convolution [27]	Tracks spike event occurrences and computes convolution based on event-triggered parameter counting.	Avoids redundant computation and improves efficiency for asynchronous spike streams.
Spike sparse convolution [231]	Restricts convolution to spatial locations containing spikes using sparse rulebooks and selector functions.	Reduces computational complexity and energy consumption by exploiting spike sparsity.
Spatio-temporal convolution [232]	Extends convolution kernels into the temporal domain with filtering and gating mechanisms.	Captures temporal dependencies and improves dynamic feature extraction.
Depthwise separable spiking convolution [233]	Factorizes convolution into temporal depthwise filtering and channel-wise projection.	Reduces parameter count and computational cost while maintaining representational power.
Pointwise spike convolution [234]	Uses 1×1 convolution to efficiently mix channel information in spike-based representations.	Efficient channel interaction with minimal computational overhead.
Frequency-aware spiking convolution [234]	Combines convolution with wavelet-based spectral transforms for frequency-domain spike processing.	Improves temporal-frequency feature representation and signal fidelity.
Binary stochastic convolution [235]	Constrains convolution kernels to binary values and trains them using probabilistic STDP.	Reduces memory usage and enables efficient neuromorphic hardware implementation.

evolved from direct ANN reutilization to specialized spike-native operations.

1) Reutilization of ANN convolution in SNN

If ANN convolution produces continuous feature maps $\mathbf{Y} = \mathrm{Conv}(\mathbf{X}, \mathbf{W})$, then in SNNs directly adopt ANN-like convolution [156], [84], the input is a spike tensor $\mathbf{S}$, where signal $\mathbf{S}(t)$ at timestep $t \in T$ is defined as $\mathbf{S}(t) \in \{0,1\}^{C\times H\times W}$, with the synaptic current $\mathbf{I}(t)$ computed as:

$$\mathbf{I}(t) = \mathrm{Conv}(\mathbf{S}(t), \mathbf{W}) \tag{40}$$

and the membrane potential $\mathbf{U}(t+1)$ evolves as:

$$\mathbf{U}(t+1) = \lambda \mathbf{U}(t) + \mathbf{I}(t) - V_{\mathrm{th}}\mathbf{S}_{\mathrm{out}}(t). \tag{41}$$

Output spikes are then generated:

$$\mathbf{S}_{\mathrm{out}}(t) = \Theta(\mathbf{U}(t) - V_{\mathrm{th}}) \tag{42}$$

and over time, the accumulated spike activity approximates ANN feature maps over rate decoding:

$$\mathbf{Y} \approx \frac{1}{T}\sum_{t=1}^{T} \mathbf{S}_{\mathrm{out}}(t). \tag{43}$$

Thus, convolutional feature extraction is preserved in the temporal domain.

2) Event-Driven Convolution

Unlike conventional convolution, which operates on dense continuous-valued tensors, event-driven convolution [27] operates directly on asynchronous spike events. The module consists of a parameter counter and a trainable convolutional layer. The parameter counter tracks the frequency with which each kernel parameter covers a specific position on the response map. For a given position A, the final response value is a linear combination of the kernel weights W:

$$Responemap(A) = \sum_{j=1}^{n} n_{AW_j} \times W_j \tag{44}$$

where W_j is the j-th parameter of the kernel and n_{AW_j} is the count of occurrences where position A was covered by W_j. When an event $e_i = (t_i, x_i, y_i)$ is received, the counter increments the corresponding indices in a parameter count feature map. Finally, a convolution is performed on this feature map using a stride equal to the kernel size. Because this operation is differentiable, the kernel parameters can be optimized during training to achieve lower loss values, allowing for more effective local feature extraction compared to traditional fixed-parameter methods.

A related development is spike sparse convolution (SSC) [231] with spike voxel coding (SVC), which exploits the inherent sparsity of spike tensors by restricting computation to active spatial locations. The SVC scheme transforms raw 3D point sets into spatio-temporal spike trains S. Given a voxelized representation V, the encoding process is defined as:

$$S = SN_m(F_m(V)) \tag{45}$$

where SN_m and F_m represent m consecutive spiking neurons and sparse convolutions, respectively. This transformation reduces storage requirements and enables the model to leverage the inherent sparsity of SNNs. The proposed SSC module optimizes 3D feature extraction by limiting computations to active spiking locations. While a Vanilla Spike Convolution (VSC) enumerates all positions in the 3D kernel space K^3, SSC introduces a selector variable α:

$$U_p^t = \sum_{k\in K^d} \alpha(W_k \cdot S_{p\oplus k}^t) \tag{46}$$

where U_p^t is the membrane potential at time t and position p, and W_k are the synaptic weights. The selector $\alpha \in \{0,1\}$ triggers a computation ($\alpha = 1$) only if the center position of the convolution kernel contains an active spike ($p \in S^t$). This mechanism is implemented on neuromorphic hardware via a rulebook that maps active spikes to their corresponding synaptic weights and target neurons:

$$K^3(p, P_{in}) = \{k \mid p + k \in P_{in}, k \in K^3\} \tag{47}$$

By filtering out empty positions, SSC avoids the cubic growth of computational complexity typical of 3D data and achieves high energy efficiency through sparse addition (AC) operations.

3) Spatio-Temporal Convolution

A key distinction between ANNs and SNNs is the explicit representation of temporal information. To leverage this property, convolution has been extended to operate across both spatial and temporal dimensions. To enhance the spatio-temporal feature extraction capabilities of SNNs, [232] proposed the Spatio-Temporal Synaptic Connection (STSC) module. The STSC is a plug-and-play module designed to expand the receptive fields of synaptic connections into the temporal dimension by combining a filtering mechanism and a gating mechanism.

In proposed research, the Temporal Response Filter (TRF) module expands the temporal receptive field by establishing a linear response of spikes over time through temporal convolution. For a given input X with N spatial channels, the filtered response $C(t,n)$ is defined as:

$$C(t,n) = \sum_{t_f=-\frac{K_F-1}{2}}^{\frac{K_F-1}{2}} W_{F,t_f,n} \times X(t-t_f,n) \tag{48}$$

where K_F denotes the temporal receptive field and W_F represents the learnable weights of the temporal depth-wise convolution. The Feedforward Lateral Inhibition (FLI) module implements a nonlinear gating mechanism inspired by biological lateral inhibition. It computes gating coefficients $D(t,n)$ to regulate input transmission. The intermediate temporal feature $S(t,m)$ and the final gating factor are given by:

$$\begin{aligned} S(t,m) &= \sum_{t_g=-\frac{K_G-1}{2}}^{\frac{K_G-1}{2}} \sum_{n=1}^{N} W_{G1,t_g,n,m} \times X(t-t_g,n) \\ D(t,n) &= \sigma\left(\sum_{m=1}^{M} W_{G2,m,n} \times \mathrm{ReLU}(S(t,m))\right) \end{aligned} \tag{49}$$

where σ is the Sigmoid function and K_G is the receptive field of the FLI module. For 3D spatial inputs, spatial-wise average pooling is performed to obtain channel-wise sparsity prior to gating factor computation.

The final output Y of the STSC module is obtained via an element-wise product of the filter and gate outputs:

$$Y = C \odot D \tag{50}$$

This output is then passed to standard spatial operations, such as fully-connected or convolutional layers, effectively endowing them with spatio-temporal dependencies.

4) Factorized and Efficient Spiking Convolution

To improve computational and energy efficiency, several works have adapted factorized convolution techniques for SNNs.

To achieve efficient temporal modeling in SNNs, pior work proposed the Spiking Depthwise Separable Temporal Convolutional Network (SDSTCN) [233]. The core of this architecture is the decoupling of temporal and channel-wise computations through Depthwise Separable Convolution (DSC). The membrane potential $V(t)$ is accumulated from input spikes $\delta_i(t)$ as follows:

$$V(t) = \sum_{i=0}^{N} w_i' \cdot \delta_i(t) \tag{51}$$

where w_i' represents the time-step weighting parameters. To maintain stability and capture historical information, a residual integration strategy is employed:

$$Output_t = LIF(r_t + Residual_t) \tag{52}$$

where r_t denotes the main branch features and $Residual_t$ represents the shortcut path features processed via a 2D convolution.

Pointwise spike convolution [234], implemented as 1×1 convolution, is commonly used to model channel-wise interactions efficiently in spike-based architectures. To overcome the low-pass filtering characteristics of vanilla spiking transformers, [234] propose the Frequency-Aware Token Mixer (FATM). FATM processes input spike sequences through three parallel branches to capture spatial, frequency, and channel-wise representations in a multiplication-free, event-driven manner. The Spatial Learner (SL) and Channel Mixer (CM) leverage the local perception capabilities of convolutional operations. For an input spike sequence S', the membrane potential outputs are defined as:

$$U^{SL} = \mathrm{Conv}_{3\times3}(S'), \quad U^{CM} = \mathrm{Conv}_{1\times1}(S') \tag{53}$$

where U^{SL} extracts local spatial features and U^{CM} facilitates cross-channel information aggregation.

The Frequency Learner (FL) utilizes a spiking Haar wavelet transform to project signals into the time-frequency domain. The spiking forward Haar transform is formulated as:

$$H_f = \mathrm{Spk}(W_{haar} \cdot \mathrm{Spk}(I \cdot W_{haar}^{\top})) \tag{54}$$

where W_{haar} denotes the transformation matrix. To preserve signal integrity across negative matrix terms, we can employ a ternary neuron model with negative spike dynamics:

$$s[n] = H_{sym}(U[n-1] - V_{th}) \tag{55}$$

where $H_{sym}(x)$ is 1 if $x \geq 0$ and -1 if $x < 0$.

To enhance parameter efficiency, researcher also adopt a block-diagonal weight structure. The $d \times d$ weight matrix is split into k blocks of size $d/k \times d/k$, reducing complexity to $O(d^2/k)$ and enabling multi-head-like subspace projections. The final FATM output is the summation of all branches:

$$U^{FATM} = U^{FL} + U^{SL} + U^{CM} \tag{56}$$

This architecture allows spiking transformer to maintain high-frequency information transmission into deep layers, significantly outperforming global attention mechanisms.

5) Stochastic Binary Convolution

To improve hardware efficiency, particularly for neuromorphic implementations, binary convolution kernels have been proposed, where weights are constrained to discrete values [235]. The ReStoCNet architecture [235] utilizes a deep residual convolutional framework composed of binary kernels to achieve memory-efficient neuromorphic computing. Information is processed as sparse spiking events, where input pixels are encoded into Poisson spike trains. The convolutional layers employ binary kernels constrained to logic states $w \in \{-1, +1\}$. At each time step, the input spike maps are convolved with these kernels to produce a post-synaptic current I_{post}, which modulates the membrane potential V_{mem} of LIF neurons:

$$\tau_{mem}\frac{dV_{mem}}{dt} = -V_{mem} + I_{post} \tag{57}$$

where τ_{mem} represents the membrane leak time constant. Residual connections are introduced between layers to perform identity mapping, mitigating accuracy degradation in deep SNNs.

Kernels are trained using a probabilistic Hybrid-STDP (HB-STDP) rule that incorporates Hebbian and anti-Hebbian mechanisms. The synaptic switching probabilities are determined by the spike timing difference $\Delta t = t_{post} - t_{pre}$. The probability of potentiation $P_{L\to H}$ is defined as:

$$P_{L\to H} = \begin{cases} p_{Hebb_pot} & 0 < \Delta t \leq t_{Hebb_pot} \\ 0 & \text{otherwise} \end{cases} \tag{58}$$

Conversely, the probability of depression $P_{H\to L}$ is given by:

$$P_{H\to L} = \begin{cases} p_{antiHebb_dep} & \Delta t \geq t_{antiHebb_dep} \\ p_{Hebb_dep} & t_{Hebb_dep} \leq \Delta t \leq 0 \\ 0 & \text{otherwise} \end{cases} \tag{59}$$

Notably, the introduction of a *dead zone* between t_{Hebb_pot} and $t_{antiHebb_dep}$ ensures that binary synapses learn only strongly correlated features, preventing the network from becoming memoryless due to rapid weight switching.

To facilitate generalized feature learning across input classes, the authors implement a mini-batch training methodology. Kernel updates are performed by averaging spike timing differences using a fixed $STDPstride$ across the output maps. This mechanism allows the binary kernels to self-learn high-level hierarchical representations while maintaining a memory footprint significantly smaller than full-precision (32-bit) networks.

E. OVERALL DIRECTION OF SNN IN THE REALM OF CV

Similar to traditional ANNs, SNNs have emerged as an influential paradigm in advancing modern computer vision and signal processing methodologies. Their application is particularly promising in domains involving temporally rich data or scenarios prioritizing computational efficiency.

1) SNN in Image Classification

Image classification refers to the task of assigning images to one of several predefined categories [1]. Within the context of SNNs, the objective remains the same: to develop a spiking model capable of accurately recognizing and categorizing images from benchmark datasets.

Although image classification has been extensively studied in traditional ANNs, this accumulated knowledge has enabled the development of SNNs that perform well across a wide range of dataset scales, from small datasets [206] to large-scale benchmarks [156], [84], [83], within a relatively short development period.

Beyond its maturity as a research topic, image classification continues to serve as an important benchmark for various lines of investigation in the SNN community. These include:

- *Design of novel SNN architectures:* Image classification offers a controlled environment for evaluating new architectural components, neuron models, and connectivity patterns, enabling researchers to assess improvements in representational power and efficiency [125], [74], [236], [237].
- *ANN-to-SNN conversion methods:* Conversion pipelines that transform trained ANNs into SNNs often rely on image classification datasets to measure how faithfully accuracy and behavior are preserved after conversion [152], [83].
- *Spike-based signal encoding schemes:* Encoding strategies determine how continuous-valued inputs are transformed into spike trains. Image classification tasks provide a standardized means to test whether encoding schemes lead to improved performance and/or energy efficiency [15], [14], [210].
- *Adversarial training and robustness analysis:* The vulnerability of SNNs to adversarial perturbations is frequently studied via image classification benchmarks, enabling systematic comparisons against ANN-based defenses and stress-testing model reliability [198], [199], [201].
- *Optimization and learning rules:* Novel training methods and biologically inspired learning rules can be validated on image classification tasks to quantify their effectiveness, convergence behavior, and scalability [200], [144], [185], [87], [238].
- *Neuromorphic datasets and integration with event-based sensors:* Image classification provides a natural testbed for integrating neuromorphic sensors such as DVS [220] and LiDAR systems [239] with SNN-based processing pipelines. This integration facilitates rigorous evaluation of real-time, event-driven perception. Moreover, the adoption of such sensors has led to the development of widely used neuromorphic datasets tailored for recognition tasks, including DVS-Gesture [240], CIFAR10-DVS [241], and N-MNIST [242], which have become foundational benchmarks for advancing spike-based vision research.

- *Hardware deployment:* Due to its well-defined structure and manageable complexity, image classification serves as an ideal benchmark for evaluating the deployment of SNNs on neuromorphic hardware platforms. Systems such as Intel Loihi [243], [244], IBM TrueNorth [245], [246], and FPGA-based accelerators [247], [248], [249] frequently rely on classification tasks to assess latency, energy efficiency, and overall feasibility of spike-driven computation in real-world settings.

2) SNN in other computer vision tasks

Beyond image recognition, numerous computer vision tasks have been extensively investigated in deep learning, each addressing a distinct objective. These tasks have likewise begun to attract attention within the SNN community, leading to a growing body of research exploring their spiking counterparts. Prominent examples include:

- *Synthetic image generation:* The goal of synthetic image generation is to produce novel, realistic images by sampling from learned data distributions [250], [251], [252]. Recent work has extended this paradigm to the spiking domain, demonstrating the feasibility of SNN-based generative models [253], [254], [255], [256], [257].
- *Object detection:* Object detection aims to identify and localize objects within an image [258]. Emerging SNN-based detection frameworks [78], [259], [239], [236] have yielded promising results, indicating substantial potential for energy-efficient detection architectures.
- *Semantic segmentation:* Semantic segmentation assigns a class label to each pixel in an image, enabling dense scene interpretation [260]. In recent years, a variety of SNN architectures have been proposed for this task [261], [195], [262], [263], showcasing the viability of spike-based segmentation.
- *Motion estimation:* Motion estimation seeks to determine object or camera movement across consecutive frames [264]. Spiking implementations of motion estimation are beginning to emerge [265], [266], [267], leveraging event-driven dynamics for efficient processing.

VIII. APPLICATIONS OF SNN IN GRAPH-RELATED PROBLEMS

A. WHAT IS GRAPH LEARNING?

Graph learning [268] concerns machine learning methods designed to operate on data represented as graphs, where relationships between entities are explicitly modeled. Unlike grid-structured data such as images, graphs exhibit irregular topology and non-Euclidean structure, making traditional convolutional operations inapplicable. A graph is formally defined as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{v_1, \ldots, v_N\}$ denotes a set of nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ represents edges encoding relationships between nodes. Each node may be associated with a feature vector, forming a feature matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$, and connectivity is typically described by an adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$.

Graph neural networks (GNNs) [268] learn representations through iterative message passing between neighboring nodes:

$$\mathbf{H}^{(l+1)} = \sigma\left(\mathbf{A}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right), \tag{60}$$

where $\mathbf{H}^{(l)}$ denotes node embeddings at layer l, $\mathbf{W}^{(l)}$ are learnable parameters, and $\sigma(\cdot)$ is a nonlinear activation function. By aggregating local neighborhood information, GNNs capture relational dependencies that are difficult to model using independent feature processing.

Graph learning encompasses several fundamental tasks depending on the prediction target and granularity.

1) Node Classification

Node classification [268] assigns labels to individual nodes based on both their features and graph structure:

$$\mathbf{y}_i = f_\theta(v_i, \mathcal{G}), \quad \mathbf{y}_i \in \{1, \ldots, K\}. \tag{61}$$

The objective is to learn node representations that incorporate contextual information propagated through neighboring nodes. Typical applications include social network analysis [269], citation networks [270], and biological interaction graphs [271].

2) Graph Classification

Graph classification [268] predicts a single label for an entire graph $y = f_\theta(\mathcal{G})$ [272], where the model aggregates node representations into a global embedding:

$$\mathbf{h}_\mathcal{G} = \mathrm{READOUT}(\{\mathbf{h}_i\}_{i=1}^N). \tag{62}$$

This task appears in molecular property prediction [273], protein structure analysis [274], and scene understanding [275].

3) Edge Prediction and Link Prediction

Link prediction [276] estimates the likelihood of relationships between node pairs $p_{ij} = f_\theta(v_i, v_j, \mathcal{G})$, which determines whether an edge exists or will form in the future. Applications include recommendation systems [277], knowledge graph completion [278], and social network evolution modeling [269].

4) Graph Representation of Images

Although images are traditionally processed on regular grids, they can be reformulated as graphs to capture long-range dependencies and adaptive connectivity [279]. An image can be represented as $\mathcal{G}_{\text{img}} = (\mathcal{V}, \mathcal{E})$, where nodes correspond to pixels, patches, or superpixels, and edges encode spatial proximity or feature similarity. Image classification can then be expressed as $y = f_\theta(\mathcal{G}_{\text{img}})$. This formulation enables flexible receptive fields, non-local reasoning, and adaptive structure learning beyond fixed convolution kernels. Graph-based representations have been explored for image classification [279], segmentation [280], and scene parsing [281].

5) Graph Generation

Graph generation [282] aims to synthesize new graphs consistent with a learned distribution $\mathcal{G}_{\text{gen}} = f_\theta(\mathbf{z})$, where $\mathbf{z}$ is a latent variable. The goal is to model $\mathcal{G}_{\text{gen}} \sim p_\theta(\mathcal{G})$, which is particularly important in molecule discovery [283] and network simulation [284].

6) Dynamic and Temporal Graph Learning

As many real-world graphs evolve over time, temporal graph learning [285] models sequences of graphs $\{\mathcal{G}_t\}_{t=1}^T$, with the objective $\mathbf{Y}_t = f_\theta(\mathcal{G}_1, \ldots, \mathcal{G}_t)$, capturing both structural and temporal dependencies. Applications include traffic prediction [286], event modeling [287], and neural activity analysis [288].

B. GRAPH LEARNING BEFORE GRAPH NEURAL NETWORKS

Prior to the emergence of GNNs, graph-structured problems were primarily addressed using a combination of graph theory, probabilistic models, and feature engineering techniques. Unlike modern representation learning approaches, these methods relied heavily on manually designed descriptors and shallow learning algorithms.

1) Hand-Crafted Structural Features

Early approaches transformed graph data into fixed-length feature vectors so that conventional machine learning models could be applied [289]. For a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, feature extraction functions were designed as $\phi : \mathcal{G} \to \mathbb{R}^d$, where $\phi(\mathcal{G})$ encodes structural statistics such as:

- node degree distributions [290],
- clustering coefficients [291],
- shortest-path statistics [292],
- spectral properties of the graph Laplacian [293],
- motif or subgraph counts [294].

These handcrafted features were then used with classifiers such as support vector machines or logistic regression. While effective for small problems, performance strongly depended on domain expertise and failed to generalize across tasks.

2) Kernel Methods on Graphs

Graph kernels [295] were introduced to measure similarity between graphs without explicit vector representations. A kernel function $k(\mathcal{G}_i, \mathcal{G}_j)$ computes similarity based on shared substructures such as walks, paths, or subtrees. Examples include random-walk kernels [296] and Weisfeiler-Lehman subtree kernels [297]. Although theoretically elegant, graph kernels suffer from high computational complexity, poor scalability to large graphs, inability to learn task-adaptive representations [295].

3) Probabilistic Graphical Models

Another line of work employed probabilistic graphical models, including Markov Random Fields (MRFs) [298] and Conditional Random Fields (CRFs) [299]. These methods modeled dependencies between neighboring nodes through joint probability distributions:

$$p(\mathbf{y}|\mathbf{X}) \propto \prod_i \psi_i(y_i, \mathbf{x}_i) \prod_{(i,j)\in\mathcal{E}} \psi_{ij}(y_i, y_j), \tag{63}$$

where ψ_i and ψ_{ij} denote unary and pairwise potentials.

Such models were widely used for structured prediction tasks like image segmentation and relational classification. However, inference was often computationally expensive and parameter learning required strong modeling assumptions.

4) Matrix Factorization and Network Embedding

Later approaches focused on learning low-dimensional node embeddings using matrix factorization or random-walk-based methods. Given an adjacency matrix $\mathbf{A}$, embeddings were learned as:

$$\mathbf{Z} \in \mathbb{R}^{N\times d}, \quad \mathbf{A} \approx \mathbf{Z}\mathbf{Z}^\top. \tag{64}$$

Methods such as DeepWalk [300] and node2vec [301] generated node representations via stochastic walks followed by shallow embedding learning. While scalable, these methods:

- separated representation learning from downstream tasks,
- lacked end-to-end optimization,
- could not naturally incorporate node features.

C. CHALLENGES AND PROGRESSION TOWARD GNN

Despite significant progress, pre-GNN methods faced several fundamental challenges:

- **Irregular structure:** Graphs lack translation invariance and fixed neighborhood size, preventing direct application of convolutional neural networks designed for grid data [302].
- **Feature engineering dependency:** Performance relied heavily on manually designed structural descriptors rather than learned representations [303].
- **Limited local-global integration:** Existing approaches struggled to jointly capture local neighborhood interactions and global graph structure [304].
- **Scalability:** Kernel methods and probabilistic inference scaled poorly with graph size [295].

The development of GNNs emerged from attempts to generalize deep learning principles to non-Euclidean domains. Two key insights enabled this transition:

- **Neighborhood Aggregation**: node representations can be iteratively refined by aggregating information from neighboring nodes [305].
- **Shared Local Operators**: analogous to convolution, the same aggregation function can be applied across all nodes [306].

This led to the message-passing formulation:

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{(l)} \mathbf{W}^{(l)} \mathbf{h}_j^{(l)}\right), \tag{65}$$

where $\mathcal{N}(i)$ denotes the neighborhood of node i. By integrating representation learning with graph structure in an end-to-end differentiable framework, GNNs unified ideas from spectral graph theory, probabilistic modeling, and neural networks.

D. SPIKING GNN DESIGNING METHODOLOGIES

This section reviews representative designing methodologies for spiking GNNs, denoting how recent works integrate graph representation learning with neuromorphic computation principles. Existing approaches differ primarily in how they combine graph topology modeling, spike-based information encoding, and temporal neuronal dynamics. As summarized in Table 5, these methods span convolution-based, attention-driven, dynamic graph, continuous-time, and normalization-oriented frameworks, each addressing key challenges such as computational efficiency, temporal modeling, training stability, and memory consumption in spiking graph learning.

1) Spiking Graph Convolutional Neural Network (SpikingGCN)

The Spiking Graph Convolutional Neural Network (SpikingGCN) [307] framework follows an end-to-end architecture composed of graph feature aggregation, spike encoding, and a neuromorphic decoding process. To incorporate the graph's topological structure alongside node features, SpikingGCN utilizes a simplified graph convolution method. Let $G = (V, A)$ be a graph with adjacency matrix A and attribute matrix $X \in \mathbb{R}^{N\times d}$. The normalized graph Laplacian is used to aggregate neighborhood information. The resulting node representation H is computed as:

$$S = \tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}, \quad H = S^K X \tag{66}$$

where $\tilde{A} = A + I$ represents the adjacency matrix with self-connections, $\tilde{D}$ is the corresponding degree matrix, and K is the number of convolution layers.

Because SNNs operate on discrete signals, the continuous float-point values in H must be transformed into binary spike trains O_{pre}. SpikingGCN employs a probability-based Bernoulli encoding scheme:

$$p(o_{ij}) \sim \text{Bernoulli}(\lambda_{ij}), \quad \lambda_{ij} = \min(h_{ij}, 1.0) \tag{67}$$

where $o_{ij} \in \{0, 1\}$ is a pre-synaptic spike for node i and feature j. This sampling process is repeated over T time steps, where T defines the resolution of the encoded message.

The spike decoder consists of a fully connected layer followed by a LIF neuron layer. The membrane potential V_m^t of a neuron at time t is updated by accumulating input spikes weighted by trainable synaptic parameters ψ:

$$V_m^t = k_m V_m^{t-1} + \sum_j \psi_j o_{ij} \tag{68}$$

where k_m is a decay factor related to the membrane time constant τ_m.

When the potential V_m^t exceeds a threshold V_{th}, the neuron fires an output spike $O_{post}^{i,t}$ according to the Heaviside function:

$$H(V_m^t) = \begin{cases} 1 & \text{if } V_m^t \geq V_{th} \\ 0 & \text{otherwise} \end{cases} \tag{69}$$

Alternatively, a refined version (SpikingGCN-N) uses a negative threshold to model inhibitory processes. After firing, the membrane potential is reset $V_m^t = V_m^t - V_{th}$.

The final classification for a node is determined by its firing rate, which is the average number of spikes generated over the T time steps $\hat{y}_i = \frac{1}{T}\sum_{t=1}^{T} O_{post}^{i,t}$.

2) SpikeGraphormer

SpikeGraphormer [308] is an end-to-end framework consisting of a Spiking Graph Attention (SGA)-driven branch and a sparse GNN branch, fused to provide comprehensive node representations. The fundamental unit of the architecture is the LIF neuron. The neuron's dynamics are governed by the following iterative equations at time step t:

$$\begin{aligned} U[t] &= H[t-1] + X[t], \\ S[t] &= \Theta(U[t] - u_{th}), \\ H[t] &= V_{reset}S[t] + (\beta U[t]) \odot (1 - S[t]), \end{aligned} \tag{70}$$

where $U[t]$ is the membrane potential, $X[t]$ is the spatial input, $H[t]$ represents the temporal output, u_{th} is the firing threshold, and $\text{Hea}(\cdot)$ is the Heaviside step function which generates a binary spike $S[t] \in \{0, 1\}$.

The SGA module transforms numerical inputs Q, K, and V into binary spikes using SNN layers. This allows the traditional quadratic attention mechanism to be replaced by linear-complexity operations. For a specific channel i, the attention calculation is expressed as:

$$\text{SGA}(Q_i, K_i, V_i) = \text{SNN}(Q_i)\left(\text{SNN}(K_i)^\top \odot \text{SNN}(V_i)\right) \tag{71}$$

Because $\text{SNN}(K_i)^\top \odot \text{SNN}(V_i)$ produces a scalar (0 or 1), the operation effectively becomes a mask operation. The overall complexity is reduced from $O(N^2)$ to $O(ND)$, where N is the number of nodes and D is the feature dimension.

To compensate for the Transformer branch's potential loss of local topological information, a parallel sparse GNN branch (e.g., GCN) is employed. The final node representation Z is obtained through a weighted fusion strategy:

$$Z = (1 - \alpha)S_l + \alpha\text{GNN}(X, A) \tag{72}$$

where S_l is the output of the L_1 spiking encoder blocks, $\text{GNN}(X, A)$ represents the structural encoding from the adjacency matrix A, and α is a manually determined hyperparameter governing the graph weight.

The total complexity of the model is $O(ND + E)$, where E is the number of edges. Given that real-world graphs are typically sparse ($E \approx N$), the complexity remains linear with respect to the number of nodes. The final node classification is performed by a linear classifier $Y = \text{CLS}(Z)$.

TABLE 5: Summary of representative Spiking GNN designing methodologies.

Model	Core Idea	Graph Processing Strategy	Spiking Mechanism	Key Advantages / Characteristics
SpikingGCN [307]	End-to-end GCN with spike encoding and neuromorphic decoding	Simplified graph convolution using normalized Laplacian aggregation $S^K X$	Bernoulli spike encoding + LIF neuron decoding with rate-based readout	Bridges classical GCN and SNN pipeline; simple architecture; supports inhibitory dynamics (SpikingGCN-N)
SpikeGraphormer [308]	Spiking Transformer–GNN hybrid framework	Parallel Spiking Graph Attention (SGA) and sparse GNN branch with feature fusion	LIF neurons convert Q, K, V into spikes; attention becomes binary masking	Reduces attention complexity from $O(N^2)$ to $O(ND)$; preserves both global attention and local topology
Dy-SIGN [309]	Dynamic graph learning with equilibrium-based spiking dynamics	Spatial graph convolutions combined with temporal spike propagation across graph snapshots	LIF neurons with implicit equilibrium training instead of BPTT	Memory-efficient gradient computation; captures temporal evolution; avoids storing full forward history
DRSGNN [310]	Dynamic reactive neuron-based spiking GNN	Pre-computed SGC feature propagation with positional encoding integration	Poisson spike encoding + adaptive-threshold reactive neurons	Learnable neuron thresholds enable diverse dynamics; incorporates long-range structure via RWPE
COS-GNN [311]	Continuous–discrete hybrid spiking graph modeling	Graph evolution modeled by ODEs driven by spike dynamics	First- and second-order spiking neurons coupled with continuous graph ODEs	Stable training with bounded gradients; captures long-term temporal and high-order correlations
SpikeNet [312]	Temporal graph representation via spike-based aggregation	Neighborhood sampling from both current graph and structural differences ΔG_t	LIF neurons with adaptive thresholds and masked summation aggregation	Efficient computation using binary indexing; models macro- and micro-dynamics in evolving graphs
STFN Graph SNN [313]	Spatial-temporal normalization for stable spiking training	Unfolded graph message passing across temporal dimension	LIF neurons with spatial-temporal feature normalization	Improves training stability by aligning membrane potentials with firing thresholds
GSAT [314]	Spiking attention mechanism for sparse edge learning	Attention coefficients learned via spike-driven neuron interactions	Poisson spike encoding + charge–fire–reset spiking attention neurons	Learns sparse attention edges biologically; replaces soft attention with firing-rate attention

3) Dynamic Spiking Graph Neural Networks (Dy-SIGN)

The Dynamic Spiking Graph Neural Networks (Dy-SIGN) [309] framework processes dynamic graph snapshots $\mathcal{G} = \{G_1, \ldots, G_T\}$ by combining spatial graph convolutions with temporal spiking dynamics and an implicit equilibrium-based training strategy.

At each time step t, the node features X_t are encoded into binary spike trains $s_{t,\tau}$ across K time latency steps. The core processing unit is the LIF neuron, where the membrane potential u and output spikes s are updated as follows:

$$\begin{aligned} u_{\tau+1,i} &= \lambda(u_{\tau,i} - V_{th} s_{\tau,i}) + \sum_j w_{ij} s_{\tau,j} + b \\ s_{\tau+1,i} &= \Theta(u_{\tau+1,i} - V_{th}) \end{aligned} \tag{73}$$

where $\lambda < 1$ is the leaky term, V_{th} is the threshold, and $\Theta(\cdot)$ is the Heaviside step function.

To mitigate the loss of graph structure details during spike propagation, Dy-SIGN establishes a direct information channel between shallow and deep layers. The potential for layer l at latency τ is formulated to incorporate original features:

$$\begin{aligned} u_\tau^1 &= \lambda u_{\tau-1}^1 + W^1 s_{\tau-1}^1 + F^1 x_{\tau-1}, \\ u_\tau^l &= \lambda u_{\tau-1}^l + W^l s_{\tau-1}^l + F^l s_{\tau-1}^{l-1}, \quad l = 2, \ldots, N \end{aligned} \tag{74}$$

where F^1 is the information compensation matrix that bridges the input directly to the embedding process.

Instead of standard BPTT, which is memory-intensive, Dy-SIGN utilizes implicit differentiation on the equilibrium state. The model approximates a fixed-point firing rate a_t^* satisfying:

$$a_t^* = \sigma\left(\frac{1}{V_{th}}(W a_t^* + F x_t^*)\right) \tag{75}$$

The gradient of parameters W^l is computed via a variation training method that simplifies non-differentiable terms:

$$\frac{\partial L}{\partial W^l} = \sum_{\tau=1}^{K} \frac{\partial L}{\partial s_\tau^{l+1}} \frac{\partial s_\tau^{l+1}}{\partial u_\tau^{l+1}} \left(\sum_{k \le \tau} \lambda^{\tau-k} s_k^l\right) \tag{76}$$

This approach allows for gradient calculation without storing the exact forward procedure, significantly reducing memory overhead.

Temporal dynamics are captured by propagating the equilibrium state $a_{t,K}$ of a graph snapshot as the initial membrane potential for the next time step: $u_{t+1,1} = a_{t,K}$ [11, 12]. The final node classification uses a concatenation of all fixed-point representations across the time window T:

$$\hat{y} = \text{FC}\left(\|_{t=1}^{T} a_t^*\right) \tag{77}$$

where $||$ denotes concatenation and FC is a fully connected layer.

4) Dynamic Reactive Spiking Graph Neural Network (DRSGNN)

The Dynamic Reactive Spiking Graph Neural Network (DRSGNN) [310] framework consists of a pre-computing feature propagation stage, Poisson-based spike encoding, a dynamic reactive spiking transformation layer, and an adaptive positional integration module.

To maintain high efficiency, graph feature propagation is treated as a pre-computing process based on Simplified Graph Convolution (SGC). For a K-step propagation, the features are aggregated as:

$$\hat{X} = \hat{A}^K X, \quad \hat{A} = \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} \tag{78}$$

where $\tilde{A}$ is the adjacency matrix with self-loops. These continuous features $\hat{X}$ are then transformed into a sequence of binary spike trains $[\hat{X}^1, \hat{X}^2, \ldots, \hat{X}^T]$ over T timesteps using Poisson coding.

The core of the architecture is the DRSGT layer, which utilizes dynamic reactive neurons. Each i-th post-neuron is assigned a unique, optimizable threshold V_{th}^i to explore diverse reactive dynamics. The membrane potential V_i^t for node k at timestep t is updated as:

$$V_i^t = \lambda V_i^{t-1} + \sum_j W_{ij}\hat{X}_j^t - V_{th}^i Z_i^{t-1} \tag{79}$$

The firing process is determined by the relative membrane potential $V_i^{t-1}(rl), k$:

$$Z_{i,k}^{t-1} = \begin{cases} 1 & \text{if } \frac{V_{i,k}^{t-1}}{V_{th}^i} > 1 \\ 0 & \text{otherwise} \end{cases} \tag{80}$$

where λ is the leak factor and W_{ij} represents the trainable synaptic weights.

To overcome the underutilization of long-range neighbor information, the model integrates relative positional information such as Random Walk Positional Encodings (RWPE). The initial positional features PE_k for node k are defined as:

$$PE_k = [RW_{kk}, RW_{kk}^1, \ldots, RW_{kk}^{d_{pos}}]^\top \tag{81}$$

These are converted into positional spikes $\hat{PE}^t$ and concatenated with the feature spikes before being processed by the DRSGT layer: $[\hat{X}^t; \hat{PE}^t]$. This allows the model to adaptively learn structural similarities and discrepancies between distant nodes.

The final prediction Y_{pred}^i for the i-th class is derived by stacking and normalizing the spiking outputs over T duration:

$$Y_{pred}^i = \frac{\sum_{t=1}^T Z_i^t}{\sum_{j=1}^c \sum_{t=1}^T Z_j^t} \tag{82}$$

The model is trained by minimizing the Mean Squared Error (MSE) loss using backpropagation with pseudo-derivatives to update both the weights W and the unique thresholds V_{th}.

5) Continuous Spiking Graph Neural Networks (COS-GNN)

The Continuous Spiking Graph Neural Networks (COS-GNN) [311] framework integrates discrete spiking dynamics with continuous ordinary differential equations (ODEs) to model graph representations over time.

The architecture defines node representations u by interacting dynamics in dual time dimensions: spiking latency τ and graph evolution time t. The unified PDE formulation for node representation u_{NT} at total latency N and time T is expressed as:

$$\begin{aligned} u_{NT} = & 2\int_0^{T-1} f\left(\int_0^N g(u_x^y, x)dx\right)dy \\ & + \int_{T-1}^{T} f\left(\int_0^N g(u_x^y, x)dx\right)dy \end{aligned} \tag{83}$$

where $g(\cdot)$ represents the SNN dynamics and $f(\cdot)$ represents the CGNN dynamics. This allows the SNN to compute spike representations at each time step, which then drive the evolution of the graph process.

In the first-order variant, node features are processed through a standard LIF model to generate binary spikes s:

$$\begin{aligned} u_{\tau+1,i} &= \lambda(u_{\tau,i} - V_{th}s_{\tau,i}) + \sum_j w_{ij}s_{\tau,j} + b, \\ s_{\tau+1,i} &= H(u_{\tau+1,i} - V_{th}) \end{aligned} \tag{84}$$

The continuous graph process evolves based on these spikes. The hidden state $H(t)$ at time step t is updated using a graph ODE that incorporates the weighted sum of spike trains:

$$\frac{dH(t)}{dt} = \ln AH(t) + \frac{\sum_{\tau=1}^N \lambda^{N-\tau} s_0^\tau}{\sum_{\tau=1}^N \lambda^{N-\tau}} \tag{85}$$

To mitigate the information loss caused by binary discretization, COS-GNN-2nd utilizes a second-order SNN that models synaptic conductance I alongside membrane potential u:

$$I_{\tau+1} = \alpha I_\tau + WX_{\tau+1}, \quad u_{\tau+1} = \beta u_\tau + I_{\tau+1,i} - R \tag{86}$$

A differentiable second-order spike representation $\hat{a}(N)$ is derived to facilitate backpropagation through the continuous layers:

$$\hat{a}(N) \approx \text{clamp}\left(\frac{\tau_{syn}\tau_{mem}}{\tau_{mem} - \tau_{syn}}\hat{I}(N), 0, \frac{V_{th}}{\Delta\tau}\right) \tag{87}$$

This representation is then integrated into a second-order graph ODE to capture high-order correlations and long-term temporal trends:

$$X'' = \sigma(F_\theta(X, t)) - \gamma X - \alpha X' \tag{88}$$

The construction is theoretically proven to be stable, effectively addressing the vanishing and exploding gradient problems common in deep GNNs. The total gradient is globally bounded by the system's thresholds and time-step constants:

$$\begin{aligned} \left\|\frac{\partial L}{\partial W_k}\right\| \leq & (1 + T\Gamma\Delta t)(1 + N\Theta\Delta\tau) \\ & \cdot \frac{V_{th}}{v\beta^2\Delta\tau}(\max|X_N^i| + \max|\bar{X}_i|) \end{aligned} \tag{89}$$

where Δt and $\Delta\tau$ represent the discretization steps for the graph process and spiking latency, respectively.

6) SpikeNet

SpikeNet [312] models temporal graphs as a sequence of snapshots $\mathcal{G} = \{G_1, G_2, \ldots, G_T\}$. It captures structural and temporal patterns through neighborhood sampling, spike-based aggregation, and a bio-inspired membrane potential mechanism.

To capture both broad evolution patterns and fine-grained changes, SpikeNet samples neighbors from the current snapshot G_t and the graph difference $\Delta G_t = G_t - G_{t-1}$. The sample set $S^t(v)$ for node v is defined as:

$$S^t(v) = SAMPLE(v, G_t) \cup SAMPLE(v, \Delta G_t) \tag{90}$$

where $SAMPLE(v, G_t)$ captures macro-dynamics and $SAMPLE(v, \Delta G_t)$ captures micro-dynamics.

The core temporal architecture uses the LIF model to simulate neuron membrane potential V^t. The iterative update rule is:

$$V^t = V^{t-1} + \frac{1}{\tau_m}\left(-(V^{t-1} - V_{reset}) + I^t\right) \tag{91}$$

where τ_m is the membrane decay constant and I^t is the pre-synaptic input. To increase flexibility, SpikeNet employs an adaptive threshold V_{th}^t that updates based on previous spikes O^t:

$$V_{th}^t = \tau_{th} V_{th}^{t-1} + \gamma O^t \tag{92}$$

A spike is fired ($O^t = 1$) via the Heaviside step function $\Theta(V^t - V_{th}^{t-1})$ when the potential reaches the threshold, after which V^t is reset to V_{reset}.

Instead of energy-intensive matrix multiplications, SpikeNet uses a "Masked Summation" operation $W[h]$. Since spikes h are binary (0 or 1), the operation indexes and accumulates relevant weight vectors:

$$\begin{aligned} h_S^{t,(k)} &= AGG(\{W^{(k)}[h_u^{t,(k-1)}], \forall u \in S^t\}) \\ h_v^{t,(k)} &= \delta^{(k)}(W^{(k)}[h_v^{t,(k-1)}] + h_S^{t,(k)}) \end{aligned} \tag{93}$$

where $\delta^{(k)}$ represents the LIF model and AGG is a permutation-invariant aggregation function (e.g., mean or sum).

The final node embedding z_v is generated by a learnable linear transformation that concatenates historical spikes from the final layer K over all T time steps:

$$z_v = Linear(h_v^{1,(K)} \parallel h_v^{2,(K)} \parallel \cdots \parallel h_v^{T,(K)}) \tag{94}$$

To address the non-differentiability of spikes, the model is trained using a surrogate Sigmoid function $\sigma(\alpha x)$ during backpropagation to approximate gradients.

7) Spatial-Temporal Feature Normalization (STFN)

The graph SNN [313] with spatial-Temporal Feature Normalization (STFN) framework consists of initial binary encoding, a spatial-temporal unfolding of message passing using LIF neurons, and a readout phase based on rate coding.

To process graph attributes $X \in \mathbb{R}^{N \times C}$ within a spiking paradigm, initial node signals are encoded into binary components $\{\tilde{X}_0, \tilde{X}_1, \ldots, \tilde{X}_{T-1}\}$ over a time window T. The layer-wise spiking graph convolution in the spatial-temporal domain is defined as:

$$\tilde{H}_t^n = \Phi(G_c(A, \tilde{H}_t^{n-1})W^n, \tilde{H}_{t-1}^n) \tag{95}$$

where $G_c(A, \tilde{H})$ represents feature propagation (e.g., GCN or GAT filters), W^n is the trainable weight matrix, and $\Phi(\cdot)$ represents the non-linear neuronal dynamics.

The framework utilizes the LIF neuron model. By applying the Euler method to the underlying differential equations, the membrane potential V update and spike generation for node i in layer n are formulated as:

$$\begin{aligned} V_{t+1}^{n,i} &= \kappa V_t^{n,i}(1 - \tilde{H}_t^{n,i}) + \sum_j W_{ij}^n G_c(A, \tilde{H}_{t+1}^{n-1,j}) \\ \tilde{H}_{t+1}^{n,i} &= g(V_{t+1}^{n,i} - V_{th}) \end{aligned} \tag{96}$$

where κ is a decay factor, V_{th} is the firing threshold, and $g(\cdot)$ is the Heaviside step function. For linear graph filters, the order of operations can be exchanged to turn multiplications into additions, significantly reducing overhead.

To coordinate membrane potential distributions with the firing threshold, STFN normalizes inputs S_t across both the feature dimension C and temporal dimension T for each node v:

$$\begin{aligned} \hat{S}_{k,v}^t &= \rho V_{th} \frac{S_{k,v}^t - E[S^v]}{\sqrt{Var[S^v] + \epsilon}}, \\ Y_{k,v}^t &= \lambda_{k,v} \hat{S}_{k,v}^t + \gamma_{k,v}. \end{aligned} \tag{97}$$

The mean $E[S^v]$ and variance $Var[S^v]$ are statistically computed as:

$$E[S^v] = \frac{1}{CT} \sum_{t=0}^{T-1} \sum_{k=0}^{C-1} S_{k,v}^t \tag{98}$$

This normalization ensures node features follow a distribution centered around the threshold, facilitating stable gradient-descent training.

The output spikes are transformed into high-level representations through a readout function R based on the firing rate:

$$\hat{y}_v = R\left(\frac{1}{T} \sum_{t=0}^{T-1} \tilde{h}_t^v\right) \tag{99}$$

The model is optimized using a cross-entropy loss with gradient substitution (surrogate gradients) to handle the non-differentiable nature of discrete spikes.

8) Graph Spiking Attention Network (GSAT)

The Graph Spiking Attention Network (GSAT) [314] framework utilizes a Graph Spiking Attention Layer (GSAL) that replaces the standard self-attention mechanism with a bio-inspired spiking process to learn sparse graph edges. The input graph $G(A, X)$ consists of adjacency matrix A and node features X. Continuous node representations h_i are first computed as:

$$h_i = x_i W, \quad i = 1, 2, \ldots, n \tag{100}$$

where W is a learnable parameter matrix. To process these in the spiking domain, a Probability-based Poisson encoder transforms h_i into discrete spike trains $z_i^{(t)}$ over T time steps:

$$z_{ik}^{(t)} = \begin{cases} 1 & \text{if } h_{ik} \geq \lambda_{ik}^{(t)} \\ 0 & \text{otherwise} \end{cases} \tag{101}$$

where $\lambda^{(t)}$ is a random vector used for sampling.

The encoded spike trains $z_i^{(t)}$ and $z_j^{(t)}$ are fed into Spiking Neurons to determine the attention between nodes i and j. This involves three core operations:

- **Charging:** The membrane potential $m_{ij}^{(t)}$ is updated by integrating the input spikes weighted by a trainable vector θ:

$$m_{ij}^{(t)} = v_{ij}^{(t-1)} + [z_i^{(t)} \parallel z_j^{(t)}]\theta^T \tag{102}$$

- **Fire:** A binary spike $c_{ij}^{(t)}$ is generated if the potential exceeds the threshold μ:

$$c_{ij}^{(t)} = \begin{cases} 1 & \text{if } m_{ij}^{(t)} \geq \mu \\ 0 & \text{otherwise} \end{cases} \tag{103}$$

- **Reset:** The membrane potential is reset using a soft reset function for the next time step:

$$v_{ij}^{(t)} = m_{ij}^{(t)} - \mu c_{ij}^{(t)} \tag{104}$$

The final attention score c_{ij} is the average firing rate over T time steps: $c_{ij} = \frac{1}{T}\sum_{t=1}^{T} c_{ij}^{(t)}$. To obtain the final attention coefficients α_{ij}, GSAT employs symmetric normalization (though other methods like Softmax can be used):

$$\alpha_{ij} = \frac{c_{ij}}{\sqrt{\sum_{k\in N_i} c_{ik}}\sqrt{\sum_{k\in N_j} c_{ki}}} \tag{105}$$

The updated node representation h_i' is then computed by aggregating messages from neighbors weighted by these sparse coefficients:

$$h_i' = \sigma\left(\sum_{j\in N_i} \alpha_{ij} h_j\right) \tag{106}$$

where σ is a non-linear activation function.

E. OVERALL DIRECTION OF SNN IN GRAPH PROBLEMS

GNNs have become an indispensable paradigm for learning over non-Euclidean domains [315], enabling effective reasoning over structured data such as molecules [316], social networks [317], and 3D scenes [318]. While conventional GNNs rely on real-valued message passing, recent advances have extended these ideas to the spiking domain, giving rise to spiking GNNs [307]. These architectures integrate temporal spiking dynamics with topological representations, thereby enabling energy-efficient processing of graph-structured inputs on neuromorphic hardware. Representative research directions include:

- *Node classification:* Node classification is one of the most well studied subject in GNN studies [315], aims to assign labels to individual vertices within a graph by exploiting both node attributes and relational structure. SGNNs have been developed to perform this task under citation network benchmarking (such as Cora [315], Pubmed [319], etc.) using BPTT and rate coding mechanisms [307] (or temporal coding in the case of [309]), enabling energy-efficient inference on large-scale relational datasets while preserving the structural inductive biases inherent to graph-based learning. Despite its maturity as a research topic, node classification remains a de facto benchmark for advancing a variety of research directions in spiking graph learning. It continues to play a pivotal role in evaluating novel SGNN architectures [307], [308], [309], [310], [320], [311], [312], [313], [314], hardware implementation [321] and in establishing theoretical foundations for solving graph-structured problems using spiking neurons [322], [323].
- *Image classification:* In the context of computer vision, SGNNs have been applied to classify images by representing pixels [309], [311] or superpixels [307] as nodes and modeling spatial relationships through graph edges. Spiking-based graph processing enables the extraction of discriminative features using event-driven mechanism, offering an energy-efficient alternative for visual recognition tasks without relying on dense convolutional operations.
- *Application-driven studies:* In addition to methodological advances, SGNNs have been explored in a variety of application-oriented contexts, including electroencephalogram-based (EEG-based) auditory attention detection [324] and brain–computer interfaces [325], as well as tactile object recognition using neuromorphic sensory data [326], among others.

IX. APPLICATIONS OF SNN IN NATURAL LANGUAGE PROCESSING

A. WHAT IS NATURAL LANGUAGE PROCESSING?

Natural Language Processing (NLP) aims to enable machines to process, understand, and generate human language [1], [327], [328]. Formally, NLP seeks to learn a mapping $f_\theta : \mathcal{X} \to \mathcal{Y}$, where $\mathcal{X}$ denotes a space of discrete token sequences and $\mathcal{Y}$ represents task-dependent outputs such as labels, structured predictions, or generated sequences. Given an input sentence represented as a sequence of tokens:

$$\mathbf{x} = (x_1, x_2, \ldots, x_T), \tag{107}$$

where T is the sequence length and $x_t \in \mathcal{V}$ belongs to a vocabulary $\mathcal{V}$ of size $|\mathcal{V}|$, the objective is to model linguistic structure, contextual dependencies, and semantic meaning across positions. Unlike images, language is inherently sequential and discrete. The ordering of tokens is essential:

$$(x_1, x_2) \neq (x_2, x_1). \tag{108}$$

Thus, NLP models must preserve sequence structure and capture long-range dependencies.

1) Text Classification

Text classification [329] assigns a label to an entire sequence $y = f_\theta(x_1, \ldots, x_T), \quad y \in \{1, \ldots, K\}$. Examples include sentiment analysis and intent detection. The model must aggregate contextual information across all tokens $\mathbf{h} = \Phi(x_1, \ldots, x_T)$, where $\mathbf{h}$ is a sequence-level representation that can be used for classification, such as:

$$y = \text{softmax}(\mathbf{W}\mathbf{h} + \mathbf{b}). \tag{109}$$

2) Language Understanding

Language understanding [330] involves structured or contextual interpretation of text, often requiring token-level predictions $\mathbf{y} = (y_1, \ldots, y_T)$, $y_t = f_\theta(x_1, \ldots, x_T)_t$. Tasks such as named entity recognition or syntactic parsing require modeling contextual dependencies $p(y_t \mid x_1, \ldots, x_T)$.

3) Topic Modeling

Topic modeling [331] aims to discover latent semantic themes within a corpus. A document is represented as a bag-of-words vector $\mathbf{d} \in \mathbb{N}^{|\mathcal{V}|}$, and modeled as a mixture of latent topics:

$$p(\mathbf{d}) = \sum_{k=1}^{K} p(z = k) \prod_{i=1}^{|\mathcal{V}|} p(w_i \mid z = k)^{d_i}. \tag{110}$$

In neural formulations, topic representations can be learned as $\mathbf{z} = f_\theta(\mathbf{d})$, where $\mathbf{z}$ captures latent semantic structure.

4) Speech Recognition

Speech recognition [332] maps acoustic signals to text sequences. Given a continuous signal $\mathbf{s}(t) \in \mathbb{R}$, features such as spectrograms are extracted $\mathbf{X} \in \mathbb{R}^{F \times T}$, where F denotes frequency bins and T denotes time steps. The task is to predict a token sequence $(x_1, \ldots, x_T) = f_\theta(\mathbf{X})$, often modeled as $p(x_1, \ldots, x_T \mid \mathbf{X})$.

B. LIMITATIONS OF NAIVE NLP MODELING

As noted in previous works [327], [328], a naive approach is to encode a sequence as a fixed-dimensional vector and apply fully connected layers $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$, where $\mathbf{x}$ may represent concatenated or one-hot encoded tokens. However, this approach presents several limitations:

- **Parameter explosion**: For one-hot encoding, $\mathbf{x}_t \in \{0, 1\}^{|\mathcal{V}|}$, leading to weight matrices of size $\mathbf{W} \in \mathbb{R}^{M \times |\mathcal{V}|}$, which becomes inefficient for large vocabularies.
- **Loss of sequential structure:** Flattening or aggregating tokens ignores order-sensitive dependencies $f(x_1, x_2) = f(x_2, x_1)$, which is undesirable in language modeling.
- **Limited context modeling:** Fully connected layers lack mechanisms for modeling long-range dependencies $p(x_t \mid x_1, \ldots, x_{t-1})$ cannot be efficiently captured without structured recurrence or attention.

To address these issues, NLP models use structured mechanisms such as embeddings [327], recurrence [66], convolution over sequences [333] and attention [68]. First, discrete tokens are mapped to dense embeddings $\mathbf{e}_t = \mathbf{E}x_t$, where $\mathbf{E} \in \mathbb{R}^{d \times |\mathcal{V}|}$. Sequence representations are then computed using structured operators $\mathbf{h}_t = \Phi(\mathbf{e}_1, \ldots, \mathbf{e}_T)$, where Φ may represent recurrent dynamics [66], temporal convolution [333], or self-attention [68].

C. SELF-ATTENTION AND THEIR SPIKING VARIANTS IN NLP PROBLEMS

Attention mechanisms [68] have become a fundamental component in modern deep learning architectures when tackling NLP problems, enabling models to selectively emphasize informative spatial, temporal, or channel-wise features. Inspired by their success in ANNs, recent studies have explored how attention mechanisms can be adapted to SNNs while preserving event-driven computation and temporal dynamics.

However, directly transferring attention designs from ANNs to SNNs is non-trivial due to the discrete spike representation, temporal accumulation of membrane potentials, and constraints imposed by neuromorphic hardware. Consequently, various works have proposed specialized attention formulations tailored for spike-based processing, including spike-driven self-attention, temporal attention modules, and lightweight channel–spatial attention mechanisms compatible with surrogate-gradient training.

To provide a structured overview, Table 6 summarizes representative attention variants developed for SNNs, highlighting their type, integration strategy, computational characteristics, and compatibility with neuromorphic principles.

1) Vanilla Self-Attention

To efficiently tackle NLP problems, prior work in traditional deep learning (DL) had proposed self-attention [68]. Given a feature matrix of $\mathbf{X} \in \mathbb{R}^{N \times d}$, where $N = HW$ is the number of spatial positions, queries, keys, and values are computed as [68]:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V. \tag{111}$$

The attention weights are computed using scaled dot-product attention $\mathbf{A}$ with $\alpha_i \in \mathbf{A}$ and the output representation $\mathbf{Y}$ can be quantified with each spatial location $\mathbf{y}_i \in \mathbf{Y}$ attends to all other locations, enabling global context modeling as:

$$\mathbf{A} = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right), \mathbf{Y} = \mathbf{A}\mathbf{V}, \mathbf{y}_i = \sum_{j=1}^{N} \alpha_{ij}\mathbf{v}_j. \tag{112}$$

2) Spiking Self-Attention

To harmonize the self-attention mechanism with the event-driven paradigm of SNNs, [193] propose Spiking Self-Attention (SSA), where it is vastly applied in later works of event-driven transformer implementations [213], [214], [336], [191]. Traditional vanilla self-attention (VSA) is unsuitable for SNNs due to the floating-point matrix multiplications and the non-biological softmax function. Given an input feature sequence $X \in \mathbb{R}^{T \times N \times D}$, first generate binary spike-form Query (Q), Key (K), and Value (V) tensors:

$$\begin{aligned} Q &= SN_Q(BN(XW_Q)), \\ K &= SN_K(BN(XW_K)), \\ V &= SN_V(BN(XW_V)) \end{aligned} \tag{113}$$

TABLE 6: Summary of spiking self-attention variants in SNNs.

Category	Core Idea	Primary Advantage
Vanilla Self-Attention [68]	Scaled dot-product attention using dense matrix multiplications and softmax normalization. Key modification vs. vanilla attention: global similarity via $\text{softmax}(QK^\top/\sqrt{d})V$.	$O(N^2D)$ complexity; strong global modeling but unsuitable for spike-based hardware.
Spiking Self-Attention [193], [214]	Binary spike queries, keys, and values processed through LIF neurons. Key modification vs. vanilla attention: removes softmax; dot-products replaced by spike accumulation and scaling.	Multiplication-reduced computation; energy-efficient global attention.
Spike-Driven Self-Attention [194]	Element-wise spike masking replaces similarity computation. Key modification vs. vanilla attention: Hadamard product and column summation substitute dot-product attention.	Linear complexity $O(ND)$; multiplication-free inference.
Average Spiking Rate Attention [192]	Attention computed via steady-state average spiking rate equilibrium. Key modification vs. vanilla attention: maps temporal spike dynamics to fixed-point surrogate attention formulation.	Enables scalable training via implicit differentiation.
Spiking Receptance Weighted KV [334]	Recurrent attention using decay-based state accumulation. Key modification vs. vanilla attention: eliminates pairwise token interaction; sequential receptive gating replaces attention matrix.	Linear $O(T)$ complexity; streaming-compatible computation.
Masked Spiking Self-Attention [335]	ANN-to-SNN converted attention with random spike masking. Key modification vs. vanilla attention: attention scores represented as spikes; stochastic masking prunes redundant events.	Up to 26.8% energy reduction with minimal accuracy loss.
Spike-Equivalent Self-Attention [160]	Mathematically equivalent spiking reformulation of quantized attention. Key modification vs. vanilla attention: decomposes attention into activation-weight and activation-activation spike operations.	Lossless ANN-to-SNN conversion with exact equivalence guarantees.

where SN represents a LIF neuron layer and BN denotes Batch Normalization. By utilizing spike-form representations, the matrix dot-product calculation is simplified to logical AND and summation operations.

Unlike VSA, SSA discards the softmax function. Since Q and K are non-negative binary spikes, the resulting attention map is naturally non-negative, rendering softmax-based normalization redundant for spike sequences. The attention operation is defined as:

$$SSA'(Q,K,V) = SN\left(\frac{QK^TV}{s}\right) \tag{114}$$

where s is a scaling factor to control the range of the internal membrane potential and ensure stable surrogate gradients.

The omission of softmax allows for computational decomposability. When the sequence length N is larger than the head dimension d, they adopt a calculation order of $Q(K^TV)$, reducing the complexity to $O(Nd^2)$. The final output is obtained through a spiking linear projection:

$$SSA(Q,K,V) = SN(BN(Linear(SSA'(Q,K,V)))) \tag{115}$$

This mechanism ensures the network remains multiplication-free and highly energy-efficient while capturing global feature dependencies.v

3) Spike-Driven Self-Attention

To fully exploit the energy-efficiency of the spike-driven paradigm, [194] propose Spike-Driven Self-Attention (SDSA). Unlike VSA which relies on computationally expensive dot-products and softmax functions, SDSA utilizes mask and sparse addition operations to achieve linear complexity and multiplication-free inference.

Given a spike input feature sequence $S \in \mathbb{R}^{T\times N\times D}$, the Query ($Q$), Key ($K$), and Value ($V$) tensors are generated through learnable linear layers. Since S is a binary spike tensor, these linear operations are transformed into sparse synaptic additions (AC). The resulting potentials are passed through a spiking neuron layer (SN) to produce binary spike tensors $Q_S, K_S, V_S \in \{0,1\}^{T\times N\times D}$.

The SDSA module replaces the traditional similarity dot-product with a Hadamard product, which functions as an element-wise mask between spikes. The attention map is derived by performing a column-wise summation followed by a spiking neuron activation, which replaces the softmax and scaling functions. The output of SDSA is defined as:

$$SDSA(Q,K,V) = \text{SN}(\text{SUM}_c(Q_S \otimes K_S)) \otimes V_S \tag{116}$$

where $\otimes$ is the Hadamard product and SUM_c denotes column-wise summation. This mechanism effectively implements a "hard" attention score that masks unimportant channels in the sparse spike Value tensor V_S.

By removing the softmax bottleneck, SDSA facilitates a computation order that is linear in both token number N and channel dimension D. The computational complexity is reduced to $O(ND)$, as opposed to the $O(N^2D)$ complexity found in VSA. Because all internal operations are restricted to binary masks and additions, the energy consumption of the attention operator is negligible, typically requiring only $0.02ND$ additions due to the high sparsity of the spike tensors.

4) Average Spiking Rate Self-Attention

To facilitate scalable language modeling in SNNs, [192] propose a spiking attention mechanism that leverages the steady-state convergence of neurons. Unlike vanilla attention, which is fundamentally non-spiking,the mechanism operates

over T_{conv} time steps until the average spiking rate (ASR) reaches an equilibrium.

At each time step t, the module computes attention using input spikes $S_x(t)$, key spikes $S_K(t)$, and value spikes $S_V(t)$. The attention operation is defined as:

$$Attn(S_x(t), S_K(t), S_V(t)) = \pi(s{\cdot}Q(S_x(t))(S_K(t))^T){\cdot}S_V(t) \tag{117}$$

where $Q(S_x(t))$ is a real-valued query matrix, π is the softmax function and s is a scaling factor ($s = 1/\sqrt{d_k}$ with d_k is the encoding dimension of key). Because S_K and S_V are binary spike matrices, the $O(n^3)$ matrix multiplications are reduced to $O(n^3)$ accumulative (ACC) operations, significantly lowering computational overhead on neuromorphic hardware.

The ASR of a layer i at equilibrium, the spiking architecture converges toward equilibrium $a_i[t] \to a_i^*$ (in case of LIF neurons with bounded random error) and satisfies the following fixed-point relation:

$$a_i^* = \sigma\left(\frac{1}{V_{th}}(W_{(i-1)}a_{(i-1)}^* + b_i)\right) \tag{118}$$

For the spiking attention layer specifically, they also utilize a surrogate steady-state equation to map the SNN dynamics to a non-spiking surrogate architecture:

$$a_{attn}^* = \sigma\left(\frac{1}{V_{th}}(Attn(a_x^*, a_k^*, a_v^*) + b_{attn})\right) \tag{119}$$

where a_x^*, a_k^*, a_v^* represent the ASRs of the Query, Key, and Value components. This equilibrium state allows for efficient training via implicit differentiation, circumventing the non-differentiability of spiking functions while ensuring that the ASR of the attention layer approximates its vanilla non-spiking counterpart.

5) Spiking Receptance Weighted Key Value (S-RWKV)

To address the $O(T^2)$ complexity and recurrent incompatibility of self-attention in SNNs, [334] propose the Spiking Receptance Weighted Key Value (S-RWKV) module. Inspired by the Receptance Weighted Key Value (RWKV) model, this module achieves linear $O(T)$ complexity by replacing dynamic matrix-matrix multiplications with element-wise products and learnable decay mechanisms.

For each time step t, the spiking input $X[t] \in \mathbb{R}^{1\times d}$ undergoes linear transformations to generate Receptance (R), Key (K), and Value (V) tensors:

$$R = X[t]M_R, \quad K = X[t]M_K, \quad V = X[t]M_V \tag{120}$$

where $M_R, M_K, M_V \in \mathbb{R}^{d\times H}$ are learnable parameters. Unlike self-attention, which requires the full sequence, S-RWKV unrolls the input sequentially, making it inherently compatible with the event-driven nature of SNNs.

The module maintains historical context using two hidden states, $A[t]$ and $B[t]$, which accumulate information using a continuous decay strategy. The states are updated as follows:

$$A[t] = \exp(K[t]) \odot V[t] + \exp(W_d) \odot A[t-1] \tag{121}$$

$$B[t] = \exp(K[t-1]) + \exp(W_d) \odot B[t-1] \tag{122}$$

where $W_d \in \mathbb{R}^{1\times d}$ is a learnable decay vector. The output $Y[t+1]$ is determined by a normalized ratio of these states, gated by the sigmoid-activated receptance $\sigma(R[t])$ and weighted by a current-step decay factor W_f:

$$Y[t+1] = SN\left(\sigma(R[t]) \odot \frac{\begin{array}{l}\exp(W_f)\\ \odot \exp(K[t])\\ \odot V[t]\\ + A[t]\end{array}}{\begin{array}{l}\exp(W_f)\\ \odot \exp(K[t])\\ + B[t]\end{array}}\right) \tag{123}$$

where SN represents a spiking neuron layer.

By utilizing sparse spiking activations, S-RWKV transforms high-cost matrix multiplications into sparse additions. This architecture eliminates the need for a softmax bottleneck and allows computation to begin before a sequence is complete. Theoretical analysis indicates that S-RWKV contributes to a $32.2\times$ reduction in operations compared to vanilla Transformers, making it highly suitable for energy-efficient language generation on asynchronous hardware.

6) Spiking Self-Attention with Random Spike Masking

To leverage the energy efficiency of Spiking Neural Networks (SNNs) for vision tasks, [335] propose the Masked Spiking Transformer (MST) based on the ANN-to-SNN conversion method. The MST adopts a Swin Transformer backbone [337] and implements a fully spiking self-attention mechanism through IF neurons. The dynamics of the IF neurons in layer l at time-step t are governed by the membrane potential $v^l(t)$ and the generation of binary spikes $s^l(t)$:

$$v^l(t) = v^l(t-1) + W^l\theta^{l-1}s^{l-1}(t) - \theta^l s^l(t) \tag{124}$$

where W^l is the weight matrix and θ^l is the firing threshold. To align with the SNN framework, layer normalization is replaced with batch normalization to ensure the post-activation distribution remains consistent between the ANN and SNN during conversion.

The spiking self-attention module computes spike-form Query (Q_{spk}), Key (K_{spk}), and Attention score (A_{spk}) matrices. For an input $X[t]$ at time t, the projections and scores are defined as:

$$Q_{spk}[t] = IF(X[t] * W_q), \quad K_{spk}[t] = IF(X[t] * W_k) \tag{125}$$

$$A_{spk}[t] = IF\left(\frac{Q_{spk}[t] * K_{spk}^T[t]}{\sqrt{d}}\right) \tag{126}$$

where d is the feature dimension of the attention head. By representing the attention scores as a spike matrix, the model maintains a fully event-driven operation compatible with neuromorphic hardware.

Inspired by the quantal synaptic failures in biological nervous systems, MST also introduce the Random Spike Masking (RSM) method to prune redundant spikes. RSM applies a binary mask M to the Query, Key, and Attention spike matrices:

$$\hat{S} = S \odot M \tag{127}$$

where M is generated based on a predefined masking ratio. This method effectively exploits the redundancy in self-attention modules, enabling a reduction in power consumption of up to 26.8% at a 75% masking ratio without significantly impacting top-1 accuracy on complex datasets like ImageNet.

7) Spike-Equivalent Self-Attention (SESA)

To address the mathematical inequivalence between vanilla Transformers and SNNs, [160] propose Spike-Equivalent Self-Attention (SESA). SESA facilitates a lossless conversion from Quantized Vanilla Self-Attention (QVSA) to an event-driven spiking paradigm by utilizing the ST-BIF+ neuron model.

SESA decomposes the self-attention mechanism into two primary operations: Activation-Weight (AW) multiplication for linear projections and Activation-Activation (AA) multiplication for spiking dot products. For a linear layer with stationary weights W and dynamic input spikes $X_{s,t} \in \{0, \pm 1\}$, the accumulated output at equilibrium T_{eq} is given by:

$$O_{Teq} = \sum_{t=0}^{Teq} (W \cdot X_{s,t}) \tag{128}$$

The AA multiplication between spiking Query (Q_s) and Key (K_s) tensors is implemented via three matrix multiplications per time-step to preserve the temporal accumulation of features. The output O_t at time t is formulated using spike tracers S_Q and S_K, which record the historical spike counts:

$$O_t = S_{Q,t} K_{s,t}^T + Q_{s,t} S_{K,t}^T - Q_{s,t} K_{s,t}^T \tag{129}$$

This ensures that $\sum_{t=0}^{Teq} O_t = Q_q K_q$, achieving exact mathematical equivalence to the quantized analog counterpart.

To maintain the integrity of the attention map, researchers also introduce Spike-Softmax, which utilizes a differential algorithm to process the accumulated input X_T. The output spike at time t is defined as the difference between the current and previous normalized states:

$$O_{s,t} = \sigma(X_t) - \sigma(X_{t-1}) \tag{130}$$

where σ denotes the softmax function. By integrating SESA with Spike-Softmax and Spike-LayerNorm, SpikeZIP-TF [160] enables Transformer-based SNNs to achieve state-of-the-art accuracy on ImageNet and natural language understanding (NLU) benchmarks with low latency.

D. OVERALL DIRECTION OF SNN IN THE REALM OF NLP

NLP has witnessed transformative progress driven by deep neural architectures [338], culminating in large language models (LLMs) [339] that excel at understanding and generating human language. Recently, these successes have begun to inspire analogous developments within the spiking domain, giving rise to spiking neural approaches for text classification [340], [334], sequence modeling [334] and language understanding [341], [334]. Notable research directions include:

- *Text classification:* Text classification tasks, such as sentiment analysis and document categorization, have served as an accessible entry point for applying SNNs to NLP [340], [334]. Dedicated spike-based architectures and optimized text-to-spike encoding schemes have demonstrated competitive accuracy alongside substantial reductions in energy consumption, highlighting the practicality of SNNs for linguistic inference tasks.
- *Language understanding:* Inspired by the transformative impact of transformer-based models, spiking variants of language encoders such as bidirectional encoder representations from transformers (BERT) [341], [191] or generative pre-trained transformer (GPT) [334], [342] have been developed to process semantic and syntactic relationships using spike-driven computations, either from using ANN2SNN conversion [212], knowledge distillation from ANN [191] or spike-based backpropagation [334], [342]. These models preserve contextual dependencies via temporal spiking dynamics, offering an energy-efficient alternative to conventional transformer architectures.
- *Topic modeling and unsupervised learning:* Leveraging biologically plausible learning rules such as Hebbian learning paradigm [343], unsupervised SNN-based frameworks have been proposed for discovering latent thematic structures in corpora. These models naturally align with the event-driven nature of textual co-occurrence patterns, enabling efficient spike-based topic extraction.
- *Speech recognition:* Speech recognition seeks to map acoustic signals to linguistic representations, enabling machines to interpret spoken language [344]. Spiking implementations leverage the temporal nature of auditory signals by encoding waveform features as spike trains and processing them through event-driven neural dynamics [141].

X. APPLICATIONS OF SNN IN REINFORCEMENT LEARNING

A. WHAT IS REINFORCEMENT LEARNING?

Reinforcement learning (RL) [345] is a machine learning framework for solving sequential decision-making problems, where an agent learns an optimal behavior through interactions with an environment. At each interaction step, the agent receives a scalar reward signal that evaluates the quality of

its action. The learning objective is to determine a policy that maximizes the expected cumulative reward over time.

A RL problem is commonly formulated as a Markov Decision Process (MDP) [346] defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ denotes the action space, $P(s'|s,a)$ represents the transition probability

$$P(s'|s,a) = \Pr(s_{t+1} = s' \mid s_t = s, a_t = a),$$

$R(s,a)$ denotes the expected reward function

$$R(s,a) = \mathbb{E}[r_t \mid s_t = s, a_t = a],$$

and $\gamma \in [0,1)$ is the discount factor controlling the importance of future rewards. At time step t, the agent observes the current state $s_t \in \mathcal{S}$ and selects an action $a_t \in \mathcal{A}$ according to a policy π. A stochastic policy [347] is defined as:

$$\pi(a|s) = \Pr(a_t = a \mid s_t = s),$$

while a deterministic policy [348] can be written as $a_t = \pi(s_t)$. The interaction between the agent and the environment produces a trajectory of $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_T)$. The objective of RL is to maximize the expected discounted return [345] defined as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}.$$

Equivalently, the optimization objective can be written as:

$$J(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right].$$

A fundamental quantity in RL is the state-value function [349], defined as the expected return starting from state s under policy π:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[G_t \mid s_t = s\right].$$

Similarly, the action-value function (Q-function) [350] evaluates the expected return when taking action a in state s and following policy π afterwards:

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}\left[G_t \mid s_t = s, a_t = a\right].$$

The relationship between the value function and the Q-function is given by:

$$V^{\pi}(s) = \sum_{a\in\mathcal{A}} \pi(a|s) Q^{\pi}(s,a).$$

Both functions satisfy the Bellman expectation [351] equations:

$$V^{\pi}(s) = \sum_{a\in\mathcal{A}} \pi(a|s) \cdot \sum_{s'\in\mathcal{S}} P(s'|s,a)\left[R(s,a) + \gamma V^{\pi}(s')\right],$$

$$Q^{\pi}(s,a) = R(s,a) + \gamma \sum_{s'\in\mathcal{S}} P(s'|s,a) \sum_{a'\in\mathcal{A}} \pi(a'|s') Q^{\pi}(s',a').$$

The optimal policy π^* maximizes the expected return $\pi^* = \arg\max_{\pi} J(\pi)$. Correspondingly, the optimal value functions are defined as:

$$V^*(s) = \max_{\pi} V^{\pi}(s), \quad Q^*(s,a) = \max_{\pi} Q^{\pi}(s,a).$$

These optimal functions satisfy the Bellman optimality equations [351]:

$$V^*(s) = \max_{a\in\mathcal{A}}\left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s')\right],$$

$$Q^*(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a) \max_{a'} Q^*(s',a').$$

Many RL algorithms aim to approximate these functions in order to derive the optimal policy

$$\pi^*(s) = \arg\max_{a} Q^*(s,a).$$

RL has been successfully applied to a wide range of sequential decision-making tasks. Prominent application domains include game playing [352], robotics control [353], autonomous driving [354], resource allocation [355], and recommendation systems [356]. The combination of RL with deep neural networks, known as Deep RL (DRL) [357], enables the approximation of value functions or policies using parameterized models which can be optimized using gradient-based methods.

B. EVOLUTION OF REINFORCEMENT LEARNING METHODS

The development of RL algorithms has progressed through several major stages, evolving from tabular dynamic programming methods to modern deep reinforcement learning approaches capable of solving high-dimensional problems. This evolution is primarily driven by improvements in value estimation, policy optimization, and function approximation techniques.

1) Dynamic Programming Methods

Early theoretical foundations of reinforcement learning originate from dynamic programming (DP) [358], which assumes complete knowledge of the environment model (also termed model-based RL in other works [359]), i.e., the transition probability $P(s'|s,a)$ and reward function $R(s,a)$. Under this setting, optimal value functions can be computed using iterative Bellman updates. The value iteration algorithm updates the value function according to

$$V_{k+1}(s) = \max_{a\in\mathcal{A}}\left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V_k(s')\right].$$

Similarly, policy iteration alternates between two steps: policy evaluation and policy improvement. During policy evaluation, the value function is computed as

$$V^{\pi}(s) = \sum_{a} \pi(a|s) \sum_{s'} P(s'|s,a)\left[R(s,a) + \gamma V^{\pi}(s')\right].$$

The policy is then improved using

$$\pi'(s) = \arg\max_a Q^\pi(s,a),$$

where

$$Q^\pi(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^\pi(s').$$

While dynamic programming methods guarantee convergence to the optimal solution, they require exact knowledge of the environment dynamics and are computationally expensive for large state spaces.

2) Temporal Difference Learning

To address the limitation of requiring a model of the environment, Temporal Difference (TD) learning [360] was introduced. TD methods estimate value functions directly from sampled transitions without explicit knowledge of $P(s'|s,a)$. The TD(0) update rule for estimating the value function is given by:

$$V(s_t) \leftarrow V(s_t) + \alpha \left[r_t + \gamma V(s_{t+1}) - V(s_t) \right],$$

where

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

is known as the temporal difference error.

One of the most influential model-free RL algorithms is Q-learning [361], which directly estimates the optimal action-value function using the update

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha \begin{bmatrix} r_t \\ + \gamma \max_{a'} Q(s_{t+1},a') \\ - Q(s_t,a_t) \end{bmatrix}.$$

Under appropriate exploration conditions, Q-learning converges to the optimal action-value function

$$Q(s,a) \to Q^*(s,a).$$

Another widely used method is SARSA [362], which updates the Q-function using the action actually taken:

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha \begin{bmatrix} r_t \\ + \gamma Q(s_{t+1},a_{t+1}) \\ - Q(s_t,a_t) \end{bmatrix}.$$

These algorithms are typically implemented in a tabular form, where the value function is stored explicitly for each (s,a) pair.

3) Function Approximation

When the state space $\mathcal{S}$ becomes large or continuous, storing value functions in tabular form becomes infeasible. To address this issue, RL algorithms began using parameterized function approximators [350], [363], [364].

Instead of representing the value function explicitly, it is approximated by a parameterized model:

$$V_\theta(s) \approx V^\pi(s), \quad Q_\theta(s,a) \approx Q^\pi(s,a),$$

where θ denotes the parameters of the approximator. The parameters are optimized by minimizing a squared Bellman error objective

$$\mathcal{L}(\theta) = \mathbb{E}\left[\left(r_t + \gamma \max_{a'} Q_\theta(s_{t+1},a') - Q_\theta(s_t,a_t) \right)^2 \right].$$

Linear function approximators [365] were initially used, where:

$$Q_\theta(s,a) = \theta^T \phi(s,a),$$

with $\phi(s,a)$ representing a feature vector.

4) Policy Gradient Methods

Another important direction in RL research is the direct optimization of policies. Instead of learning value functions, policy gradient methods [366] parameterize the policy itself as $\pi_\theta(a|s)$. The objective function is defined as:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right].$$

The gradient of this objective can be derived using the policy gradient theorem [366]:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a_t|s_t) Q^{\pi_\theta}(s_t,a_t) \right]$$

where the parameters are then updated as $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$. A commonly used estimator replaces the Q-function with the advantage function

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s).$$

5) Actor-Critic Methods

Actor–critic methods [363] combine value-based and policy-based approaches. The policy (actor) is parameterized as $\pi_\theta(a|s)$, while a value function (critic) estimates

$$V_w(s) \approx V^\pi(s).$$

The critic is trained using temporal difference learning

$$\delta_t = r_t + \gamma V_w(s_{t+1}) - V_w(s_t).$$

The actor parameters are updated using the gradient

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a_t|s_t) \delta_t \right].$$

This framework significantly improves sample efficiency and stability compared to pure policy gradient methods.

6) Deep Reinforcement Learning

The introduction of deep neural networks as function approximators led to the emergence of DRL [299]. In DRL, value functions or policies are parameterized using deep networks

$$Q_\theta(s,a), \quad V_\theta(s), \quad \pi_\theta(a|s).$$

A landmark algorithm in this field is the Deep Q-Network (DQN) [352], which minimizes the loss

$$\mathcal{L}(\theta) = \mathbb{E}\left[\left(y_t - Q_\theta(s_t,a_t) \right)^2 \right],$$

where the target value is defined as

$$y_t = r_t + \gamma \max_{a'} Q_{\theta^-}(s_{t+1}, a').$$

Here, θ^- denotes the parameters of a target network that is periodically updated to stabilize training. More recent algorithms further improve stability and performance through advanced policy optimization techniques such as trust region methods and clipped policy objectives. For example, the policy update in proximal policy optimization (PPO) [367] maximizes the objective

$$L^{\text{PPO}}(\theta) = \mathbb{E}\left[\min\left(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t\right)\right],$$

where

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}.$$

These modern reinforcement learning algorithms enable agents to learn complex behaviors in high-dimensional environments such as robotics [368], autonomous driving [369], and strategic games [370].

C. SPIKING IMPLEMENTATION OF POLICY GRADIENT IN RL

SNNs have increasingly been integrated with RL algorithms to leverage the temporal dynamics, event-driven computation, and energy efficiency of neuromorphic systems. In contrast to conventional deep neural networks, SNNs process information through discrete spike events governed by biologically inspired neuron dynamics, which introduces unique challenges when adapting gradient-based RL algorithms. As a result, existing approaches employ a wide range of strategies, including R-STDP [371], surrogate-gradient BPTT [368], and energy-based learning rules [372]. These implementations span both policy-gradient methods and value-based algorithms, translating classical RL frameworks into spike-driven computational models. Table 7 summarizes the main characteristics of representative spiking RL models, highlighting their underlying RL paradigm, neuron dynamics, training methodology, and key architectural contributions.

1) First attempt on spiking policy design

To the best of our survey, the first spiking policy officially published is constructed by mapping the theoretical framework of energy-based statistical models onto a network of LIF neurons [372]. The core of the model is a Restricted Boltzmann Machine where binary nodes are separated into visible and hidden layers. The visible layer is further partitioned into state nodes s and action nodes a. The energy function for a configuration of state s, action a, and hidden nodes h is defined as:

$$\begin{aligned} E(s,a,h;\theta) = &- \sum_i \sum_l s_i w_{il}^{sh} h_l \\ &- \sum_j \sum_l a_j w_{jl}^{ah} h_l \end{aligned} \tag{131}$$

where w_{il}^{sh} and w_{jl}^{ah} represent the undirected weights between state/hidden and action/hidden neurons, respectively. In this framework, the negative free-energy $-F(s,a;\theta)$ serves as an approximator for the state-action value function $Q(s,a)$. The equilibrium free-energy is expressed as:

$$\begin{aligned} F(s,a;\theta) = &- \sum_{i,l} s_i w_{il}^{sh} \hat{h}_l - \sum_{j,l} a_j w_{jl}^{ah} \hat{h}_l \\ &+ \sum_l [\hat{h}_l \ln \hat{h}_l + (1-\hat{h}_l)\ln(1-\hat{h}_l)] \end{aligned} \tag{132}$$

where $\hat{h}_l = \sigma(\sum_i w_{il}^{sh} s_i + \sum_j w_{jl}^{ah} a_j)$ is the probability that hidden node l is active, and σ is the logistic sigmoid function.

To implement this in a SNN, binary nodes are replaced by LIF neurons. The membrane potential V_m of these neurons evolves according to:

$$\begin{aligned} \tau_m \frac{dV_m}{dt} = &- (V_m - V_{rest}) \\ &+ R_m(I_e + I_{syn}(t) + I_{noise}) \end{aligned} \tag{133}$$

where τ_m is the time constant, V_{rest} is the resting potential, and I_{syn} is the sum of synaptic currents from pre-synaptic spikes.

The bridge between continuous-time spikes and the discrete-time free energy-based reinforcement learning (FERL) framework is the average instantaneous pseudo-free-energy (iFE). This quantity allows the free energy to be calculated sequentially using raw spike trains $s(t), a(t), h(t)$. The instantaneous estimate $\hat{f}(t;\theta)$ is defined as:

$$\begin{aligned} \hat{f}(t;\theta) = &E(s(t),a(t),h(t);\theta) \\ &+ \sum_l \begin{bmatrix} h_l(t)\ln \hat{h}_l(t) \\ + (1-h_l(t))\ln(1-\hat{h}_l(t)) \end{bmatrix} \end{aligned} \tag{134}$$

where $\hat{h}_l(t)$ is a low-pass filtered estimate of the hidden neuron's firing probability, updated as $\hat{h}_l(t) \leftarrow \hat{h}_l(t-\Delta t) + \alpha_h(h_l(t) - \hat{h}_l(t-\Delta t))$.

The parameters θ (the weights) are updated using a SARSA-like [362] learning rule modulated by a global temporal difference (TD) error. The update rule for weights w takes the form:

$$\begin{aligned} \Delta w \propto &(r_{k+1} + \gamma Q(s_{k+1}, a_{k+1}) - Q(s_k, a_k)) \\ &\cdot \text{Local Hebbian Term} \end{aligned} \tag{135}$$

Specifically, for state-hidden weights w_{il}^{sh}, the update is:

$$\begin{aligned} w_{il}^{sh} \leftarrow w_{il}^{sh} + \alpha(&r_{k+1} - \gamma F(s_{k+1}, a_{k+1};\theta) \\ &+ F(s_k, a_k;\theta)) s_i \hat{h}_l \end{aligned} \tag{136}$$

This represents a local Hebbian learning rule modulated by the global reward-based TD error.

For history-dependent tasks, a memory layer is introduced with fixed recurrent connections w^{mm} following a circular Gaussian distribution to maintain long-lived activity patterns:

$$w_{ij}^{mm} = g_s \exp((\cos(x_i - x_j) - 1)/g_w) - g_b \tag{137}$$

where x_i is the position of the neuron on a circle, allowing the network to retain information from past observations.

TABLE 7: Summary of spiking reinforcement learning implementations.

Model	RL Framework	Neuron Model	Training Method	Key Idea / Contribution
Spiking RBM Policy [372]	Free-Energy RL (FERL) / SARSA	LIF	Hebbian learning modulated by TD-error	Maps Restricted Boltzmann Machine energy model to a spiking network. Uses instantaneous pseudo-free-energy (iFE) computed from spike trains to approximate the Q-function.
SVPG [371]	Policy Gradient (REINFORCE)	Stochastic SRM neurons	Reward-modulated STDP (R-STDP)	Implements a variational policy gradient in a recurrent WTA circuit. Demonstrates equivalence between policy gradient and local R-STDP learning rules.
Spiking PPO [373]	PPO (actor–critic)	LIF	Surrogate-gradient STBP	Integrates discrete-time spiking dynamics with PPO training. Uses surrogate gradients to enable end-to-end backpropagation through spike events.
Spiking PPO (Mission Planning) [374]	PPO (actor–critic)	LIF	Gradient descent with clipped objective	Applies SNN-based actor–critic to satellite mission scheduling formulated as a MDP, exploiting temporal spike processing for visibility windows.
Spiking DDPG [375]	DDPG (continuous control)	LIF	STBP with surrogate gradients	Hybrid architecture with spiking actor and deep critic. Enables neuromorphic deployment while retaining gradient-based training.
PopSAN [368]	TD3 / PPO (actor–critic)	Current-based LIF	Surrogate-gradient BPTT	Uses population coding for state encoding and rate-based decoding for continuous actions, enabling efficient spike-based policy networks.
DSQN [376], [377], [216]	Deep Q-Network	LIF + non-spiking LI output	Surrogate-gradient BPTT	Spiking feature extraction with non-spiking output neurons to estimate continuous Q-values from membrane potentials.
BrainQN [378]	Deep Q-Network	LIF	Surrogate gradient learning	Uses spike-frequency encoding and rate-based Q-value decoding to perform spatiotemporal reinforcement learning.

2) Spiking Variational Policy Gradient (SVPG)

The Spiking Variational Policy Gradient (SVPG) [371] model is constructed by implementing a variational policy gradient framework within a network of stochastic spiking neurons organized into competitive circuits.

The fundamental building block is the spike response model (SRM) neuron, a stochastic variant of the LIF model. The firing probability $\rho(l)$ at spike time step l depends exponentially on the membrane potential $u(l)$ and an inhibitory term $I(l)$ from the Winner-Take-All (WTA) circuit:

$$\rho(l) = \exp\{u(l) - I(l)\} \tag{138}$$

The membrane potential $u(l)$ is determined by the weighted sum of filtered presynaptic spike trains S_j:

$$u(l) = \sum_{j \in N(\cdot)} w_j \int_0^\infty \kappa(y) S_j(l-y) dy + b \tag{139}$$

where $\kappa(y)$ is the excitatory postsynaptic potential (EPSP) kernel and b is the intrinsic excitability.

The model utilizes an recurrent WTA (RWTA) network consisting of three types of neurons: state neurons s, hidden WTA circuits h, and an action WTA circuit a. The network is fully connected with symmetric weights W and self-activation parameters b. The RL policy is defined through an energy function $E(v)$ over the firing states v:

$$E(v) = v^T W v + b^T v \tag{140}$$

The policy $\pi(v_a|s)$ is the marginal distribution of the joint action-hidden probability distribution $p(v_a, v_h|s) = \frac{1}{Z(s)} \exp\{E(v)\}$.

To handle the intractability of calculating the marginal distribution with many hidden neurons, mean-field inference is used to approximate the distribution. For each neuron i, the approximated firing probability q_i is updated iteratively:

$$q_i = \frac{1}{Z(q_{G(i)})} \exp\{w_{row,i}^T q + w_{col,i}^T q + b_i\} \tag{141}$$

where $G(i)$ represents neurons in the same WTA circuit. The authors prove that the fixed point of the RWTA network's spiking dynamics is equivalent to this mean-field inference solution.

The model transforms the global REINFORCE policy gradient [379] into a local R-STDP rule. The parameter update $\Delta w_{ij} = R(l) \cdot STDP(l)$ is shown to be equivalent to an approximated policy gradient when the STDP coefficients are set as follows:

$$\begin{pmatrix} W_{pre}, W_{post}, \\ A^+(w_{ij}), A^-(w_{ij}) \end{pmatrix} = \langle v_i, v_j, -1/\hat{\rho}, -1/\hat{\rho} \rangle \tag{142}$$

The resulting weight update for a synapse w_{ij} under reward r and discount γ is:

$$\frac{\partial J(\pi)}{\partial w_{ij}} = \sum_t \gamma^t r_t [q_i(v_j - q_j) + q_j(v_i - q_i)] \tag{143}$$

This enables the network to optimize the global expected return $J(\pi)$ using only local firing states v and firing probabilities q modulated by the environmental reward.

3) Spiking PPO

The spiking version of PPO [367] model is constructed by integrating discrete-time spiking neuron dynamics into an on-policy reinforcement learning framework, utilizing surrogate gradients for direct training [373].

The fundamental computational unit is the LIF neuron. For algorithmic implementation, the continuous membrane potential $v(t)$ is discretized into a recurrent formulation where the potential at tick $t+1$ is defined as:

$$v(t+1) = v(t)e^{-\frac{dt}{\tau}} + RI(t) \tag{144}$$

where τ is the time constant, dt is the integration step, R is membrane resistance, and $I(t)$ is the pre-synaptic current. The input current to neuron i in layer n is calculated as:

$$I_i^n(t) = b_i^n + \sum_{j=1}^{l(n-1)} w_{ij}^n z_j^{n-1}(t-1) \tag{145}$$

where b is the bias and w are synaptic weights. A spike $z(t+1)$ is generated when the potential reaches a threshold v_{thr}:

$$z(t+1) = \begin{cases} 1 & \text{if } v(t) \geq v_{thr} \\ 0 & \text{otherwise} \end{cases} \tag{146}$$

Upon firing, the model implements a hard-reset mechanism, returning $v(t+1)$ to a resting potential v_{rest}.

To overcome the non-differentiable nature of the spike function, the model employs Spatio-Temporal BackPropagation (STBP). A rectangular surrogate gradient function $h(v)$ is used to approximate the derivative during the backward pass:

$$h(v) = \frac{1}{a}\text{sign}\left(|v - v_{thr}| < \frac{a}{2}\right) \tag{147}$$

where a represents the peak width of the derivative.

The network utilizes a feed-forward structure consisting of a shared feature extractor and two separate output heads (the action head and the critic head).

- **Input Encoding:** Continuous sensor readings are fed directly to the first layer as currents, where they are multiplied by learnable weights to generate spikes for only one timestep.
- **Output Neurons:** To capture environmental complexity within a single tick (timestep = 1), the output heads utilize non-spiking LIF neurons.

The parameters θ are optimized using the PPO-Clip objective function L, which restricts the divergence between the new policy π_θ and the old policy π_{θ_k}:

$$L(\theta_k, \theta, s, a) = \min \begin{pmatrix} r(\theta, \theta_k)A^{\pi_k}(s,a), \\ \text{clip}\begin{pmatrix} r(\theta, \theta_k), \\ 1-\epsilon, \\ 1+\epsilon \end{pmatrix} A^{\pi_k}(s,a) \end{pmatrix} \tag{148}$$

where $r(\theta, \theta_k) = \frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)}$ is the probability ratio and $A^{\pi_k}(s,a)$ is the advantage function.

In another approach, a spiking PPO for satellite mission planning [374] can be constructed by first formalizing the mission scheduling problem as a MDP and then implementing a specialized actor-critic architecture using spiking neurons.

The satellite mission planning process is modeled as a single-intelligence MDP quintuple $\langle S, A, r, P, \gamma \rangle$. The state space S is a joint representation of mission status s_I and satellite operation status s_{Sat}:

$$S = s_I \times s_{Sat} \tag{149}$$

where s_I includes the observation status (observed or unobserved), the next visible window times (w_{start_i}, w_{end_i}), and the proposed observation timings $(t_{obs_start_i}, t_{obs_end_i})$ for each mission. The action space A is defined by the decision to observe or not at each time step:

$$A = A_1 \times A_2 \times \cdots \times A_n, \quad A_n \in [0, 1] \tag{150}$$

where 0 represents an unobserved action and 1 represents an observed action.

The spiking component of the network utilizes the LIF neuron model. The membrane potential $V(t)$ evolves according to the differential equation:

$$\tau_m \frac{dV(t)}{dt} = -V(t) + RI(t) \tag{151}$$

where τ_m is the membrane time constant, R is resistance, and $I(t)$ is the input current. When $V(t)$ reaches a threshold V_{th}, the neuron emits a spike and resets its potential:

$$V(t) \geq V_{th} \Rightarrow V(t) = V_{reset} \tag{152}$$

This discrete spiking communication is used to capture the temporal dependencies inherent in pulse-like satellite visibility windows.

The model uses a shared custom feature extractor to transform raw observation data o into high-dimensional feature vectors f:

$$f = \phi_2(W_2 \cdot \phi_1(W_1 \cdot o + b_1) + b_2) \tag{153}$$

where ϕ represents ReLU activation functions, W represents weight matrices, and b denotes bias terms. This feature vector is then processed through SNN layers:

$$\begin{aligned} h_1 &= \phi_3(W_3 \cdot f + b_3), \quad S_1 = LIF(h_1) \\ h_2 &= \phi_4(W_4 \cdot s_1 + b_4), \quad S_2 = LIF(h_2) \end{aligned} \tag{154}$$

The output of the final spiking layer S_2 is mapped to the action distribution $\pi_\theta(a|s)$ in the actor network and the state-value $V_\phi(s)$ in the critic network through fully connected layers.

The construction utilizes a clipped objective function to ensure stable updates to the policy parameters θ:

$$L^{CLIP}(\theta) = \mathbb{E}_t \begin{bmatrix} \min(r_t(\theta)\hat{A}_t, \\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t) \end{bmatrix} \tag{155}$$

where $r_t(\theta)$ is the probability ratio between the new and old policy, and $\hat{A}_t$ is the advantage estimate. The total loss

function L for optimizing the combined actor and critic networks is:

$$L = L_{policy} + c_1 L_{value} - c_2 L_{entropy} \tag{156}$$

where c_1 and c_2 are weight coefficients for the value loss (error in state evaluation) and entropy loss (which encourages exploration), respectively.

4) Spiking DDPG Model

The spiking implementation of DDPG model [380] is constructed by partitioning the actor-critic architecture into two distinct neural paradigms: a spiking network for action generation and a deep network for policy evaluation [375].

The agent is composed of a spiking actor and a deep critic. While both are utilized during the training phase, only the spiking actor is deployed during inference to ensure compatibility with neuromorphic hardware. The actor generates optimal actions a given the state s, and the critic evaluates these actions by estimating the Q-value. The actor is optimized by minimizing a loss function $L = -Q(s, a)$ where Q is the output of the deep critic network.

The actor network consists of LIF neurons. To train the SNN, the model employs the STBP algorithm, which allows for the calculation of gradients through time. Because the spike function is non-differentiable, a rectangular function is used for gradient approximation:

$$\sigma'(u) \approx \frac{1}{a} \cdot \text{rect}\left(\frac{u - V_{th}}{a}\right) \tag{157}$$

where u is the membrane potential and V_{th} is the firing threshold.

To solve complex manipulation tasks, the construction utilizes a hierarchical learning scheme that decomposes the mission into two subgoals: Box-Reach and Box-Transfer. Each subgoal is defined by a specific reward function R designed to shape the agent's behavior.

For the reach task, the reward function is constructed to encourage an elliptical approach to the object, minimizing the risk of collision before grasping. This is achieved through weighted distance amplifiers a_{mx} and a_{my}:

$$R_{reach} = a_{mx}\Delta D_x + a_{my}\Delta D_y \tag{158}$$

where $a_{mx} > a_{my}$ prioritizes horizontal alignment before vertical approach.

A unique feature of the model's construction is the collinear grasping condition, which determines if the gripper is positioned to grasp the object successfully. This is calculated using the determinant area c of the triangle formed by the left finger (l), the box center (b), and the right finger (r):

$$c = \frac{1}{2}\left|(l_x - b_x)(b_y - r_y) - (b_x - r_x)(l_y - b_y)\right| \tag{159}$$

When $c = 0$, the three points are collinear, indicating a proper grasping orientation.

Sensory data from the environment (joint positions, velocities, and object coordinates) are collected as ROS messages and encoded into spikes to be processed by the actor. The state vector s for the reach task is defined as:

$$s = \{D_x, Dir_x, D_y, Dir_y, c, u_x, u_y, b_x, b_y\} \tag{160}$$

where D represents distance, Dir direction, u velocity, and b the object position.

5) Population-Coded Spiking Actor Network

The Population-Coded Spiking Actor Network (PopSAN) [368] model is constructed as a hybrid architecture consisting of a spiking actor network for energy-efficient inference and a deep critic network for optimal training through reinforcement learning.

Each dimension of the observation space s_i is encoded into the activity of a neuron population E_i using Gaussian receptive fields. The stimulation strength A_E for a neuron in the population is defined as:

$$A_E = \exp\left(-\frac{1}{2}\left(\frac{s_i - \mu}{\sigma}\right)^2\right) \tag{161}$$

where μ and σ are learnable parameters initialized to cover the observation space. These strengths are converted into spikes $o(t)$ through either probabilistic encoding or deterministic encoding using one-step soft-reset IF neurons.

The core SNN utilizes current-based LIF neurons. The state of a neuron in layer k at timestep t is governed by its synaptic current c and membrane voltage v:

$$\begin{aligned} c(t)^{(k)} &= d_c \cdot c(t-1)^{(k)} + W^{(k)} o(t)^{(k-1)} + b^{(k)} \\ v(t)^{(k)} &= d_v \cdot v(t-1)^{(k)} \cdot (1 - o(t-1)^{(k)}) \\ &\quad + c(t)^{(k)} \end{aligned} \tag{162}$$

where d_c and d_v are decay factors and $W^{(k)}$ represents the weight matrix. A spike $o(t)^{(k)}$ is generated when $v(t)^{(k)}$ exceeds a threshold, followed by a hard-reset.

The M-dimensional action a is generated by decoding the activity of output populations D_i over T timesteps. The firing rate $fr(i)$ is first computed:

$$fr(i) = \frac{\sum_{t=1}^{T} o(t)^{(K)}}{T} \tag{163}$$

The final real-valued action a_i is then calculated as a weighted sum of the firing rates:

$$a_i = W_d^{(i)} \cdot fr(i) + b_d^{(i)} \tag{164}$$

where W_d and b_d are learnable decoding weights and biases.

The model is trained by backpropagating gradients from a global loss function L (determined by DRL algorithms like TD3 [381] or PPO [367]) through the decoder, SNN, and encoder. To handle the non-differentiable spike function, a rectangular pseudo-gradient $z(v)$ is used:

$$z(v) = \begin{cases} 1 & \text{if } |v - V_{th}| < a \\ 0 & \text{otherwise} \end{cases} \tag{165}$$

The gradients for the SNN weights are collected across all timesteps T:

$$\nabla_{W^{(k)}} L = \sum_{t=1}^{T} o(t)^{(k-1)} \cdot \nabla_{c(t)^{(k)}} L \tag{166}$$

This allows the entire network, including the receptive field parameters μ and σ of the encoder, to be optimized using gradient descent.

D. SPIKING IMPLEMENTATIONS OF DEEP-Q NETWORK

1) Deep Spiking Q-Network (DSQN)

The Deep Spiking Q-Network (DSQN) [376], [377] is constructed by integrating LIF spiking neurons for feature extraction and Leaky Integrate (LI) non-spiking neurons for value estimation, enabling end-to-end reinforcement learning.

The fundamental processing units are LIF neurons. Their discrete-time dynamics are governed by charging, firing, and resetting equations:

$$\begin{aligned} H_t &= V_{t-1} + \frac{1}{\tau}(-(V_{t-1} - V_{reset}) + X_t) \\ S_t &= \Theta(H_t - V_{th}) \\ V_t &= H_t(1 - S_t) + V_{reset} S_t \end{aligned} \tag{167}$$

where H_t is the membrane voltage after charging, S_t is the output spike (1 if firing, 0 otherwise), V_t is the voltage after the firing/resetting phase, and X_t is the external input. $\Theta(x)$ represents the Heaviside step function.

To represent the continuous Q-value used in RL, the model employs non-spiking LI neurons in the final layer. These neurons are defined by setting the firing threshold V_{th} to infinity, removing the firing and resetting dynamics:

$$V_t = V_{t-1} + \frac{1}{\tau}(-(V_{t-1} - V_{reset}) + X_t) \tag{168}$$

The Q-value for a state-action pair (s, a) is decoded from the membrane voltage of these neurons over a simulation time T. The model utilizes the maximum membrane voltage (max-mem) statistic to ensure stability and robustness:

$$Q(s, a; \theta) = \max_{1 \le t \le T} V_t \tag{169}$$

This approach allows the network to capture the optimal representation of the value function from the temporal spike-train.

The DSQN follows an architecture similar to the traditional DQN but replaces ReLU-based layers with spiking layers. It typically consists of:

- **Input/Convolutional Layers**: Process high-dimensional sensory input (e.g., Atari frames) and convert it into spike-trains.
- **LIF Neuronal Layers**: Perform deep feature extraction using spike-driven communication.
- **LI Neuronal Layer**: The final output layer that generates Q-values for each possible action.

The network is trained by minimizing the mean-squared Bellman error:

$$L(\theta) = \mathbb{E}_{s,a,r,s'} \left[\begin{pmatrix} r + \gamma \max_{a'} Q(s', a'; \theta^-) \\ - Q(s, a; \theta) \end{pmatrix}^2 \right] \tag{170}$$

Gradients are calculated using spike-based backpropagation through time. To handle the non-differentiable firing function, a surrogate gradient $\sigma'(x)$ (such as the arctangent function) is used during the backward pass:

$$\sigma'(x) = \frac{1}{1 + (\pi x)^2} \tag{171}$$

The final weight updates $\frac{\partial Q}{\partial \theta_i}$ are accumulated across all simulation timesteps T for internal layers, while the output layer gradient is determined specifically at the timestep t' where the maximum voltage occurred.

On the aspect of practical deployment [216], DSQN can be constructed by replacing the traditional artificial neurons of a DQN with LIF neurons and adapting the parameters for execution on the Intel Loihi neuromorphic chip.

To bridge the gap between discrete spikes and continuous value estimation, the output layer consists of non-spiking neurons. This is achieved by setting the firing threshold of the output neurons to infinity. The Q-values are read directly from the membrane potentials of these output neurons after a simulation time T:

$$Q(s, a) = V_{output}(T) \tag{172}$$

The network state is reset after every inference step to maintain a clear comparison with traditional DQNs.

To deploy the DSQN on Loihi, the weights must be converted to 8-bit integers. During training, weight quantization is applied for the forward pass while maintaining floating-point precision for the backward pass. The quantized weight w_{quant} is derived as follows:

$$\begin{aligned} w_{quant} &= \left\lfloor \frac{w_{float}}{scale} \right\rfloor, \\ scale &= \frac{max(w_{float}) - min(w_{float})}{q_{max} - q_{min}} \end{aligned} \tag{173}$$

where $q_{max} = 254$ and $q_{min} = -256$ for mixed-sign weights. On the Loihi chip, the compartment current is further refined to account for hardware-specific decay terms δ_u and weight exponents:

$$\begin{aligned} u_i(t) = u_i(t-1) \cdot (2^{12} - \delta_u) \cdot 2^{-12} \\ + 2^{6+wgtExp} \cdot \sum_j w_{ij} \cdot s_j(t) \end{aligned} \tag{174}$$

The network is trained using backpropagation with surrogate gradients to minimize the temporal difference loss L:

$$L(s, a|\theta_i) = \begin{pmatrix} r + \gamma \max_{a'} Q(s', a'|\theta_i^-) \\ - Q(s, a|\theta_i) \end{pmatrix}^2 \tag{175}$$

where θ_i are the parameters of the Q-network and θ_i^- are the parameters of the target network, updated every few episodes to ensure training stability.

2) Brain Q-Network

The Brain Q-Network (BrainQN) [378] model is constructed by mapping the reinforcement learning objective of a DQN onto a SNN architecture capable of spatiotemporal information processing.

The fundamental unit of the BrainQN is the LIF neuron, which simulates biological neuron behavior by integrating synaptic inputs into a membrane potential $V(t)$ that undergoes natural decay. The continuous-time dynamics are defined as:

$$\tau_m \frac{dV(t)}{dt} = -V(t) + R_m I(t) \tag{176}$$

where τ_m is the membrane time constant, R_m is the leakage resistance, and $I(t)$ is the input current. In a discrete-time algorithmic implementation across layers n and ticks t, the membrane potential u for neuron i is updated as:

$$u^n_{t+1,i} = u^n_{t,i} f(o^n_{t,i}) + \sum_j w^n_{ij} o^{n-1}_{t+1,j} + b^n_i \tag{177}$$

where $f(\cdot)$ is an attenuation function simulating leakage, w represents synaptic weights, and o is the binary spike output ($o \in \{0, 1\}$). A spike is triggered if the potential exceeds a threshold V_{th}.

The model utilizes frequency encoding to translate environmental states into spike trains. The Q-values for actions are derived from the firing rates of the output layer. Specifically, the Q-value Q_T for a state at timestep T is calculated as the weighted sum of the average spikes generated by the second-to-last layer $(L-1)$ over N timesteps:

$$Q_T = w_L \frac{1}{N} \sum_{t=1}^{N} o_{t,L-1} \tag{178}$$

where w_L represents the learnable weights of the final layer. The agent then selects the action A_T that maximizes this value: $A_T = \arg\max_a Q(S_T, a; w)$.

Direct training is achieved via Surrogate Gradient Learning (SGL), which backpropagates gradients through both the spatial (layer) and temporal (timestep) domains. To address the non-differentiable nature of the spike function, an arc tangent surrogate function $H'(x)$ is used:

$$\Theta'(x) = \frac{\alpha}{2(1 + (\pi \alpha x)^2)} \tag{179}$$

This enables the model to integrate information across time and space, enhancing its ability to filter high-frequency noise.

Alternatively, the model can be constructed by training a conventional DQN with ReLU activations and converting it into an SNN using IF neurons. This process involves weight and threshold normalization to ensure the firing rates of the spiking neurons match the graded activations of the original analog neurons:

$$w_l \leftarrow \frac{V^{l-1}_{th}}{V^l_{th}}, \quad V^l_{th} \leftarrow 1 \tag{180}$$

where V_{th} is balanced across layers to ensure efficient information transmission.

E. OVERALL DIRECTION OF SNN IN RL

RL studies how agents learn to interact with an environment and maximize cumulative reward through trial-and-error experience [345], [359]. Owing to its generality and scalability, RL has demonstrated strong potential across a wide range of application domains, including (but not limited to) unmanned aerial vehicle (UAV) control [382], robotics [383], networking and communications [384], NLP [385], business intelligence [385], and healthcare [385]. Motivated by the temporal and event-driven nature of decision-making, recent research has begun to investigate spiking counterparts of classical model-free RL formulations [359], leading to a growing body of work in spiking RL.

- *Spike-based policy optimization:* A prominent research direction leverages biologically inspired learning rules such as R-STDP [372], [371], as well as surrogate-gradient-based training [373], to construct spiking analogues of policy optimization algorithms. Representative examples include spiking variants of proximal policy optimization (PPO) [373], [371] and hybrid frameworks that combine spiking actors with conventional critics [375]. Beyond that, several studies had emphasized system-level and engineering considerations over algorithmic development contributions [368], [374].
- *Deep spiking Q-networks:* Another active line of research focuses on spiking adaptations of value-based methods, particularly DQN [376]. In these models, action-value functions are represented using spiking neurons through membrane potentials, firing rates, or surrogate-gradient formulations [377]. While much of the literature concentrates on architectural design and training strategies, recent work has also addressed fundamental challenges in deep spiking Q-learning, including robustness and generalization [378], [215], information vanishing across time [386], spike sparsity and efficiency [387], neuromorphic hardware deployment [216], and performance degradation in ANN-to-SNN conversion pipelines [158].

XI. HARDWARE ORIENTED SNN RESEARCH

The field of neuromorphic computing has emerged to address the limitations of Von Neumann architectures by bringing memory and computational elements into close proximity [388]. SNNs, often referred to as the third generation of neural networks, employ computational primitives that mimic biological neurons and synapses, leveraging temporal dynamics for improved performance and efficiency. Unlike traditional ANNs, SNNs are event-driven and utilize discrete binary spikes for communication [26].

A. GENERAL HARDWARE ARCHITECTURE AND MAPPING

Neuromorphic processors typically distribute memory and processing elements (PEs) across multiple neurocores [389], [388]. Each core contains local memory for weights and neuron states, control circuitry, and input/output interfaces.

Inter-core communication is predominantly handled via the Address Event Representation (AER) protocol [390], which transmits the unique address of a spiking neuron over an asynchronous Network on Chip (NoC) [391], [392].

1) Core Computation and Temporal Dynamics

Within a digital neurocore, a single PE typically updates all assigned neurons sequentially in a time-multiplexed manner [393], [394], [395], [396]. This approach reduces computational costs compared to ANNs because a spiking neuron update often simplifies to an addition and a comparison rather than a multiply-accumulate (MAC) operation [394], [59]. At each algorithmic time step t, the membrane potential V_{mj} for a neuron j in layer l is integration of synaptic weights $w_{i,j}$ from incoming spikes x_i [26]:

$$V_{mj}^{l}(t) = V_{mj}^{l}(t-1) \times \lambda + \sum_{i} w_{i,j} \times x_i^{l-1}(t-1) \tag{181}$$

where λ represents the leak rate parameter. To further optimize efficiency, hardware can utilize event-driven updates, where the membrane potential is only re-evaluated upon the receipt of an input spike rather than at every discretized time step:

$$V_{mj}(t_{event}) = \begin{pmatrix} V_{mj}(t_{prev}) \\ \times \exp\left(-\dfrac{t_{event} - t_{prev}}{\tau}\right) \end{pmatrix} + w_{i,j} \tag{182}$$

This approach leverages the inherent sparsity of SNNs, allowing the architecture to deactivate idle functional units and scale power consumption proportional to network activity [397], [398].

2) Mapping Strategies and Workload Balancing

The strategy used to map a network topology onto available neurocores is critical for maximizing throughput and minimizing latency. There are two primary parallelization paradigms:

- **Parallelization along depth:** Each neurocore is dedicated to evaluating a single layer [393], [399]. While this enables pipelined processing, it can lead to uneven computational loads (as mentioned in an example of [388]).
- **Parallelization along spatial dimensions:** Each core computes a sub-part of a neural layer. This allows for time-multiplexing deep networks onto fewer cores [400]. However, in CNNs [1], where weights are shared, this can introduce memory redundancy between neurocores, leading to a loss of memory efficiency.

To address unbalanced event-driven workloads in many-core systems, designers employ methods such as channel-balanced workload scheduling (CBWS) [401], which partitions input channels into groups with approximately equal spiking activity. Furthermore, mapping algorithms prioritize minimizing the physical distance d between cores for connected neurons to reduce NoC delay and energy costs [26]:

$$\text{minimize} \sum_{i,j} w_{i,j} \cdot d(\text{core}(i), \text{core}(j)) \tag{183}$$

3) Hardware Optimization and Scaling

Efficient system design often involves weight quantization or binarization, where weights are constrained to few bits (e.g., $w \in \{-1, 1\}$) to reduce the memory footprint and replace multipliers with simple XNOR [402], [403] or AND logic gates [404]. These architectural optimizations enable SNNs to achieve energy-per-spike operations as low as 23.6 pJ on processors like Intel Loihi [126] or even 2.1 pJ on mixed-signal systems like BrainScaleS [129]. Ultimately, the energy advantage stems from an activity ratio significantly less than one, meaning computation only occurs when relevant data events are present [26]:

$$\text{Activity Ratio} = \frac{\text{Active Time}}{\text{Total Time}} \ll 1 \tag{184}$$

B. HARDWARE-AWARE NEURON AND SYNAPSE MODELING

The physical implementation of neurons and synapses involves a critical trade-off between biological plausibility and hardware resource efficiency [388]. For digital designs, the LIF model is widely favored due to its computational efficiency, as it captures essential temporal dynamics with minimal operations [405], [406]. To implement this in digital hardware like FPGAs, the continuous-time equations are discretized using the Euler method [134], [407]. The membrane potential V_m update at timestep $n+1$ is expressed as [407]:

$$V_m[n+1] = V_m[n] + \begin{pmatrix} -(V_m[n] - V_{rest}) \\ + R_m I[n] \\ \hline \tau_m \end{pmatrix} \Delta t \tag{185}$$

where Δt is the simulation timestep and τ_m is the membrane time constant.

To further optimize hardware efficiency, constant parameters like $\Delta t/\tau_m$ are often approximated to the nearest power of two (2^{-k}) [408], allowing costly multiplication or division units to be replaced by simple bit-shifting operations [409]. The simplest form, the IF neuron, removes leak and refractory mechanisms to reduce resource usage to an adder for integration, a comparator for threshold detection, and memory (such as BRAM or registers) for state storage [406], [9], [410].

Conversely, more complex dynamics like those in the HH or AdEx models require significantly more resources, often utilizing thousands of logic cells compared to the dozen required for an LIF neuron [410], [411], [412]. In mixed-signal and analog systems, ionic channel dynamics are emulated using analog transistors biased in the subthreshold regime. In this regime, the transistor's intrinsic exponential behavior

naturally mirrors the biophysics of neural membranes, governed by:

$$C_m \frac{dV_m}{dt} = -g_L(V_m - E_L) + \sum_i g_i(V_m - E_i) + I_{ext} \quad (186)$$

where g_L and g_i represent the leak and ionic conductances, respectively [411], [412].

Synapse modeling distinguishes between static efficacy and dynamic behavior. While most hardware treats synaptic weights as fixed scalar values representing LTP [174], specialized systems like Neurogrid [406] and ROLLS [413] implement Short Term Potentiation (STP). STP modulates the excitatory current based on recent activity, which can be modeled using neurotransmitter release x and binding dynamics y:

$$\begin{aligned} \frac{dx}{dt} &= -\frac{x}{\tau_x} + \sum_k \delta(t - t_{pre}^k) \\ I_{syn}(t) &= g_{max} \cdot y \cdot (V_{post} - E_{syn}) \end{aligned} \quad (187)$$

This allows the hardware to emulate the rise and decay phases of synaptic kinetics, providing a more bio-realistic foundation for neural simulation. Additionally, digital implementations may use trace-based techniques, which record recent spiking activity to calculate plasticity without storing every precise spike time [83].

C. ARCHITECTURAL OPTIMIZATION THROUGH APPROXIMATE COMPUTING

SNN hardware leverages the inherent fault tolerance of neural networks to optimize power and area by trading off precise numerical accuracy for computational efficiency [28], [388]. Because SNNs can compensate for approximations during training, significant gains are achieved by replacing traditional high-precision arithmetic units with simpler logic and stochastic mechanisms [414].

1) Stochastic Synapses and Hardware Variability

Implementing stochastic synapses (where connections are activated probabilistically) can improve generalization and reduce memory requirements [415], [416], [417]. In such a system, a synapse j may transmit a signal to neuron i with a probability p_{ij}, which can be modeled as:

$$S_{out}(t) = \begin{cases} 1 & \text{if } rand() < p_{ij} \text{ and } S_{in}(t) = 1 \\ 0 & \text{otherwise} \end{cases} \quad (188)$$

To reduce the area and power overhead of generating pseudo-random numbers via Linear-Feedback Shift Registers (LFSR) [418], emerging hardware utilizes the intrinsic cycle-to-cycle variability of nanoelectronic devices like memristors [419] or VO_2 oscillators [420] to implement native stochasticity.

2) Quantization and Binarization

Aggressive quantization minimizes memory footprint and simplifies synaptic operations. Binary Neural Networks (BNNs) constrain weights to binary values (e.g., $w \in \{-1, 1\}$), allowing hardware to replace MAC units with simple XNOR gates and bit-counters [402], [403], [421]. For SNNs, weight quantization has demonstrated energy gains ranging from $2.2\times$ up to $3.1\times$ with an accuracy loss of only $\approx 3\%$ [422]. The quantized weight w_{quant} is typically derived using a sign function:

$$w_{quant} = \text{sign}(w_{float}) \quad (189)$$

Furthermore, weight-binarized SNNs (WB-SNN) [423] reduce memory from multi-bit floating points to just 1-bit per synapse, dramatically increasing the number of neurons implementable on a single chip.

3) Algorithmic Bypass Strategies

Hardware-specific strategies like "leak-upon-load" are used to further reduce computation [393]. In this approach, leakage operations for the membrane potential are only performed when a neuron is actually loaded for an update, rather than at every algorithmic time step. The decayed potential is calculated based on the time elapsed since the last update $(t - t_{prev})$:

$$V_m(t) = V_m(t_{prev}) \cdot \exp\left(-\frac{t - t_{prev}}{\tau}\right) \quad (190)$$

Additionally, single-spike IF neurons restrict each neuron to firing at most once per recognition cycle, which simplifies output determination and reduces the frequency of memory accesses, further lowering dynamic power consumption.

D. HIGHER-LEVEL BRAIN-INSPIRED MECHANISMS

Beyond individual neurons, hardware implementations incorporate structural motifs and topological organizations found in the brain to enhance functionality and decision-making capabilities [424], [425]. These higher-level mechanisms allow SNN accelerators to process complex spatio-temporal patterns and implement competitive dynamics that improve classification performance [11].

1) Winner-Take-All (WTA) and Lateral Inhibition

WTA circuits implement competitive dynamics where neurons within a layer inhibit one another, ensuring that only the unit with the strongest activation fires while others are suppressed [426], [427], [428]. This lateral inhibition is essential for decision-making and feature sharpening. In hardware, this is often implemented through an inhibitory layer feedback or recurrent connections where a firing neuron j sends a large negative weight $-w_{inh}$ to its neighbors k [179]:

$$V_j(t) = V_j(t-1) + \sum_i w_{ij} S_i^{in}(t) - \sum_{k \neq j} w_{inh} S_k(t) \quad (191)$$

where $S_k(t)$ represents the spikes of neighboring neurons. Variants such as kWTA [429] allow for k neurons to fire before the entire layer is inhibited, facilitating more complex ensemble representations.

2) Reservoir Computing: Liquid State Machines (LSMs)

LSMs utilize a "liquid pool" consisting of a large number of neurons randomly interconnected with recurrent synapses [430], [431], [432]. These recurrent connections allow the network to maintain a transient internal state, making it highly effective for time-dependent applications like speech recognition. Because the complex computational graphs of LSMs do not always map efficiently to standard layered neurocores, specialized hardware using FPGAs [433], [434], [435], photonics [436], or 3D ICs [437] is often employed. The membrane potential V_j of a neuron in the reservoir is governed by both external inputs S^{in} and recurrent spikes S^{rec}:

$$V_j(t) = \alpha V_j(t-1) + \sum_{k \in In} w_{kj}^{in} S_k^{in}(t) + \sum_{i \in Res} w_{ij}^{rec} S_i^{rec}(t-1) \tag{192}$$

where w_{ij}^{rec} are the fixed, random recurrent weights that project input into a high-dimensional state space [26].

E. EVENT-DRIVEN SENSORS AND HYBRID SYSTEMS

To fully exploit the architectural benefits of SNN accelerators, they are frequently integrated with bio-inspired sensors that operate asynchronously via the AER protocol [438], [439]. Unlike traditional frame-based sensors that deliver at a specific sampling frequency a fixed amount of data, event-driven sensors only communicate data when a change in the sensed environment is detected [438].

1) Bio-Inspired Sensing Modalities

The most prominent examples of these technologies include:

- **Dynamic Vision Sensors:** These image sensors mimic the biological retina by only transmitting pixel-level changes in light intensity [438], [439]. Each event e_i generated by a DVS is typically represented as a tuple:

$$e_i = \{x_i, y_i, t_i, p_i\} \tag{193}$$

  where (x_i, y_i) are the spatial coordinates, t_i is the precise timestamp, and $p_i \in \{-1, 1\}$ represents the polarity (increase or decrease in intensity) [26]. This allows for a massive reduction in data bandwidth and provides a high dynamic range exceeding 120 dB [438].
- **Silicon Cochlea Circuits:** These audio recording systems emulate biological hearing by decomposing input waves into frequency-dependent spike streams [440]. Spike generation in a specific frequency band f at time t can be modeled as:

$$S_{f,t} = \Theta(E_{f,t} - \theta_f) \tag{194}$$

  where $E_{f,t}$ is the energy in the frequency band, θ_f is the threshold, and Θ is the Heaviside step function [441].

2) Hybrid Neural Networks

Recent research has explored Hybrid Neural Networks, which combine the high accuracy of formal ANNs with the energy efficiency of SNNs [442]. These systems often utilize traditional frame-based processing for initial feature extraction (such as using a CNN) followed by an SNN for low-power classification or temporal integration [443].

Communication between layers in these hybrid systems is handled via the AER protocol, where information can be encoded in multi-bit packets to preserve precision while maintaining event-driven efficiency [443]. The output y of such a cascade architecture can be expressed as:

$$y = \text{SNN}(\text{ANN}(x)) \tag{195}$$

Alternatively, parallel hybrid models can allocate tasks based on the required precision or temporal requirements:

$$y = \alpha \cdot \text{ANN}(x) + (1 - \alpha) \cdot \text{SNN}(x) \tag{196}$$

where α is a task-specific weighting factor. One implementation of a hybrid network on an FPGA achieved 97.09% accuracy on MNIST with a power consumption of only 7 μJ per frame [442].

F. ON-CHIP LEARNING AND ADAPTATION

While inference is common in deployed neuromorphic systems, implementing on-chip training presents a fundamental shift toward general accelerators that can autonomously adapt to their sensing environments via online learning [444], [388], [26]. This capability is critical for edge applications where data cannot be sent to the cloud, yet it imposes significant architectural challenges regarding memory density and computational complexity [28].

1) Local Learning Mechanisms and STDP

The most hardware-friendly approach to on-chip learning is STDP, a form of local Hebbian learning where synaptic weight updates depend solely on the relative timing of pre- and post-synaptic spikes [172], [175], [173], [123]. This locality minimizes inter-core communication since the necessary data is resident at the synapse [445], [446], [447], [448]. The weight change Δw is traditionally modeled as a function of the time difference $\Delta t = t_{post} - t_{pre}$ [26], [28]:

$$\Delta w = \begin{cases} A_+ \exp(-\Delta t/\tau_+) & \text{if } \Delta t > 0 \text{ (LTP)} \\ -A_- \exp(\Delta t/\tau_-) & \text{if } \Delta t < 0 \text{ (LTD)} \end{cases} \tag{197}$$

Digital implementations often replace these exponential terms with trace-based approximations to avoid complex calculations [449]. In this scheme, each neuron maintains a temporal variable or trace (T) that records recent activity and decays exponentially [450]. The weight is updated only upon the arrival of a spike:

$$w(t) = \begin{cases} w(t-1) - T_{post} & \text{at pre-synaptic spike} \\ w(t-1) + T_{pre} & \text{at post-synaptic spike} \end{cases} \tag{198}$$

2) Reward-Modulated and Supervised Adaptation

To enhance classification performance, systems like Loihi [126], [26] and ROLLS [413], [26] integrate reward signals to implement R-STDP [451]. This method utilizes an eligibility trace (E_t) to store a "memory" of recent STDP activity, which is then gated by a global neuromodulator such as dopamine ($R(t)$) [452], [28]. The weight update derivative is expressed as:

$$\dot{w}_{ij} = R(t) \cdot E_{tij}(t) \tag{199}$$

The evolution of the eligibility trace captures the temporal coincidence detection [130]:

$$\dot{E}_t = -\frac{E_t}{\tau_E} + \text{STDP}(\Delta t)\delta(t - t_{spike}) \tag{200}$$

For deep networks, on-chip supervised learning remains difficult due to the non-differentiable nature of spikes and the high memory cost of storing gradients [448], [127], [183]. Recent hardware-aware solutions include BP-STDP [184], which approximates backpropagation using local STDP rules, and ODESA [202], which utilizes local attention signals to bypass global error propagation. In BP-STDP, the output error $\epsilon_o(t)$ determines the direction of the weight change for hidden neurons:

$$\Delta w_{h,o}(t) = \alpha \cdot \epsilon_o(t) \cdot \sum_{t-k}^{t} S_h(t) \tag{201}$$

3) Hardware Resource Challenges

Implementing these rules requires substantial additional circuitry beyond the standard inference engine. This includes counters to track spike timings, barrel shifters for trace decay, and specialized logic like CORDIC [450] or Piecewise Linear (PWL) [453] modules to evaluate nonlinear functions. Furthermore, while inference memory only stores weights, online learning requires memory for states, gradients, and traces [59]. For backpropagation-based methods, memory requirements for storing activations across simulation time steps (T) scale as [26]:

$$\text{Memory Complexity} = O(T \times N) \tag{202}$$

4) Catastrophic Forgetting and Robustness

A major architectural hurdle for hardware-resident networks is catastrophic forgetting, where the system loses accuracy on previously learned tasks when exposed to new data [454], [455], [456]. To mitigate this, hardware can implement homeostatic mechanisms such as synaptic scaling [457], which globally adjusts synaptic strengths to maintain stable firing rates [26]:

$$w_i \to w_i \cdot \frac{\rho_{target}}{\rho_{actual}} \tag{203}$$

Additionally, neuroplasticity can be leveraged for fault tolerance [458], allowing the network to autonomously compensate for hardware defects or variability in memristive synapses [459].

G. LARGE-SCALE AND LOW-POWER ACCELERATORS

Neuromorphic engineering has produced several specialized hardware platforms designed to emulate the massive parallelism and energy efficiency of the biological brain, summarization can be found in Table 8.

1) Large-Scale Digital Neuromorphic Processors

The primary objective of digital large-scale accelerators is to provide high flexibility and scalability for neural simulations using discretized time steps.

- **IBM TrueNorth [460]:** A pioneering digital architecture featuring 1 million neurons with 256 million synapses distributed across 4,096 neurosynaptic cores. It operates at an ultra-low power consumption of approximately 65-70 mW. Each core implements a simplified membrane potential update:

$$V_m[t+1] = V_m[t] + \sum_j w_{ij} S_j[t] - \theta \cdot S_i[t] \tag{204}$$

  where θ is the reset value and weights w_{ij} are 4-bit signed integers. It achieves an energy efficiency of approximately 26 pJ per spike.
- **Intel Loihi [126]:** A highly programmable processor integrating 128 neuromorphic cores with three conventional x86 cores. It supports 30,000 neurons and 130 million synapses. A key differentiator is its on-chip plasticity engine which enables real-time learning via rules like STDP. The programmable dynamics are modeled as:

$$\begin{aligned} u[t+1] &= (1-d_u)u[t] + \sum_j w_{ij} S_j[t] \\ v[t+1] &= (1-d_v)v[t] + u[t] \end{aligned} \tag{205}$$

  where d_u and d_v are decay constants for current and voltage. Loihi demonstrates an efficiency of 23.6 pJ per spike.
- **SpiNNaker [128]:** A massively parallel system based on ARM968 processors (18 per chip) designed for large-scale neural simulations. The architecture is highly scalable, supporting systems with up to 1 million cores connected via packet-switched networks that mimic biological connectivity. While less energy-efficient than ASICs (40.5 pJ per spike), its software-based approach allows for the implementation of complex, flexible learning models.
- **Darwin [396]:** A digital neuromorphic co-processor featuring 2,048 neurons and a hierarchical routing scheme. It utilizes a packet-based communication protocol and a configurable topology to support different network types. Darwin achieves an energy efficiency of 45 pJ per spike.

2) Analog and Mixed-Signal Platforms

These platforms leverage the physical dynamics of transistors in the **subthreshold regime** to emulate neural behaviors, offering high throughput for biological modeling.

TABLE 8: Summary of representative neuromorphic hardware platforms

Platform	Implementation	Technology	Neuron Capacity	Synapse Capacity	Network Topology	On-Chip Learning
BrainScaleS [129]	Analog	65-180 nm	$\sim$ 512 per wafer	$\sim$ 130k per wafer	Hierarchical	STDP only
Neurogrid [406]	Analog	180 nm	$\sim$ 65k	$\sim$ 100M shared	Tree multicast	No
TrueNorth [460]	Digital	28 nm	1M	256M	2D mesh (unicast)	No
SpiNNaker [128]	Digital (ARM multi-core)	130 nm	$\sim 10^3$ per core	$\sim 10^6$ per core	2D mesh (multicast)	Yes
Loihi [126]	Digital	14 nm FinFET	1024 per core	130k per core	2D mesh	Yes
Darwin [396]	Digital	180 nm	2048	Variable	Hierarchical routing	Limited
Dynap-SE [461]	Mixed-signal	180 nm	256 per chip	64k per chip	Configurable	Yes

- **BrainScaleS [129]:** A wafer-scale mixed-signal platform that operates at 10,000 times biological real-time. Each wafer incorporates 200,000 AdEx neurons and 49 million synapses. It achieves a record energy efficiency of 2.1 pJ per spike at accelerated time.
- **Neurogrid [406]:** A mixed-analog-digital system designed for real-time biological simulations. It supports 65,536 neurons and 100 million synapses per core, emulating bio-realistic mechanisms such as ionic channels through sub-threshold analog operation.
- **Dynap-SE [461]:** A commercial mixed-signal platform containing 256 neurons per chip and 64,000 synapses. It is primarily research-oriented and provides a development board for real-world application prototyping.

H. EMERGING TECHNOLOGIES AND FUTURE DIRECTIONS

To overcome the density and connectivity limitations of traditional 2D silicon architectures, researchers are exploring high-density integration and novel materials that move beyond traditional paradigm. These technologies aim to emulate the massive parallelism and energy efficiency of the biological brain by leveraging physical phenomena for computation.

1) 3D Integration and Stacking

3D integration technology addresses the connectivity bottleneck by stacking logic and memory layers vertically, connected through Through Silicon Vias (TSVs) [462]. This approach offers significantly higher bandwidth and shorter interconnection lengths compared to planar designs [463]. Current research explores several 3D topological strategies:

- **Layer-per-tier:** Assigning one neural layer to each 3D plan to parallelize deep networks [464], [465].
- **Separated Tiers:** Dedicated tiers for memory and logic, which has been shown to increase throughput by stacking memory directly on top of the computing elements [464], [466].

Monolithic 3D implementations have demonstrated power savings of approximately 20% compared to equivalent 2D designs [464]. The reduction in power consumption P and delay L can be expressed as a function of the reduced wire length l:

$$P \propto \sum C_{wire} \cdot V^2 \cdot f, \quad L \propto \sum R_{wire} \cdot C_{wire} \tag{206}$$

where the significant reduction in average wire length l directly minimizes the parasitic capacitance C_{wire} and resistance R_{wire} of the inter-neuron communication paths [467], [468].

2) Memristive Crossbar Arrays and In-Memory Computing

Another major direction is the use of Non-Volatile Memory (NVM) devices, such as Phase-Change Memory (PCM) or Metal-Oxide Resistive RAM, organized in crossbar arrays [469], [444], [470], [471], [472]. These designs implement In-Memory Computing, where the memory element itself serves as the synaptic weight, effectively removing the need for data transfer between a processor and external storage [469], [463], [473], [474].

The fundamental advantage of these arrays is their ability to perform the dot-product operation natively using Kirchhoff's laws [470]. By applying voltages V_i (representing input spikes or intensities) across the rows, the resulting current I_j at the columns represents the weighted sum:

$$I_j = \sum_i G_{i,j} \cdot V_i \tag{207}$$

where $G_{i,j}$ is the conductance (the inverse of resistance) of the memristive device at the crosspoint (i, j), representing the synaptic weight $w_{i,j}$. This implementation potentially increases power efficiency by 1-2 orders of magnitude compared to standard CPUs or GPUs [463].

The weight adaptation in these devices is governed by their physical state change, where the rate of change in resistance R is a function of the applied voltage and historical state [26]:

$$\frac{dR}{dt} = f(V, R, \text{history}) \tag{208}$$

Despite their potential, these devices currently face challenges regarding imperfections, such as weight update nonlinearity, resistance drift, and high cycle-to-cycle variability, which can degrade the accuracy of the mapped SNN [475], [476], [477].

3) Photonic and Quantum-Inspired Spiking

Emerging research is also pushing into non-electronic substrates:

- **Photonic Spiking Chips:** These utilize optical pulses for spike communication and processing, offering operational frequencies in the GHz range with minimal transmission heat [478].
- **Quantum-Inspired SNNs:** These architectures explore utilizing quantum tunneling for synaptic updates [479]

and quantum superposition for representing multiple spike states simultaneously [26], [237].

While these technologies are currently at an early research stage, they represent a path toward brain-scale systems with billions of neurons and trillions of synapses, operating with biological-level energy budgets.

XII. EMERGING RESEARCH BEYOND NEURONS AND SYNAPSES

Traditional SNN models typically rely on simplified point neurons and synaptic connections. However, recent neuromorphic research has begun exploring additional biological mechanisms that contribute to neural computation, particularly astrocytes and dendritic structures. Astrocytes participate in tripartite synapses and modulate neuronal activity through slow calcium-mediated signaling, enabling long-timescale regulation of network dynamics. Dendritic structures, on the other hand, introduce compartmentalized and nonlinear processing within individual neurons, allowing the integration of multi-timescale inputs and facilitating more biologically plausible learning mechanisms. Motivated by these insights, several computational frameworks have incorporated astrocytic signaling and dendritic dynamics into SNN architectures. Table 9 summarizes representative models in this emerging research direction, highlighting their biological inspiration, neuron models, learning mechanisms, and main computational contributions.

A. ASTROCYTE ORIENTED RESEARCH

1) LSM-Integrated Astrocyte Spiking Neural Networks

The construction of the proposed LSM-integrated astrocyte SNN [140] utilizes the snnTorch library [23] to develop units capable of representing information across diverse temporal scales. To investigate the computational contributions of glial cells, artificial neurons and astrocytes are integrated into a LSM reservoir designed to mimic biological signaling dynamics.

The artificial astrocytes utilize a simplified "gatekeeper" framework to represent slower signaling dynamics, where membrane potential serves as a surrogate for intracellular calcium activity. This is implemented via a system of coupled differential equations:

$$\begin{aligned} U_a[t+1] &= \beta_a U_a[t] + I_{syn}[t+1] - R U_a[t] \\ I_{syn}[t+1] &= \alpha I_{syn}[t] + I_{in}[t+1] \end{aligned} \tag{209}$$

Here, I_{syn} represents the synaptic current, which possesses its own decay dynamics governed by the parameter α. By setting both the membrane decay rate β_a and the synaptic decay rate α to 0.3, the astrocytes are required to integrate inputs for significantly longer periods than neurons before reaching their threshold. Furthermore, unlike neurons, artificial astrocytes do not generate action potentials; their potential is reset to zero ($R = 1$) only after the threshold has been exceeded for ten consecutive timesteps.

The units are organized into an LSM architecture containing a reservoir of four all-to-all recurrently connected subnetworks: two consisting of neurons and two of astrocytes. This configuration specifically replicates the "tripartite synapse" connectivity observed in biological systems, where astrocytes receive input from and provide feedback to neuronal populations. Input data is fed into one neuron subnetwork, which provides all-to-all downstream connections to an astrocyte subnetwork, while the astrocytes in turn send outputs to the other neuron subnetwork.

2) Spiking Neuron-Astrocyte Networks

The construction of the Spiking Neuron-Astrocyte Network (SNAN) [139] follows a tripartite synaptic architecture designed to enhance the performance of standard SNNs in image recognition tasks. The system is implemented using the Brian2 simulator and is organized into three primary units: a preprocessing layer for rate-coded spike conversion, the SNAN learning layer, and a k-nearest neighbors (k-NN) classifier.

The network utilizes adaptive integrate-and-fire (AIF) neurons in the postsynaptic layer. The membrane potential v is modeled by integrating fast excitatory neurotransmission (g_{exc1}), slow excitatory gliotransmission (g_{exc2}), and inhibitory inputs (g_{inh}). The evolution of the membrane potential is defined as:

$$\begin{aligned} \tau \frac{dv}{dt} = &(v_{rest} - v) + g_{exc1}(v_{exc1} - v) \\ &+ g_{exc2}(v_{exc2} - v) + g_{inh}(v_{inh} - v) \end{aligned} \tag{210}$$

where v_{rest} is the resting potential and v_{exc} are the reversal potentials. The conductances g decay according to $\tau_x \frac{dx}{dt} = -x$. To maintain homeostasis and facilitate competition, the network incorporates lateral inhibition, where interneurons (IN) receive signals from postsynaptic neurons and inhibit their neighbors.

The astrocytic component is modeled as a point process following a simplified Postnov model. The astrocyte does not generate action potentials but instead integrates presynaptic activity through a three-stage signaling pathway: synaptic coupling, IP_3 production, and calcium (Ca^{2+}) elevation.

- **Synaptic Coupling:** A synaptic variable z spikes synchronously with the presynaptic neuron and decays exponentially:

$$\tau_S \frac{dz}{dt} = -\frac{z}{d_S} \tag{211}$$

  where τ_S is the synaptic delay and d_S controls relaxation.
- **IP$_3$ Production:** The accumulation of z triggers IP_3 production (S_m):

$$\begin{aligned} \tau_{Sm} \frac{dSm}{dt} = &(1 + \tanh(S_{Sm}(z - h_{Sm})))(1 - S_m) \\ &- \frac{S_m}{d_{Sm}} \end{aligned} \tag{212}$$

  where h_{Sm} is the activation threshold and S_{Sm} defines the sigmoid steepness.

TABLE 9: Summary of emerging SNN research incorporating biological mechanisms beyond neurons and synapses.

Model	Biological Component	Neuron Model	Learning Method	Key Contribution
LSM-Integrated Astrocyte SNN [140]	Astrocytes (tripartite synapse)	Spiking neurons + artificial astrocytes	Reservoir computing	Integrates artificial astrocytes into a Liquid State Machine to model slow calcium-like dynamics and long-timescale information integration.
Spiking Neuron-Astrocyte Network (SNAN) [139]	Astrocytes (gliotransmission)	Adaptive Integrate-and-Fire (AIF)	Unsupervised STDP	Implements tripartite synapses where astrocytes modulate synaptic conductances through calcium signaling.
Astrocytic Reservoir SNN [137]	Astrocytic calcium lattice	Izhikevich neurons	Activity-driven plasticity	Uses astrocytes as a slow reservoir to store working memory traces through calcium-dependent synaptic modulation.
Astrocyte-Modulated Bursting SNN [136]	Astrocytic gliotransmission	Izhikevich neurons	Mean-field modulation	Astrocytes regulate neuronal bursting dynamics and suppress hyperexcitation to maintain stable information encoding.
Situation-Based Memory SNAN [138]	Astrocytic calcium signaling	Izhikevich neurons	STDP + astrocyte feedback	Astrocytes detect synchronous neuronal activity and enhance memory storage through calcium-triggered synaptic potentiation.
CD-DT-DSNN [132]	Dendritic temporal processing	Multi-branch dendritic neuron	BPTT with surrogate gradient	Introduces heterogeneous dendritic branches with combined decay and dynamic thresholds to capture multi-timescale temporal features.
NADOL [135]	Multi-compartment dendrites	Soma + basal + apical compartments	Local dendritic error learning	Uses dendritic compartments to separate feedforward and feedback signals, enabling hardware-friendly online credit assignment.
DH-SNN [133]	Temporal dendritic heterogeneity	Multi-branch DH-LIF neuron	BPTT with surrogate gradient	Encodes multi-timescale temporal memory through dendritic branches with heterogeneous decay constants.
DLHM [131]	Basal/apical dendritic compartments	Multi-compartment neuron	Hybrid local + global learning	Combines local dendritic prediction learning with global plateau-potential error signals for efficient credit assignment.

- **Calcium Dynamics:** Elevated S_m drives Ca^{2+} oscillations (c):

$$\tau_c \frac{dc}{dt} = c - c^4 f(c, c_e) + (r + \beta S_m) \tag{213}$$

When c exceeds the threshold c_{thresh}, the astrocyte releases gliotransmitters, activating the slow excitatory conductance g_{exc2} in the postsynaptic neuron.

Learning is governed by an unsupervised STDP rule. The synaptic weights w are updated based on the relative timing of pre- and postsynaptic activation using traces α that decay exponentially. The weight-dependent update rule is expressed as:

$$\begin{aligned} &w \to w + \eta_{pre}\alpha_{post1} \text{ (Presynaptic activation)} \\ &w \to w + \eta_{post}\alpha_{pre}\alpha_{post2} \text{ (Postsynaptic activation)} \end{aligned} \tag{214}$$

Unlike neurons, the astrocyte-to-neuron coupling only increases weights during astrocytic activation, as the significant timescale difference makes bidirectional STDP unsuitable. The architecture forms tripartite synapses by coupling astrocytes to the synapses rather than directly to the neurons, allowing a single astrocyte to modulate multiple connections within its territory.

3) Spiking Neuron Networks with Astrocytic Reservoirs

The proposed computational framework realizes working memory (WM) through the synergetic interaction of a SNN and an astrocytic lattice, operating across disparate temporal scales [137]. The SNN provides millisecond-range processing, while the astrocytes function as a slow "memory reservoir" that maintains information patterns for several seconds.

The neuronal layer consists of $W \times H$ excitatory neurons modeled using the computationally efficient Izhikevich formalism. The evolution of the membrane potential V and the recovery variable U for a neuron at coordinates (i, j) is governed by:

$$\begin{aligned} \frac{dV(i,j)}{dt} &= 0.04V(i,j)^2 + 5V(i,j) - U(i,j) \\ &\quad + 140 + I_{app}(i,j) + I_{syn}(i,j) \\ \frac{dU(i,j)}{dt} &= a(bV(i,j) - U(i,j)) \end{aligned} \tag{215}$$

where I_{app} denotes external stimulation and I_{syn} represents the sum of synaptic currents from presynaptic neighbors. The synaptic weight g_{syn} is specifically modulated by a baseline weight η and an astrocyte-dependent term ν_{Ca}:

$$g_{syn}(i,j) = \eta + \nu_{Ca}(m,n) \tag{216}$$

This architecture assumes a non-specific random connectivity where the distance between synaptic connections follows an exponential distribution $f_R(r) = \frac{1}{\lambda}\exp(-\frac{r}{\lambda})$.

Astrocytes are organized in a two-dimensional lattice $(M \times N)$, where each cell integrates activity from a local neuronal ensemble N_a. The signaling pathway initiates with the release of glutamate G into the extracellular space in response to neuronal spikes:

$$\frac{dG(i,j)}{dt} = -\alpha_{glu}G(i,j) + k_{glu}\Theta(V(i,j) - 30) \tag{217}$$

The accumulation of glutamate triggers the production of inositol 1,4,5-trisphosphate (IP_3), whose concentration dy-

namics are modeled as:

$$\frac{dIP_3(m,n)}{dt} = \frac{IP_3^* - IP_3(m,n)}{\tau_{IP3}} + J_{PLC\delta}(m,n) + J_{glu}(m,n) + diff_{IP3}(m,n) \quad (218)$$

Here, J_{glu} represents glutamate-induced production that is activated only when a sufficient fraction of neurons in the ensemble fire synchronously. Elevated IP_3 induces the release of calcium from the endoplasmic reticulum, governed by:

$$\frac{d[Ca^{2+}](m,n)}{dt} = J_{ER} - J_{pump} + J_{leak} + J_{in} - J_{out} + diff_{Ca} \quad (219)$$

Spatial coordination is facilitated by the diffusion terms $diff_{IP3}$ and $diff_{Ca}$, which utilize discrete Laplace operators to simulate gap junction coupling between neighboring astrocytes.

Information is encoded through astrocyte-induced potentiation of excitatory synapses, functioning as a form of short-term plasticity. The feedback ν_{Ca} is activated when the astrocytic calcium concentration crosses a critical threshold $[Ca^{2+}]_{thr}$:

$$\nu_{Ca} = \nu_{Ca}^* \Theta([Ca^{2+}](m,n) - [Ca^{2+}]_{thr}) \quad (220)$$

This modulation is contingent upon the fraction of synchronously spiking neurons F_{astro} being sufficient to maintain the signal. By operating at the timescale of calcium elevations (several seconds), the astrocytic network effectively stores traces of neuronal activations, allowing the SNN to successfully retrieve noisy or distorted information patterns through localized spatial synchronization.

4) Astrocyte-Modulated Bursting Spiking Neural Networks

The investigated computational framework incorporates a SNN integrated with an astrocytic layer to regulate spatiotemporal firing patterns and prevent hyperexcitation during information encoding [136]. The system utilizes a mean-field approach to model the interaction between neuronal activity and astrocytic gliotransmission within a tripartite synaptic architecture.

The neuronal population is represented using the Izhikevich model, selected for its computational efficiency and biological relevance, similarly to Equation 215 in [137]. Neurons are initialized in a tonic spiking mode with parameters $a = 0.02, b = 0.5, c = -40$ mV, and $d = 100$. The total synaptic current $I_{syn,i}$ is determined by the sum of neurotransmitter concentrations $y_{j,i}$ from connected presynaptic neurons j, scaled by weights $w_{j,i}$. The dynamics of the synaptic neurotransmitter $y_{j,i}$ are governed by:

$$\frac{dy_{j,i}}{dt} = -\frac{y_{j,i}}{\tau_y} + b_y \delta(t - t_{sp}) \quad (221)$$

where τ_y (4 ms) is the relaxation time constant and b_y represents the fraction of neurotransmitter release upon a spike at time t_{sp}.

The model focuses on glutamatergic synapses where astrocytes provide homeostatic regulation. Presynaptic activity triggers the release of extrasynaptic glutamate X_e, modeled via a mean-field approach:

$$\frac{dX_e}{dt} = -\frac{X_e}{\tau_X} + b_X \delta(t - t_{sp,e}) \quad (222)$$

where τ_X (100 ms) is significantly slower than the synaptic decay. This glutamate binds to astrocytic receptors, inducing the release of gliotransmitters Y_j into the extracellular space. The functional relationship between neurotransmitter and gliotransmitter concentrations is expressed as:

$$\frac{dY_j}{dt} = -\alpha_Y Y_j + \frac{\beta_Y}{1 + \exp(-X_e + X_{thr})} \quad (223)$$

where α_Y is the clearance rate (120 ms) and X_{thr} is the activation threshold.

Astrocytes modulate neuronal activity by inducing depression in synaptic transmission. The synaptic current $I_{syn,i}$ in the postsynaptic neuron is modified by a feedback term dependent on the gliotransmitter concentration Y_j:

$$I_{syn,i} = \sum_{j=1}^{M} y_{j,i} w_{j,i} \left(1 - \frac{\gamma_Y}{1 + \exp(-Y_j + Y_{thr})}\right) \quad (224)$$

Here, γ_Y represents the strength of astrocytic modulation, which serves to suppress hyperexcitation and restore structured information patterns in the neuronal raster.

The network architecture consists of 300 neurons in a 4:1 ratio of excitatory to inhibitory units, maintaining an excitation-inhibition balance. Connectivity is "all-to-all" with specific connection probabilities (5% for excitatory and 20% for inhibitory synapses). External information is supplied via a stimulation current $I_{stim,i}$ derived from 300×300 pixel images S_i:

$$I_{stim,i} = S_i \times A_S \quad (225)$$

where A_S is the stimulus amplitude. By operating at a slower timescale than neurons, the astrocytic layer acts as an "inertial" buffer, regulating the transition between bursting and tonic firing modes to facilitate sensory encoding.

5) Spiking Neuron-Astrocyte Networks for Situation-Based Memory

The construction of the proposed SNAN utilizes a multi-layer architecture designed to facilitate situation-based learning through the synergetic interaction of fast-spiking neuronal dynamics and slow astrocytic calcium buffering [138]. The system is organized into three interacting layers: a 79×79 lattice of pyramidal neurons, a 40×40 lattice of inhibitory interneurons, and a 26×26 lattice of astrocytes.

Individual neurons are modeled using the Izhikevich formalism to balance computational efficiency with biological plausibility using similarly to Equation 215 in [137]. The total synaptic current $I_{syn,i}$ is defined by:

$$I_{syn,i} = \sum_{k=1}^{N_i} \frac{w_{syn,k}(E_{syn} - V_i)}{1 + \exp(-V_{pre,k}/k_{syn})} \quad (226)$$

where E_{syn} is the reversal potential (0 mV for excitatory and -90 mV for inhibitory synapses) and $w_{syn,k}$ represents the synaptic weight. Synaptic connectivity is established randomly, with the distance r between connected neurons following an exponential distribution $f_R(r) = \frac{1}{\lambda}\exp(-r/\lambda)$.

The network employs Spike-Timing-Dependent Plasticity (STDP) for pyramidal-pyramidal (EE) and interneuron-pyramidal (IE) connections. The weight update δw for EE synapses depends on the timing difference Δt between pre- and postsynaptic spikes:

$$\delta w_{synEE,k}(\Delta t) = \begin{cases} g_{synEE}\exp(\frac{\Delta t}{\tau}), & \Delta t \leq 0 \\ -g_{synEE}\exp(\frac{\Delta t}{\tau}), & \Delta t > 0 \end{cases} \tag{227}$$

For IE connections, the rule incorporates the postsynaptic firing rate f and a threshold f^* via the Heaviside step function H, allowing interneurons to selectively inhibit subnetworks that were inactive during training:

$$\delta w_{synIE,k}(\Delta t) = \begin{cases} g_{synIE}\exp(\frac{\Delta t}{\tau})H(f^* - f), & \Delta t \leq 0 \\ -g_{synIE}\exp(\frac{\Delta t}{\tau}), & \Delta t > 0 \end{cases} \tag{228}$$

Astrocytes integrate neuronal activity through the release of glutamate G_i into the extrasynaptic space, modeled as:

$$\frac{dG_i}{dt} = -\alpha_{glu}G_i + k_{glu}H(V_i - 30\text{mV}) \tag{229}$$

When the fraction of synchronously spiking neurons in an ensemble reaches a threshold F_{act} (0.75), glutamate triggers the production of inositol 1,4,5-trisphosphate (IP_3), which subsequently induces calcium (Ca^{2+}) release from the endoplasmic reticulum. The intracellular calcium dynamics for the m-th astrocyte follow the model:

$$\begin{aligned} \frac{d[Ca^{2+}]_m}{dt} &= J_{ER} - J_{pump} + J_{leak} \\ &\quad + J_{in} - J_{out} + J_{Gca} \end{aligned} \tag{230}$$

Spatial coordination is achieved via gap junction diffusion currents J_{Gca} and J_{Gip3}. Finally, elevated calcium concentrations $[Ca^{2+}]_m$ exceeding a threshold provide positive feedback by potentiating excitatory synaptic weights:

$$w_{synEE} = w_{synEE}\begin{pmatrix} 1 \\ +\nu^*_{Ca}H\begin{pmatrix}[Ca^{2+}]_m \\ -[Ca^{2+}]_{thr}\end{pmatrix}\end{pmatrix} \tag{231}$$

B. DENDRITIC ORIENTED RESEARCH

1) Dendritic Spiking Neural Networks with Combined Decay and Dynamic Threshold

The implementation of the proposed Dendritic Spiking Neural Network (DSNN) [132] utilizes a specialized neuron model that incorporates combined membrane potential decay (CD) and a dynamic firing threshold (DT) to capture multi-scale temporal dependencies. The termed CD-DT-DSNN, extends the capabilities of traditional point neurons by simulating the nonlinear computational properties of biological dendritic structures.

The model facilitates differentiated temporal processing through heterogeneous dendritic branches. Each dendritic segment d integrates external input current I^d governed by a trainable timing factor α_d, which allows the capture of features across various timescales. The evolution of the dendritic current i^d is defined as:

$$i^d_{t+1} = \alpha_d i^d_t + (1 - \alpha_d)I^d_{t+1} \tag{232}$$

where the timing factor α_d is constrained to the range $(0, 1)$ via a sigmoid function. The neuronal soma integrates these currents and updates its membrane potential u while employing a soft reset mechanism based on the Heaviside step function $H(x)$. The spiking output o_{t+1} is determined by the relationship between the potential and the threshold:

$$o_{t+1} = H(u_{t+1} - u^{th}_{t+1}) \tag{233}$$

To enhance long-term memory and response sensitivity, the model replaces standard exponential decay with a gated fusion of exponential and linear decay. This mechanism is controlled by a trainable decay factor $\delta_t \in (0, 1)$, allowing the network to dynamically switch between retaining early weak stimuli and rapidly clearing historical potentials. The membrane potential update is expressed as:

$$\begin{aligned} u_{t+1} = &[1 - \delta_t(1 - \tau_{exp})]u_t \\ &- (1 - \delta_t)\tau_{lin} \\ &+ (1 - \beta)\sum_d Ri^d_{t+1} - o_t u^{th}_{t+1} \end{aligned} \tag{234}$$

In this formalism, τ_{exp} and τ_{lin} represent the exponential and linear decay constants, respectively, while β denotes the somatic membrane potential decay rate. When $\delta_t = 0$, the model defaults to linear decay ($u_t - \tau_{lin}$), facilitating the accumulation of long-term dependencies. Conversely, $\delta_t = 1$ invokes exponential decay ($\tau_{exp}u_t$), prioritizing high-speed responses to current inputs.

The system incorporates an adaptive threshold u^{th}_{t+1} to dynamically regulate firing rates based on historical activity, thereby preventing neuron silence and accelerating convergence. The threshold is determined by:

$$\begin{aligned} u^{th}_{t+1} &= \theta\eta_{t+1} + u_0 \\ \eta_{t+1} &= \rho\eta_t + (1 - \rho)o_t \end{aligned} \tag{235}$$

where η is an adaptive time constant, ρ is a trainable adaptive factor, θ is a scaling constant (1.2), and u_0 is the base threshold (0.5). If the neuron fires ($o_t = 1$), the threshold increases to suppress redundant activity; if it remains silent ($o_t = 0$), the threshold decays to increase sensitivity.

The network is structured as a multi-layer feedforward architecture comprising an input layer, hidden layers of stacked dendritic neurons, and an output classification layer. Training is performed using the BPTT algorithm. To manage the non-differentiable nature of discrete spikes, a Gaussian surrogate gradient is utilized during the backward pass to update synaptic weights and heterogeneous neuronal parameters synchronously. Sparse connections are employed

between layers to reduce parameter complexity and avoid explosion while ensuring balanced input coverage.

2) Neuromorphic Architecture for Dendritic Online Learning

The Neuromorphic Architecture for Dendritic Online Learning (NADOL) [135] architecture is designed to resolve the credit assignment problem in SNNs by utilizing multi-compartment neurons inspired by the morphology of cortical pyramidal cells. The network is structured into an input layer of Poisson spiking neurons, a hidden layer of three-compartment neurons, and an output layer of two-compartment neurons.

In the hidden layer, individual neurons consist of a soma, a basal dendritic compartment for feedforward sensory integration, and an apical dendritic compartment for feedback signal processing. Input spikes are first filtered into continuous spike trains $s_{input}(t)$ via a response kernel $\kappa(t)$. The membrane voltages for the basal (V_i^{0b}) and apical (V_i^{0a}) compartments are defined as:

$$\begin{aligned} V_i^{0b}(t) &= \sum_{j=1}^{784} W_{ij}^0 s_{input,j}(t) + b_{0i} \\ V_i^{0a}(t) &= \sum_{j=1}^{10} Y_{ij} s_j^1(t) \end{aligned} \tag{236}$$

where W^0 represents the feedforward synaptic matrix, b_0 is the bias, and Y denotes a fixed random feedback weight matrix. The somatic membrane potential $V_i^0(t)$ integrates these signals according to the leak conductance g_l and the conductances from the basal (g_b) and apical (g_a) dendrites:

$$\begin{aligned} \tau \frac{dV_i^0(t)}{dt} =& (V_R - V_i^0(t)) \\ &+ \frac{g_b}{g_l}(V_i^{0b}(t) - V_i^0(t)) \\ &+ \frac{g_a}{g_l}(V_i^{0a}(t) - V_i^0(t)) \end{aligned} \tag{237}$$

In this model, $\tau = C_m/g_l$ is the membrane time constant. The instantaneous firing rate $\phi_i^0(t)$ is determined by a sigmoid nonlinear function σ scaled by the maximum firing rate ϕ_{max}.

Learning is achieved through a two-phase training scheme: a forward phase and a target phase. During the target phase, a teaching current $I(t)$ drives the output neurons toward a desired state. At the conclusion of these phases, the hidden layer computes plateau potentials α^f (forward) and α^t (target) as follows:

$$\begin{aligned} \alpha_i^f &= \sigma\left(\frac{1}{\Delta t_1}\int_{t_1-\Delta t_1}^{t_1} V_i^{0a}(t)dt\right), \\ \alpha_i^t &= \sigma\left(\frac{1}{\Delta t_2}\int_{t_2-\Delta t_2}^{t_2} V_i^{0a}(t)dt\right) \end{aligned} \tag{238}$$

The discrepancy between these potentials serves as the local error signal for updating the hidden layer's feedforward weights W^0. The gradient for the weight update is approximated by:

$$\frac{\partial L_0}{\partial W^0} \approx -k_b(\alpha^t - \alpha^f)\phi_{max}\sigma'(V_f^0)s_{input}^f \tag{239}$$

where $k_b = g_b/(g_l + g_b + g_a)$ and V_f^0 is the membrane potential during the forward phase.

To facilitate efficient on-chip learning, the continuous dynamics are discretized using the Euler method. To mitigate the hardware cost of multipliers and nonlinear functions, the architecture utilizes piecewise linear approximation (PLA) for sigmoid functions and shift-and-add operations in place of standard multiplication. The discrete soma potential is calculated as:

$$\begin{aligned} V_i^0(n+1) =& V_i^0(n) + \\ & \frac{\Delta n}{\tau}\begin{bmatrix} (V_R - V_i^0(n)) \\ + \frac{g_b}{g_l}(V_i^{0b}(n) - V_i^0(n)) \\ + \frac{g_a}{g_l}(V_i^{0a}(n) - V_i^0(n)) \end{bmatrix} \end{aligned} \tag{240}$$

3) Spiking Neural Networks with Temporal Dendritic Heterogeneity

The proposed Spiking Neural Networks with Temporal Dendritic Heterogeneity (DH-SNN) [133] architecture utilizes a novel multi-compartment spiking neuron model, termed DH-LIF, which integrates temporal dendritic heterogeneity into the standard LIF framework. This model is designed to capture multi-timescale temporal dynamics by abstracting the cable properties of biological dendrites into discrete temporal memory units.

A DH-LIF neuron consists of a single soma compartment and multiple independent dendritic branches (d). Unlike vanilla LIF neurons that only maintain a single timescale memory in the soma, the DH-LIF neuron enables multi-timescale processing through heterogeneous timing factors assigned to each dendritic branch. The discrete-time dynamics of the dendritic current i^d and the somatic membrane potential u are governed by the following system of equations:

$$\begin{aligned} i^d[t+1] &= \alpha_d i^d[t] + (1-\alpha_d) I^d[t+1] \\ u[t+1] &= \beta u[t] + (1-\beta)\sum_d R i^d[t+1] - o[t]u_{th} \\ o[t+1] &= H(u[t+1] - u_{th}) \end{aligned} \tag{241}$$

In this formulation, α_d represents the timing factor of the d-th dendritic branch, defining the decay rate of the dendritic memory, while β denotes the timing factor of the somatic membrane potential. I^d is the external synaptic current injected into a specific branch, and u_{th} is the firing threshold. The Heaviside step function $H(\cdot)$ determines the output spike o. To ensure numerical stability, both α_d and β are constrained to the interval $[0, 1]$ via a sigmoid function.

To prevent the parameter explosion often associated with multi-compartment models, the DH-SNN incorporates a sparse connection restriction. Synaptic inputs are randomly and non-overlappingly distributed across the dendritic

branches of a neuron. For a neuron with D branches and M inputs, the index set of synapses P_d connected to branch d is restricted such that:

$$\begin{aligned} &\sum_d |P_d| = M, \\ &\bigcup_d P_d = \{0, 1, \ldots, M-1\}, \\ &|P_d| \approx \frac{M}{D} \end{aligned} \tag{242}$$

This architecture allows different branches to simultaneously match variable timescales within the input signal—for instance, using branches with large timing factors for long-term memory of low-frequency components and branches with small timing factors for rapid responses to high-frequency transients.

The landscape of temporal heterogeneity is optimized through the automatic learning of synaptic weights and timing factors (α_d, β). This is achieved using an adapted spiking version of the BPTT algorithm. To address the non-differentiability of discrete spikes, a multi-Gaussian surrogate gradient H' is employed during the backward pass:

$$\begin{aligned} H' = \frac{\partial o}{\partial u} =& \gamma(1+h)N(u|u_{th}, \sigma^2) \\ &- \gamma h N(u|\sigma, (s\sigma)^2) \\ &- \gamma h N(u|-\sigma, (s\sigma)^2) \end{aligned} \tag{243}$$

Crucially, the dendritic currents i^d are not subject to the somatic reset mechanism, allowing them to preserve historical information even after the neuron fires, which significantly enhances the network's long-term memory capabilities compared to vanilla SNNs.

4) Spike-Driven Multi-Scale Learning with Hybrid Dendritic Mechanisms

The investigated computational framework, termed Dendritic Learning with Hybrid Mechanisms (DLHM) [131], realizes an efficient SNN by integrating micro-scale local learning with macro-scale global learning. This architecture utilizes multi-compartment neurons to segregate feedforward sensory information from feedback error signals, facilitating a biologically plausible solution to the credit assignment problem.

The network is organized into a three-layer hierarchy. Hidden layer neurons are modeled with three compartments: basal dendrites for feedforward input, apical dendrites for feedback signals, and a soma for integration and output. The somatic membrane potential V_s for a hidden neuron i evolves as:

$$\begin{aligned} \tau \frac{dV_{s,h,i}(t)}{dt} =& (V_R - V_{s,h,i}(t)) \\ &+ \frac{g_b}{g_l}(V_{b,h,i}(t) - V_{s,h,i}(t)) \\ &+ \frac{g_a}{g_l}(V_{a,h,i}(t) - V_{s,h,i}(t)) \end{aligned} \tag{244}$$

where V_R is the resting potential, and g_l, g_b, g_a represent the leak, basal, and apical conductances, respectively. A critical feature of this model is the dendritic prediction of somatic activity, where the predicted membrane potential V_{pre} is formulated to approximate the soma's state:

$$V_{pre,h,i} = \frac{g_l V_R + g_b V_{b,h,i} + g_a V_{a,h,i}}{g_l + g_b + g_a} \tag{245}$$

The output layer employs two-compartment neurons, omitting the apical compartment, where the somatic potential is nudged by a teaching current $I_i(t)$ during training phases to represent desired target values.

At the micro-scale, synaptic weights are adjusted to minimize the error between the predicted dendritic rate and the actual somatic firing. The local plasticity variable $LP_i(t)$ for a synapse i with weight w_i is determined by a three-factor rule:

$$LP_i(t) = (S(t) - \phi(V_{pre}))\frac{\partial}{\partial w_i} V_w(t) \tag{246}$$

where $S(t)$ is the somatic spike train, $\phi(V_{pre})$ is the predicted rate, and V_w is the local dendritic potential. This mechanism ensures somatodendritic consistency, which enhances learning convergence and robustness against noise.

The macro-scale learning follows a two-phase strategy comprising a forward phase and a target phase. During these phases, the network utilizes burst-related plateau potential (PP) dynamics generated in the apical dendrites to compute error signals. At the conclusion of each phase, PP variables α^f (forward) and α^t (target) are integrated over a specific window Δt:

$$\alpha_i = \frac{1}{\Delta t}\int_t^{t+\Delta t} V_{a,h,i}(t)dt \tag{247}$$

The global weight update for the hidden layer is then driven by the discrepancy between these two potentials, facilitating gradient descent across the network:

$$\Delta W_h \propto -k_b(\alpha^t - \alpha^f)\phi'_{max}(V_{s,h})s_{input} \tag{248}$$

By combining these multi-scale mechanisms, the DLHM model demonstrates superior power efficiency and robustness, achieving state-of-the-art performance for dendritic-based learning on neuromorphic hardware.

XIII. AVAILABLE TOOLS AND FRAMEWORKS

Aside from surveying from investigating state-of-the-art research work, we also conducted a survey on existing open-source framework that are available to the neuromorphic research community. For more information please see Table 10 and 11.

XIV. CONCLUSION

In conclusion, the evolution of neural computing is increasingly leaning toward the biological efficiency of the human brain to overcome the scalability and energy bottlenecks of traditional DNNs. This review has systematically navigated the landscape of SNNs, tracing their roots from fundamental

TABLE 10: List of available open-source software frameworks for SNN simulation with description and URL for installation/documentation (part 1).

Framework	Training Paradigm	How to use	Description
Brian2 [480]	STDP for supervised learning	Python programming	Brian2 is an open-source simulator for spiking neural networks, implemented in Python, C++, and Cython. It was actively maintained at the time this survey was conducted. https://github.com/brian-team/brian2
Nest [481]	STDP for supervised learning, R-STDP for model-free RL	Python programming	NEST is an open-source tool developed in Python and C++ for computational neuroscience, enabling the simulation and analysis of large-scale neural networks. It supports detailed modeling of individual neurons, synapses, and their connectivity, as well as various plasticity mechanisms to study learning processes and gain insights into fundamental brain functions. It was actively maintained at the time this survey was conducted. https://github.com/nest/nest-simulator
Nengo [482]	STDP, PES for supervised learning	Python programming	Nengo is a Python-based library built on the Neural Engineering Framework (NEF) for constructing and simulating large-scale neural models. It enables the creation of complex spiking and non-spiking neural simulations with practical default settings using only a few lines of code. It was actively maintained at the time this survey was conducted. https://github.com/nengo/nengo
NengoDL [85]	ANN2SNN conversion, surrogate gradient BP for supervised learning	Python programming	NengoDL enables the integration of biologically realistic neural models with deep learning components, using a Python-based interface and a TensorFlow/Keras backend. Its documentation includes examples demonstrating high-accuracy models through rate-based conversion methods. However, it had not been actively maintained in the two years prior to the time this survey was conducted. https://github.com/nengo/nengo-dl
SpykeTorch [483]	STDP for supervised learning, R-STDP for model-free RL	Python programming	This framework is built on PyTorch and supports convolutional spiking neural networks that restrict each neuron to at most one spike, using rank-order encoding. It had not been actively maintained for the four years prior to the time this survey was conducted. https://github.com/miladmozafari/SpykeTorch
BindsNet [484]	STDP, ANN2SNN conversion for supervised learning; R-STDP for model-free RL	Python programming	BindsNET is a Python package for simulating spiking neural networks on CPUs or GPUs using PyTorch tensor operations. It is designed to support the development of biologically inspired machine learning algorithms and was actively maintained at the time this survey was conducted. https://github.com/BindsNET/bindsnet
Lava [485]	Hardware deployment tools	Python programming	Lava is an open-source framework for developing applications on neuromorphic hardware platforms. It supports conventional CPUs as well as Intel's Loihi architecture, and its compiler and runtime are designed to be extensible to additional hardware backends. It was actively maintained at the time this survey was conducted. https://github.com/lava-nc/lava
lava-dl [486]	ANN2SNN conversion, surrogate gradient BP for supervised learning	Python programming	lava-dl is a deep learning library within the Lava framework that provides tools for offline training, online training, and inference for a variety of Deep Event-Based Networks. It was actively maintained at the time this survey was conducted. https://github.com/lava-nc/lava
Slayer PyTorch [145]	Surrogate gradient BP for supervised learning	Python programming	This repository contains the original implementation from the SLAYER paper, which addresses the temporal credit assignment problem in spiking neural networks by enabling backpropagation through time using surrogate gradients. It had not been actively maintained for the five years preceding this survey, and its maintenance and development have since been transitioned to the lava-dl [486] framework. https://github.com/bamsumit/slayerPytorch
snnTorch [23]	STDP, Surrogate gradient BP for supervised learning	Python programming	snnTorch is a Python library that enables gradient-based training of spiking neural networks. Built on top of PyTorch, it leverages GPU-accelerated tensor computation for efficient simulation of spiking neuron networks, with ready-to-use neuron models integrated directly into the PyTorch workflow and operable as recurrent activation units. It was actively maintained at the time this survey was conducted. https://github.com/jeshraghian/snnTorch
BrainPy [487]	STDP, surrogate gradient BP, online and offline learning for supervised learning	Python programming	BrainPy is a flexible, efficient, and extensible framework for computational neuroscience and brain-inspired computing, built around Just-In-Time (JIT) compilation. It offers an integrated ecosystem for brain dynamics programming, supporting model construction, simulation, training, and analysis. It was actively maintained at the time this survey was conducted. https://github.com/brainpy/BrainPy

biological neuron theories to the sophisticated synaptic modeling and training methodologies used today. By bridging the gap between neuroscience and machine learning, SNNs offer a high-performance, low-power alternative essential for the next generation of scientific advancement in many fields, such as computer vision, graph processing, natural language processing and RL. Our exploration of various frameworks highlights a maturing software ecosystem that facilitates the deployment of these temporally dynamic models onto neuromorphic hardware. As we look toward the future, the convergence of advanced neuromorphic hardware and refined spike-based learning algorithms promises to redefine the boundaries of artificial intelligence.

ACKNOWLEDGMENTS

We are sincerely thankful for the constructive criticisms and insights of reviewers for the betterment of our research.

REFERENCES

[1] I. Goodfellow, Y. Bengio, and A. Courville, "Deep learning." MIT press, 2016.

[2] N. T. Luu, C. Onuoha, and T. C. Thang, "Blind image quality assessment with multimodal prompt learning," in 2023 IEEE 15th International Conference on Computational Intelligence and Communication Networks (CICN). IEEE, 2023, pp. 614–618.

[3] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga et al., "Pytorch: An imperative style, high-performance deep learning library," Advances in neural information processing systems, vol. 32, 2019.

TABLE 11: List of available open-source software frameworks for SNN simulation with description and URL for installation/documentation (part 2).

Framework	Training Paradigm	How to use	Description
Norse [488]	BPTT for supervised leanrning	Python programming	Norse is designed to leverage the benefits of biologically inspired neural components, which are sparse and event-driven, distinguishing them from conventional artificial neural networks. It extends PyTorch with a set of primitives for building such bio-inspired neural systems and was actively maintained at the time this survey was conducted. https://github.com/norse/norse.
snn_toolbox [122]	ANN conversion for supervised learning	Python programming	The SNN Conversion Toolbox (SNN-TB) is a framework for converting rate-based artificial neural networks into spiking neural networks and executing them with different spike encoding schemes. It had not been actively maintained for the four years prior to the time this survey was conducted. https://github.com/NeuromorphicProcessorProject/snn_toolbox
GeNN [489]	ANN2SNN conversion, STDP for supervised learning	Python, C/C++ programming	GeNN is a GPU-accelerated neural network simulation environment that uses code generation for NVIDIA CUDA and AMD HIP. Developed in Python and C/C++, it allows users to customize models during development using GeNNCode, a C-like specification language. It was actively maintained at the time this survey was conducted. https://genn-team.github.io/.
PySNN [490]	STDP for supervise learning	Python programming	This is a spiking neural network framework built on top of PyTorch for efficient simulation on both CPU and GPU. It is designed primarily for correlation-based learning methods and follows PyTorch's highly modular and flexible architecture, eliminating the need for users to learn a new framework. It had not been actively maintained for the six years prior to the time this survey was conducted. https://github.com/Basuller/PySNN.
CARLsim [491]	STP, STDP for supervised learning	C/C++ programming	CARLsim is a GPU-accelerated, user-friendly library for simulating large-scale spiking neural network models with substantial biological realism. It supports the execution of networks composed of Izhikevich neurons with realistic synaptic dynamics on both standard x86 CPUs and commodity GPUs. Offering a PyNN-like interface in C/C++, it enables detailed specification of parameters at the synapse, neuron, and network levels. It had not been actively maintained for the eight years prior to the time this survey was conducted. https://github.com/UCI-CARL/CARLsim3.
Auryn [142]	STDP for supervised learning	C++ programming	Auryn is a simulator designed for recurrent spiking neural networks with support for synaptic plasticity. It had not been actively maintained for the nine months prior to the time this survey was conducted. https://github.com/fzenke/auryn.
NeuCube [492]	STDP for supervised learning	MATLAB, Java, Python (PyNN), C++ programming	NeuCube is a pioneering development environment and computational architecture for creating brain-inspired artificial intelligence, supporting applications across multiple domains. It is available on platforms including PCs, SpiNNaker, GPUs, and cloud-based systems, and was actively maintained at the time this survey was conducted. Dedicated software installations: https://kedri.aut.ac.nz/R-and-D-Systems/neucube, MATLAB source: https://github.com/KEDRI-AUT/NeuCube-Matlab, Python source: https://github.com/KEDRI-AUT/NeuCube-Py
FNS [493]	STDP for supervised learning	Interactive UI with Python and MATLAB programming	FNS is an event-driven spiking neural network framework written in Java and oriented toward data-driven brain simulations. It integrates detailed spiking and synaptic modeling with an event-driven execution scheme, enabling heterogeneous modules, multi-scale connectivity, delayed connections, and plastic synapses while maintaining efficient simulation performance. It had not been actively maintained for the five years prior to the time this survey was conducted. https://www.fnsneuralsimulator.org/.
SpikingJelly [124]	STDP, Surrogate gradient BP for supervised learning, R-STDP for model-free RL	Python programming	SpikingJelly is an open-source deep learning framework for SNN based on PyTorch with a large base of recent SOTA SNN architecture implementations and training methods for modern SNN. The framework is being actively maintained by the time this survey was conducted. https://github.com/fangwei123456/spikingjelly.
BrainCog [494]	STDP, surrogate gradient BP, Hebbian, STP, BCM for supervised learning, R-STDP for model-free RL	Python programming	BrainCog is an open-source spiking neural network–based cognitive intelligence platform designed for brain-inspired artificial intelligence, embodied AI, and brain simulation. It was actively maintained at the time this survey was conducted. https://github.com/BrainCog-X/Brain-Cog

[4] B. Rokh, A. Azarpeyvand, and A. Khanteymoori, "A comprehensive survey on model quantization for deep neural networks in image classification," ACM Transactions on Intelligent Systems and Technology, vol. 14, no. 6, pp. 1–50, 2023.

[5] Y. He and L. Xiao, "Structured pruning for deep convolutional neural networks: A survey," IEEE transactions on pattern analysis and machine intelligence, vol. 46, no. 5, pp. 2900–2919, 2023.

[6] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," International journal of computer vision, vol. 129, no. 6, pp. 1789–1819, 2021.

[7] J. D. Nunes, M. Carvalho, D. Carneiro, and J. S. Cardoso, "Spiking neural networks: A survey," IEEE Access, vol. 10, pp. 60 738–60 764, 2022.

[8] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," IEEE Signal Processing Magazine, vol. 36, no. 6, pp. 51–63, 2019.

[9] W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski, "Neuronal dynamics: From single neurons to networks and models of cognition." Cambridge University Press, 2014.

[10] A. A. Ratnaparkhi, E. Pilli, and R. Joshi, "Survey of scaling platforms for deep neural networks," in 2016 International Conference on Emerging Trends in Communication Technologies (ETCT). IEEE, 2016, pp. 1–6.

[11] M. Davies, A. Wild, G. Orchard, Y. Sandamirskaya, G. A. F. Guerra, P. Joshi, P. Plank, and S. R. Risbud, "Advancing neuromorphic computing with loihi: A survey of results and outlook," Proceedings of the IEEE, vol. 109, no. 5, pp. 911–934, 2021.

[12] A. Basu, L. Deng, C. Frenkel, and X. Zhang, "Spiking neural network integrated circuits: A review of trends and future directions," in 2022 IEEE custom integrated circuits conference (CICC). IEEE, 2022, pp. 1–8.

[13] D. Khodagholy, J. N. Gelinas, T. Thesen, W. Doyle, O. Devinsky, G. G. Malliaras, and G. Buzsáki, "Neurogrid: recording action potentials from

the surface of the brain," Nature neuroscience, vol. 18, no. 2, pp. 310–315, 2015.
[14] N. Luu, D. Luu, P. N. Nam, and T. C. Thang, "Improvement of spiking neural network with bit plane coding," in 2024 IEEE 16th International Conference on Computational Intelligence and Communication Networks (CICN). IEEE, 2024.
[15] L. Trong Nhan, L. Trung Duong, P. Ngoc Nam, and T. Cong Thang, "Improvement of spiking neural network with bit planes and color models," IEEE Access, vol. 13, pp. 198 607–198 622, 2025.
[16] L. Deng, Y. Wu, X. Hu, L. Liang, Y. Ding, G. Li, G. Zhao, P. Li, and Y. Xie, "Rethinking the performance comparison between snns and anns," Neural networks, vol. 121, pp. 294–307, 2020.
[17] N. T. Luu, "Accuracy-robustness trade off via spiking neural network gradient sparsity trail," arXiv preprint arXiv:2509.23762, 2025.
[18] B. Rajendran, A. Sebastian, M. Schmuker, N. Srinivasa, and E. Eleftheriou, "Low-power neuromorphic hardware for signal processing applications: A review of architectural and system-level design approaches," IEEE Signal Processing Magazine, vol. 36, no. 6, pp. 97–110, 2019.
[19] M. Dampfhoffer, T. Mesquida, A. Valentian, and L. Anghel, "Are snns really more energy-efficient than anns? an in-depth hardware-aware study," IEEE Transactions on Emerging Topics in Computational Intelligence, vol. 7, no. 3, pp. 731–741, 2022.
[20] D.-A. Nguyen, X.-T. Tran, and F. Iacopi, "A review of algorithms and hardware implementations for spiking neural networks," Journal of Low Power Electronics and Applications, vol. 11, no. 2, p. 23, 2021.
[21] Queensland Brain Institute, "Axons and cable transmission in neurons," Online resource, Mar 2021, accessed: 2025-11-29. [Online]. Available: https://qbi.uq.edu.au/brain/brain-anatomy/axons-cable-transmission-neurons
[22] K. Yamazaki, V.-K. Vo-Ho, D. Bulsara, and N. Le, "Spiking neural networks and their applications: A review," Brain sciences, vol. 12, no. 7, p. 863, 2022.
[23] J. K. Eshraghian, M. Ward, E. O. Neftci, X. Wang, G. Lenz, G. Dwivedi, M. Bennamoun, D. S. Jeong, and W. D. Lu, "Training spiking neural networks using lessons from deep learning," Proceedings of the IEEE, 2023.
[24] A. Tavanaei, M. Ghodrati, S. R. Kheradpisheh, T. Masquelier, and A. Maida, "Deep learning in spiking neural networks," Neural networks, vol. 111, pp. 47–63, 2019.
[25] C. Zhou, H. Zhang, L. Yu, Y. Ye, Z. Zhou, L. Huang, Z. Ma, X. Fan, H. Zhou, and Y. Tian, "Direct training high-performance deep spiking neural networks: a review of theories and methods," Frontiers in Neuroscience, vol. 18, p. 1383844, 2024.
[26] A. H. Khan, X. Cao, C. Luo, S. Zhang, W. Guo, V. N. Katsikis, and S. Li, "Spiking neural networks: a comprehensive survey of training methodologies, hardware implementations and applications," Artificial Intelligence Science and Engineering, vol. 1, no. 3, pp. 175–207, 2025.
[27] K. Svoboda and T. Adegbija, "Spiking neural network architecture search: A survey," 2026. [Online]. Available: https://arxiv.org/abs/2510.14235
[28] M. Karamimanesh, E. Abiri, M. Shahsavari, K. Hassanli, A. van Schaik, and J. Eshraghian, "Spiking neural networks on fpga: A survey of methodologies and recent advancements," Neural Networks, vol. 186, p. 107256, 2025.
[29] S. Deng, D. Yu, C. Lv, X. Du, L. Jiang, X. Zhao, W. Tong, X. Zheng, W. Fang, P. Zhao et al., "Edge intelligence with spiking neural networks," arXiv preprint arXiv:2507.14069, 2025.
[30] E. R. Kandel, J. H. Schwartz, T. M. Jessell, S. Siegelbaum, A. J. Hudspeth, S. Mack et al., "Principles of neural science." McGraw-hill New York, 2000, vol. 4.
[31] M. Bear, B. Connors, and M. A. Paradiso, in Neuroscience: Exploring the brain, enhanced edition: Exploring the brain. Jones & Bartlett Learning, 2020.
[32] P. Dayan and L. F. Abbott, "Theoretical neuroscience: computational and mathematical modeling of neural systems." MIT press, 2005.
[33] A. L. Hodgkin and A. F. Huxley, "A quantitative description of membrane current and its application to conduction and excitation in nerve," The Journal of physiology, vol. 117, no. 4, p. 500, 1952.
[34] M. London and M. Häusser, "Dendritic computation," Annual Review of Neuroscience, vol. 28, no. 1, p. 503–532, Jul. 2005. [Online]. Available: http://dx.doi.org/10.1146/annurev.neuro.28.061604.135703
[35] P. Poirazi, T. Brannon, and B. W. Mel, "Pyramidal neuron as two-layer neural network," Neuron, vol. 37, no. 6, p. 989–999, Mar. 2003. [Online]. Available: http://dx.doi.org/10.1016/S0896-6273(03)00149-1
[36] S. Chavlis and P. Poirazi, "Dendrites endow artificial neural networks with accurate, robust and parameter-efficient learning," Nature Communications, vol. 16, no. 1, Jan. 2025. [Online]. Available: http://dx.doi.org/10.1038/s41467-025-56297-9
[37] M. Pagkalos, S. Chavlis, and P. Poirazi, "Introducing the dendrify framework for incorporating dendrites to spiking neural networks," Nature Communications, vol. 14, no. 1, Jan. 2023. [Online]. Available: http://dx.doi.org/10.1038/s41467-022-35747-8
[38] G. Eyal, M. B. Verhoog, G. Testa-Silva, Y. Deitcher, R. Benavides-Piccione, J. DeFelipe, C. P. J. de Kock, H. D. Mansvelder, and I. Segev, "Human cortical pyramidal neurons: From spines to spikes via models," Frontiers in Cellular Neuroscience, vol. Volume 12 - 2018, 2018. [Online]. Available: https://www.frontiersin.org/journals/cellular-neuroscience/articles/10.3389/fncel.2018.00181
[39] L. Beaulieu-Laroche, E. H. Toloza, M.-S. van der Goes, M. Lafourcade, D. Barnagian, Z. M. Williams, E. N. Eskandar, M. P. Frosch, S. S. Cash, and M. T. Harnett, "Enhanced dendritic compartmentalization in human cortical neurons," Cell, vol. 175, no. 3, pp. 643–651.e14, Oct. 2018. [Online]. Available: http://dx.doi.org/10.1016/j.cell.2018.08.045
[40] C. Liu, J. Ma, S. Li, and D. Zhou, "Dendritic integration inspired artificial neural networks capture data correlation," in Advances in Neural Information Processing Systems, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 79 325–79 349. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/90b31ad371165eaac2dc6de8993fded7-Paper-Conference.pdf
[41] A. Morabito, Y. Zerlaut, D. Dhanasobhon, E. Berthaux, C. M. Pinho, A. Tzilivaki, G. Moneron, L. Cathala, P. Poirazi, A. Bacci, D. A. DiGregorio, J. Lourenço, and N. Rebola, "Distinct dendritic integration strategies control dynamics of inhibition in the neocortex," Neuron, vol. 113, no. 18, pp. 2962–2978.e10, Sep. 2025. [Online]. Available: http://dx.doi.org/10.1016/j.neuron.2025.05.029
[42] I. Neutelings, "Neural networks," Online resource, 2021, accessed: 2025-11-29. [Online]. Available: https://tikz.net/neural_networks/
[43] S. Haykin and N. Network, "A comprehensive foundation," Neural networks, vol. 2, no. 2004, p. 41, 2004.
[44] M. A. Boyacioglu, Y. Kara, and Ö. K. Baykan, "Predicting bank financial failures using neural networks, support vector machines and multivariate statistical methods: A comparative analysis in the sample of savings deposit insurance fund (sdif) transferred banks in turkey," Expert systems with applications, vol. 36, no. 2, pp. 3355–3366, 2009.
[45] N. T. Luu, T. C. Truong, and D. T. Luu, "Universal quantum tomography with deep neural networks," arXiv preprint arXiv:2407.01734, 2024.
[46] H. Rahmanifard and T. Plaksina, "Application of artificial intelligence techniques in the petroleum industry: a review," Artificial Intelligence Review, vol. 52, no. 4, pp. 2295–2318, 2019.
[47] O. Araque, I. Corcuera-Platas, J. F. Sánchez-Rada, and C. A. Iglesias, "Enhancing deep learning sentiment analysis with ensemble techniques in social applications," Expert Systems with Applications, vol. 77, pp. 236–246, 2017.
[48] M. Jahnavi, "Introduction to neural networks, advantages and applications, towards data science (jul 9," 2017.
[49] D. Wang, H. He, and D. Liu, "Adaptive critic nonlinear robust control: A survey," IEEE transactions on cybernetics, vol. 47, no. 10, pp. 3429–3451, 2017.
[50] S. Haykin, "Neural networks and learning machines, 3/e," 2009.
[51] ——, "Neural networks expand sp's horizons," IEEE Signal Processing Magazine, vol. 13, no. 2, pp. 24–49, 2002.
[52] F. Amato, A. López, E. M. Peña-Méndez, P. Vaňhara, A. Hampl, and J. Havel, "Artificial neural networks in medical diagnosis," pp. 47–58, 2013.
[53] G. Dede and M. H. Sazlı, "Speech recognition with artificial neural networks," Digital Signal Processing, vol. 20, no. 3, pp. 763–768, 2010.
[54] T. H. Le, "Applying artificial neural networks for face recognition," Advances in Artificial Neural Systems, vol. 2011, no. 1, p. 673016, 2011.
[55] S. Sevgican, M. Turan, K. Gökarslan, H. B. Yilmaz, and T. Tugcu, "Intelligent network data analytics function in 5g cellular networks using machine learning," Journal of Communications and Networks, vol. 22, no. 3, pp. 269–280, 2020.
[56] K. B. Prakash, M. D. Rangaswamy, and A. Raja Raman, "Ann for multilingual regional web communication," in International Conference on Neural Information Processing. Springer, 2012, pp. 473–478.

[57] R. D. Singh, A. Mittal, and R. K. Bhatia, "3d convolutional neural network for object recognition: a review," Multimedia Tools and Applications, vol. 78, no. 12, pp. 15 951–15 995, 2019.

[58] C. Stergiou and D. Siganos, "Neural networks and their uses," Surprise 96 Journal, vol. 4, 1996.

[59] W. Maass, "Networks of spiking neurons: the third generation of neural network models," Neural networks, vol. 10, no. 9, pp. 1659–1671, 1997.

[60] W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity," The Bulletin of Mathematical Biophysics, vol. 5, no. 4, p. 115–133, Dec. 1943. [Online]. Available: http://dx.doi.org/10.1007/BF02478259

[61] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain." Psychological Review, vol. 65, no. 6, p. 386–408, 1958. [Online]. Available: http://dx.doi.org/10.1037/h0042519

[62] M. Minsky and S. A. Papert, Perceptrons: An Introduction to Computational Geometry. The MIT Press, Sep. 2017. [Online]. Available: http://dx.doi.org/10.7551/mitpress/11301.001.0001

[63] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," nature, vol. 323, no. 6088, pp. 533–536, 1986.

[64] J. J. Hopfield, "Neural networks and physical systems with emergent collective computational abilities." Proceedings of the national academy of sciences, vol. 79, no. 8, pp. 2554–2558, 1982.

[65] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in Proceedings of International Conference on Machine Learning (ICML), 2010, inproceedings, pp. 807–814.

[66] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning internal representations by error propagation," Tech. Rep., 1985.

[67] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proceedings of the IEEE conference on computer vision and pattern recognition, 2016, pp. 770–778.

[68] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," Advances in neural information processing systems, vol. 30, 2017.

[69] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, A. Bridgland, C. Meyer, S. A. A. Kohl, A. J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A. W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis, "Highly accurate protein structure prediction with alphafold," Nature, vol. 596, no. 7873, p. 583–589, Jul. 2021. [Online]. Available: http://dx.doi.org/10.1038/s41586-021-03819-2

[70] S. Ghosh-Dastidar and H. Adeli, "Spiking neural networks," International journal of neural systems, vol. 19, no. 04, pp. 295–308, 2009.

[71] N. C. Thompson, K. Greenewald, K. Lee, and G. F. Manso, "The computational limits of deep learning," arXiv preprint arXiv:2007.05558, vol. 10, 2020.

[72] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," arXiv preprint arXiv:2001.08361, 2020.

[73] M. Horowitz, "1.1 computing's energy problem (and what we can do about it)," in 2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC). IEEE, Feb. 2014, p. 10–14. [Online]. Available: http://dx.doi.org/10.1109/ISSCC.2014.6757323

[74] N. T. Luu, D. T. Luu, P. N. Nam, and T. C. Thang, "Hybrid layer-wise ann-snn with surrogate spike encoding-decoding structure," arXiv preprint arXiv:2509.24411, 2025.

[75] E. M. Izhikevich, "Which model to use for cortical spiking neurons?" IEEE transactions on neural networks, vol. 15, no. 5, pp. 1063–1070, 2004.

[76] M. E. Nelson, "Electrophysiological models," Databasing the brain: from data to knowledge, pp. 285–301, 2004.

[77] L. F. Abbott, "Lapicque's introduction of the integrate-and-fire model neuron (1907)," Brain research bulletin, vol. 50, no. 5-6, pp. 303–304, 1999.

[78] S. R. Kheradpisheh, M. Ganjtabesh, S. J. Thorpe, and T. Masquelier, "Stdp-based spiking deep convolutional neural networks for object recognition," Neural Networks, vol. 99, pp. 56–67, 2018.

[79] P. U. Diehl and M. Cook, "Unsupervised learning of digit recognition using spike-timing-dependent plasticity," Frontiers in Computational Neuroscience, vol. Volume 9 - 2015, 2015. [Online]. Available: https://www.frontiersin.org/journals/computational-neuroscience/articles/10.3389/fncom.2015.00099

[80] S. Y. Woo, S. Pak, and S.-T. Lee, "Deep spiking neural networks with integrate and fire neuron using steep switching device," Solid-State Electronics, vol. 214, p. 108860, 2024.

[81] L. Shupe and E. Fetz, "An integrate-and-fire spiking neural network model simulating artificially induced cortical plasticity," Eneuro, vol. 8, no. 2, 2021.

[82] W. Gerstner and W. M. Kistler, "Spiking neuron models: Single neurons, populations, plasticity." Cambridge university press, 2002.

[83] Y. Hu, H. Tang, and G. Pan, "Spiking deep residual networks," IEEE Transactions on Neural Networks and Learning Systems, vol. 34, no. 8, pp. 5200–5205, 2021.

[84] W. Fang, Z. Yu, Y. Chen, T. Huang, T. Masquelier, and Y. Tian, "Deep residual learning in spiking neural networks," Advances in Neural Information Processing Systems, vol. 34, pp. 21 056–21 069, 2021.

[85] D. Rasmussen, "NengoDL: Combining deep learning and neuromorphic modelling methods," arXiv, vol. 1805.11144, pp. 1–22, 2018. [Online]. Available: http://arxiv.org/abs/1805.11144

[86] W. Gerstner, "Spike-response model," Scholarpedia, vol. 3, no. 12, p. 1343, 2008.

[87] S. M. Bohte, J. N. Kok, and J. A. La Poutré, "Spikeprop: backpropagation for networks of spiking neurons." in ESANN, vol. 48. Bruges, 2000, pp. 419–424.

[88] R. Gütig and H. Sompolinsky, "The tempotron: a neuron that learns spike timing–based decisions," Nature neuroscience, vol. 9, no. 3, pp. 420–428, 2006.

[89] A. R. R. Casti, A. Omurtag, A. Sornborger, E. Kaplan, B. Knight, J. Victor, and L. Sirovich, "A population study of integrate-and-fire-or-burst neurons," Neural computation, vol. 14, no. 5, pp. 957–986, 2002.

[90] A. J. Langdon, M. Breakspear, and S. Coombes, "Phase-locked cluster oscillations in periodically forced integrate-and-fire-or-burst neuronal populations," Physical Review E—Statistical, Nonlinear, and Soft Matter Physics, vol. 86, no. 6, p. 061903, 2012.

[91] D. H. Elijah and M. A. Montemurro, "Burst structure can code different stimulus features in thalamic neuron models," BMC Neuroscience, vol. 10, no. Suppl 1, p. P88, 2009.

[92] S. Coombes, M. R. Owen, and G. D. Smith, "Mode locking in a periodically forced integrate-and-fire-or-burst neuron model," Physical Review E, vol. 64, no. 4, p. 041914, 2001.

[93] E. M. Izhikevich, "Resonate-and-fire neurons," Neural networks, vol. 14, no. 6-7, pp. 883–894, 2001.

[94] K. Nakada, T. Asai, and H. Hayashi, "Analog vlsi implementation of resonate-and-fire neuron," International journal of neural systems, vol. 16, no. 06, pp. 445–456, 2006.

[95] A. Liu, F. Moro, S. Billaudelle, and M. Payvand, "A linear implementation of an analog resonate-and-fire neuron," arXiv preprint arXiv:2511.12297, 2025.

[96] L. Marangio, S. Galatolo, L. Fronzoni, S. Chillemi, and A. Di Garbo, "Phase-locking patterns in a resonate and fire neural model with periodic drive," Biosystems, vol. 184, p. 103992, 2019.

[97] B. AlKhamissi, M. ElNokrashy, and D. Bernal-Casas, "Deep spiking neural networks with resonate-and-fire neurons," arXiv preprint arXiv:2109.08234, 2021.

[98] T. E. Huber, J. Lecomte, B. Polovnikov, and A. von Arnim, "Scaling up resonate-and-fire networks for fast deep learning," in European Conference on Computer Vision. Springer, 2024, pp. 241–258.

[99] Y. Zerlaut and A. Destexhe, "A mean-field model for conductance-based networks of adaptive exponential integrate-and-fire models," arXiv preprint arXiv:1703.00698, 2017.

[100] E. M. Izhikevich, "Polychronization: computation with spikes," Neural computation, vol. 18, no. 2, pp. 245–282, 2006.

[101] C. Eliasmith, T. C. Stewart, X. Choo, T. Bekolay, T. DeWolf, Y. Tang, and D. Rasmussen, "A large-scale model of the functioning brain," science, vol. 338, no. 6111, pp. 1202–1205, 2012.

[102] M. C. Avery, N. Dutt, and J. L. Krichmar, "A large-scale neural network model of the influence of neuromodulatory levels on working memory and behavior," Frontiers in computational neuroscience, vol. 7, p. 133, 2013.

[103] X. Fang, S. Duan, and L. Wang, "Memristive izhikevich spiking neuron model and its application in oscillatory associative memory," Frontiers in Neuroscience, vol. 16, p. 885322, 2022.

[104] T. Kubota, K. Nakajima, and H. Takahashi, "Information processing using a single izhikevich neuron," in Artificial Life Conference Proceedings. MIT Press One Rogers Street, Cambridge, MA 02142-1209, USA journals-info . . . , 2018, pp. 550–557.
[105] L. Chen and S. A. Campbell, "Exact mean-field models for spiking neural networks with adaptation," Journal of Computational Neuroscience, vol. 50, no. 4, pp. 445–469, 2022.
[106] R. Brette and W. Gerstner, "Adaptive exponential integrate-and-fire model as an effective description of neuronal activity," Journal of neurophysiology, vol. 94, no. 5, pp. 3637–3642, 2005.
[107] S. Millner, A. Grübl, K. Meier, J. Schemmel, and M.-O. Schwartz, "A vlsi implementation of the adaptive exponential integrate-and-fire neuron model," Advances in neural information processing systems, vol. 23, 2010.
[108] R. Naud, N. Marcille, C. Clopath, and W. Gerstner, "Firing patterns in the adaptive exponential integrate-and-fire model," Biological cybernetics, vol. 99, no. 4, pp. 335–347, 2008.
[109] T. Górski, D. Depannemaecker, and A. Destexhe, "Conductance-based adaptive exponential integrate-and-fire model," Neural Computation, vol. 33, no. 1, pp. 41–66, 2021.
[110] E. Lewis, "Neuroelectric potentials derived from an extended version of the hodgkin-huxley model," Journal of theoretical biology, vol. 10, no. 1, pp. 125–158, 1966.
[111] B. A. y Arcas, A. L. Fairhall, and W. Bialek, "Computation in a single neuron: Hodgkin and huxley revisited," Neural computation, vol. 15, no. 8, pp. 1715–1749, 2003.
[112] A. Moujahid, A. d'Anjou, F. Torrealdea, and F. Torrealdea, "Energy and information in hodgkin-huxley neurons," Physical Review E—Statistical, Nonlinear, and Soft Matter Physics, vol. 83, no. 3, p. 031912, 2011.
[113] J. H. Goldwyn, N. S. Imennov, M. Famulare, and E. Shea-Brown, "Stochastic differential equation models for ion channel noise in hodgkin-huxley neurons," Physical Review E—Statistical, Nonlinear, and Soft Matter Physics, vol. 83, no. 4, p. 041908, 2011.
[114] T. Wang, Y. Wang, J. Shen, L. Wang, and L. Cao, "Predicting spike features of hodgkin-huxley-type neurons with simple artificial neural network," Frontiers in computational neuroscience, vol. 15, p. 800875, 2022.
[115] X. Fang, S. Duan, and L. Wang, "Memristive hodgkin-huxley spiking neuron model for reproducing neuron behaviors," Frontiers in Neuroscience, vol. 15, p. 730566, 2021.
[116] A. Amirsoleimani, M. Ahmadi, A. Ahmadi, and M. Boukadoum, "Brain-inspired pattern classification with memristive neural network using the hodgkin-huxley neuron," in 2016 IEEE International Conference on Electronics, Circuits and Systems (ICECS). IEEE, 2016, pp. 81–84.
[117] Y. Xu, S. Gao, Z. Li, R. Yang, and X. Miao, "Adaptive hodgkin–huxley neuron for retina-inspired perception," Advanced Intelligent Systems, vol. 4, no. 12, p. 2200210, 2022.
[118] H. Yedjour, B. Meftah, D. Yedjour, and O. Lézoray, "The hodgkin–huxley neuron model for motion detection in image sequences," Neural Computing and Applications, vol. 34, no. 2, pp. 1123–1133, 2022.
[119] C. Meunier and I. Segev, "Playing the devil's advocate: is the hodgkin–huxley model useful?" TRENDS in Neurosciences, vol. 25, no. 11, pp. 558–563, 2002.
[120] A. F. Strassberg and L. J. DeFelice, "Limitations of the hodgkin-huxley formalism: Effects of single channel kinetics on transmembrane voltage dynamics," Neural computation, vol. 5, no. 6, pp. 843–855, 1993.
[121] Y. Cao, Y. Chen, and D. Khosla, "Spiking deep convolutional neural networks for energy-efficient object recognition," International Journal of Computer Vision, vol. 113, no. 1, pp. 54–66, 2015.
[122] B. Rueckauer, I.-A. Lungu, Y. Hu, M. Pfeiffer, and S.-C. Liu, "Conversion of continuous-valued deep networks to efficient event-driven networks for image classification," Frontiers in neuroscience, vol. 11, p. 682, 2017.
[123] P. U. Diehl, D. Neil, J. Binas, M. Cook, S.-C. Liu, and M. Pfeiffer, "Fast-classifying, high-accuracy spiking deep networks through weight and threshold balancing," in 2015 International joint conference on neural networks (IJCNN). ieee, 2015, pp. 1–8.
[124] W. Fang, Y. Chen, J. Ding, Z. Yu, T. Masquelier, D. Chen, L. Huang, H. Zhou, G. Li, and Y. Tian, "Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence," Science Advances, vol. 9, no. 40, p. eadi1480, 2023. [Online]. Available: https://www.science.org/doi/abs/10.1126/sciadv.adi1480
[125] W. Fang, Z. Yu, Y. Chen, T. Masquelier, T. Huang, and Y. Tian, "Incorporating learnable membrane time constant to enhance learning of spiking neural networks," in Proceedings of the IEEE/CVF international conference on computer vision, 2021, pp. 2661–2671.
[126] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday, G. Dimou, P. Joshi, N. Imam, S. Jain et al., "Loihi: A neuromorphic manycore processor with on-chip learning," Ieee micro, vol. 38, no. 1, pp. 82–99, 2018.
[127] P. A. Merolla, J. V. Arthur, R. Alvarez-Icaza, A. S. Cassidy, J. Sawada, F. Akopyan, B. L. Jackson, N. Imam, C. Guo, Y. Nakamura et al., "A million spiking-neuron integrated circuit with a scalable communication network and interface," Science, vol. 345, no. 6197, pp. 668–673, 2014.
[128] E. Painkras, L. A. Plana, J. Garside, S. Temple, F. Galluppi, C. Patterson, D. R. Lester, A. D. Brown, and S. B. Furber, "Spinnaker: A 1-w 18-core system-on-chip for massively-parallel neural network simulation," IEEE Journal of Solid-State Circuits, vol. 48, no. 8, pp. 1943–1953, 2013.
[129] C. Pehle, S. Billaudelle, B. Cramer, J. Kaiser, K. Schreiber, Y. Stradmann, J. Weis, A. Leibfried, E. Müller, and J. Schemmel, "The brainscales-2 accelerated neuromorphic system with hybrid plasticity," Frontiers in Neuroscience, vol. Volume 16 - 2022, 2022. [Online]. Available: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2022.795876
[130] E. M. Izhikevich, "Dynamical systems in neuroscience." MIT press, 2007.
[131] S. Yang, Y. Pang, H. Wang, T. Lei, J. Pan, J. Wang, and Y. Jin, "Spike-driven multi-scale learning with hybrid mechanisms of spiking dendrites," Neurocomputing, vol. 542, p. 126240, 2023.
[132] Q. Zhou, W. Wang, and M. Qiao, "Dendritic spiking neural networks with combined membrane potential decay and dynamic threshold for sequential recognition," Applied Sciences, vol. 16, no. 2, p. 748, 2026.
[133] H. Zheng, Z. Zheng, R. Hu, B. Xiao, Y. Wu, F. Yu, X. Liu, G. Li, and L. Deng, "Temporal dendritic heterogeneity incorporated with spiking neural networks for learning multi-timescale dynamics," Nature Communications, vol. 15, no. 1, p. 277, 2024.
[134] S. Yang, T. Gao, J. Wang, B. Deng, B. Lansdell, and B. Linares-Barranco, "Efficient spike-driven learning with dendritic event-based processing," Frontiers in Neuroscience, vol. 15, p. 601109, 2021.
[135] S. Yang, H. Wang, Y. Pang, M. R. Azghadi, and B. Linares-Barranco, "Nadol: Neuromorphic architecture for spike-driven online learning by dendrites," IEEE Transactions on Biomedical Circuits and Systems, vol. 18, no. 1, pp. 186–199, 2023.
[136] S. V. Stasenko and V. B. Kazantsev, "Information encoding in bursting spiking neural network modulated by astrocytes," Entropy, vol. 25, no. 5, p. 745, 2023.
[137] S. Y. Gordleeva, Y. A. Tsybina, M. I. Krivonosov, M. V. Ivanchenko, A. A. Zaikin, V. B. Kazantsev, and A. N. Gorban, "Modeling working memory in a spiking neuron network accompanied by astrocytes," Frontiers in Cellular Neuroscience, vol. 15, p. 631485, 2021.
[138] S. Gordleeva, Y. A. Tsybina, M. I. Krivonosov, I. Y. Tyukin, V. B. Kazantsev, A. A. Zaikin, and A. N. Gorban, "Situation-based memory in spiking neuron-astrocyte network," arXiv preprint arXiv:2202.07218, 2022.
[139] J. Lorenzo, J.-A. Rico-Gallego, S. Binczak, and S. Jacquir, "Spiking neuron-astrocyte networks for image recognition," Neural computation, vol. 37, no. 4, pp. 635–665, 2025.
[140] C. S. Yang, S. J. Gates, D. De Zoysa, J. Choe, W. Losert, and C. B. Hart, "Characterizing learning in spiking neural networks with artificial astrocytes," Neurocomputing, p. 132805, 2026.
[141] J. Wu, E. Yılmaz, M. Zhang, H. Li, and K. C. Tan, "Deep spiking neural networks for large vocabulary automatic speech recognition," Frontiers in neuroscience, vol. 14, p. 199, 2020.
[142] F. Zenke and W. Gerstner, "Limits to high-speed simulations of spiking neural networks using general-purpose computers," Frontiers in neuroinformatics, vol. 8, p. 76, 2014.
[143] I. Sporea and A. Grüning, "Supervised learning in multilayer spiking neural networks," Neural computation, vol. 25, no. 2, pp. 473–509, 2013.
[144] F. Zenke and S. Ganguli, "Superspike: Supervised learning in multilayer spiking neural networks," Neural computation, vol. 30, no. 6, pp. 1514–1541, 2018.
[145] S. B. Shrestha and G. Orchard, "Slayer: Spike layer error reassignment in time," Advances in neural information processing systems, vol. 31, 2018.
[146] D. Huh and T. J. Sejnowski, "Gradient descent for spiking neural networks," Advances in neural information processing systems, vol. 31, 2018.

[147] J. H. Lee, T. Delbruck, and M. Pfeiffer, "Training deep spiking neural networks using backpropagation," Frontiers in neuroscience, vol. 10, p. 508, 2016.
[148] S. Lian, J. Shen, Q. Liu, Z. Wang, R. Yan, and H. Tang, "Learnable surrogate gradient for direct training spiking neural networks." in Ijcai, 2023, pp. 3002–3010.
[149] Y. Guo, Y. Chen, Z. Hao, W. Peng, Z. Jie, Y. Zhang, X. Liu, and Z. Ma, "Take a shortcut back: Mitigating the gradient vanishing for training spiking neural networks," Advances in Neural Information Processing Systems, vol. 37, pp. 24 849–24 867, 2024.
[150] B. Rueckauer, I.-A. Lungu, Y. Hu, and M. Pfeiffer, "Theory and tools for the conversion of analog to spiking convolutional neural networks," arXiv preprint arXiv:1612.04052, 2016.
[151] S. Deng and S. Gu, "Optimal conversion of conventional artificial neural networks to spiking neural networks," arXiv preprint arXiv:2103.00476, 2021.
[152] T. Bu, W. Fang, J. Ding, P. Dai, Z. Yu, and T. Huang, "Optimal ann-snn conversion for high-accuracy and ultra-low-latency spiking neural networks," arXiv preprint arXiv:2303.04347, 2023.
[153] Y. Li, S. Deng, X. Dong, R. Gong, and S. Gu, "A free lunch from ann: Towards efficient, accurate spiking neural networks calibration," in International conference on machine learning. PMLR, 2021, pp. 6316–6325.
[154] H. Gao, J. He, H. Wang, T. Wang, Z. Zhong, J. Yu, Y. Wang, M. Tian, and C. Shi, "High-accuracy deep ann-to-snn conversion using quantization-aware training framework and calcium-gated bipolar leaky integrate and fire neuron," Frontiers in Neuroscience, vol. 17, p. 1141701, 2023.
[155] Y. Xiao, X. Tian, Y. Ding, P. He, M. Jing, and L. Zuo, "Multi-bit mechanism: A novel information transmission paradigm for spiking neural networks," arXiv preprint arXiv:2407.05739, 2024.
[156] A. Sengupta, Y. Ye, R. Wang, C. Liu, and K. Roy, "Going deeper in spiking neural networks: Vgg and residual architectures," Frontiers in neuroscience, vol. 13, p. 95, 2019.
[157] B. Han and K. Roy, "Deep spiking neural network: Energy efficiency through time based coding," in European conference on computer vision. Springer, 2020, pp. 388–404.
[158] W. Tan, D. Patel, and R. Kozma, "Strategy and benchmark for converting deep q-networks to event-driven spiking neural networks," in Proceedings of the AAAI conference on artificial intelligence, vol. 35, no. 11, 2021, pp. 9816–9824.
[159] J. Wang, X. Deng, W. Wei, D. Zhang, S. Wang, Q. Sun, J. Zhang, H. Liu, N. Xie, and M. Zhang, "Training-free ann-to-snn conversion for high-performance spiking transformers," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 40, no. 3, 2026, pp. 2128–2136.
[160] K. You, Z. Xu, C. Nie, Z. Deng, Q. Guo, X. Wang, and Z. He, "Spikezip-tf: Conversion is all you need for transformer-based snn," arXiv preprint arXiv:2406.03470, 2024.
[161] W. Gerstner, M. Lehmann, V. Liakoni, D. Corneil, and J. Brea, "Eligibility traces and plasticity on behavioral time scales: experimental support of neohebbian three-factor learning rules," Frontiers in neural circuits, vol. 12, p. 53, 2018.
[162] G. Bellec, F. Scherr, A. Subramoney, E. Hajek, D. Salaj, R. Legenstein, and W. Maass, "A solution to the learning dilemma for recurrent networks of spiking neurons," Nature communications, vol. 11, no. 1, p. 3625, 2020.
[163] T. Bohnstingl, S. Woźniak, A. Pantazi, and E. Eleftheriou, "Online spatio-temporal learning in deep neural networks," IEEE Transactions on Neural Networks and Learning Systems, vol. 34, no. 11, pp. 8894–8908, 2022.
[164] M. Xiao, Q. Meng, Z. Zhang, D. He, and Z. Lin, "Online training through time for spiking neural networks," Advances in neural information processing systems, vol. 35, pp. 20 717–20 730, 2022.
[165] M. P. E. Apolinario and K. Roy, "S-tllr: Stdp-inspired temporal local learning rule for spiking neural networks," arXiv preprint arXiv:2306.15220, 2023.
[166] J. Kaiser, H. Mostafa, and E. Neftci, "Synaptic plasticity dynamics for deep continuous local learning (decolle)," Frontiers in Neuroscience, vol. 14, p. 424, 2020.
[167] M. P. Apolinario, K. Roy, and C. Frenkel, "Tess: A scalable temporally and spatially local learning rule for spiking neural networks," in 2025 International Joint Conference on Neural Networks (IJCNN). IEEE, 2025, pp. 1–9.
[168] F. M. Quintana, F. Perez-Peña, P. L. Galindo, E. O. Neftci, E. Chicca, and L. Khacef, "Etlp: event-based three-factor local plasticity for online learning with neuromorphic hardware," Neuromorphic Computing and Engineering, vol. 4, no. 3, p. 034006, 2024.
[169] T. Ortner, L. Pes, J. Gentinetta, C. Frenkel, and A. Pantazi, "Online spatio-temporal learning with target projection," arXiv preprint arXiv:2304.05124, 2023.
[170] C. Frenkel, M. Lefebvre, and D. Bol, "Learning without feedback: Fixed random learning signals allow for feedforward training of deep neural networks," Frontiers in neuroscience, vol. 15, p. 629892, 2021.
[171] G.-q. Bi and M.-m. Poo, "Synaptic modifications in cultured hippocampal neurons: dependence on spike timing, synaptic strength, and postsynaptic cell type," Journal of neuroscience, vol. 18, no. 24, pp. 10 464–10 472, 1998.
[172] N. Caporale and Y. Dan, "Spike timing–dependent plasticity: a hebbian learning rule," Annu. Rev. Neurosci., vol. 31, no. 1, pp. 25–46, 2008.
[173] S. Song, K. D. Miller, and L. F. Abbott, "Competitive hebbian learning through spike-timing-dependent synaptic plasticity," Nature neuroscience, vol. 3, no. 9, pp. 919–926, 2000.
[174] D. E. Feldman, "Synaptic mechanisms for plasticity in neocortex," Annual review of neuroscience, vol. 32, no. 1, pp. 33–55, 2009.
[175] T. Masquelier and S. J. Thorpe, "Unsupervised learning of visual features through spike timing dependent plasticity," PLoS computational biology, vol. 3, no. 2, p. e31, 2007.
[176] C. C. Bell, V. Z. Han, Y. Sugawara, and K. Grant, "Synaptic plasticity in a cerebellum-like structure depends on temporal order," Nature, vol. 387, no. 6630, pp. 278–281, 1997.
[177] K. S. Burbank, "Mirrored stdp implements autoencoder learning in a network of spiking neurons," PLoS computational biology, vol. 11, no. 12, p. e1004566, 2015.
[178] A. Tavanaei, T. Masquelier, and A. S. Maida, "Acquisition of visual features through probabilistic spike-timing-dependent plasticity," in 2016 International Joint Conference on Neural Networks (IJCNN). IEEE, 2016, pp. 307–314.
[179] T. Masquelier, R. Guyonneau, and S. J. Thorpe, "Competitive stdp-based spike pattern learning," Neural computation, vol. 21, no. 5, pp. 1259–1276, 2009.
[180] F. Paredes-Vallés, K. Y. Scheper, and G. C. De Croon, "Unsupervised learning of a hierarchical spiking neural network for optical flow estimation: From events to global motion perception," IEEE transactions on pattern analysis and machine intelligence, vol. 42, no. 8, pp. 2051–2064, 2019.
[181] R. V. Florian, "Reinforcement learning through modulation of spike-timing-dependent synaptic plasticity," Neural computation, vol. 19, no. 6, pp. 1468–1502, 2007.
[182] E. M. Izhikevich, "Solving the distal reward problem through linkage of stdp and dopamine signaling," Cerebral cortex, vol. 17, no. 10, pp. 2443–2452, 2007.
[183] F. Ponulak, "Resume-new supervised learning method for spiking neural networks. institute of control and information engineering, poznoń university of technology," Tech. rep, Tech. Rep., 2005.
[184] A. Tavanaei and A. Maida, "Bp-stdp: Approximating backpropagation using spike timing dependent plasticity," Neurocomputing, vol. 330, pp. 39–47, 2019.
[185] F. Liu, W. Zhao, Y. Chen, Z. Wang, T. Yang, and L. Jiang, "Sstdp: Supervised spike timing dependent plasticity for efficient spiking neural network training," Frontiers in Neuroscience, vol. 15, p. 756876, 2021.
[186] M. Stemmler and C. Koch, "How voltage-dependent conductances can adapt to maximize the information encoded by neuronal firing rate," Nature neuroscience, vol. 2, no. 6, pp. 521–527, 1999.
[187] X. Li, W. Wang, F. Xue, and Y. Song, "Computational modeling of spiking neural network with learning rules from stdp and intrinsic plasticity," Physica A: Statistical Mechanics and its Applications, vol. 491, pp. 716–728, 2018.
[188] C. Lee, P. Panda, G. Srinivasan, and K. Roy, "Training deep spiking convolutional neural networks with stdp-based unsupervised pre-training followed by supervised fine-tuning," Frontiers in neuroscience, vol. 12, p. 435, 2018.
[189] P. Panda and K. Roy, "Unsupervised regenerative learning of hierarchical features in spiking deep networks for object recognition," in 2016 international joint conference on neural networks (IJCNN). IEEE, 2016, pp. 299–306.
[190] Y. Guo, W. Peng, Y. Chen, L. Zhang, X. Liu, X. Huang, and Z. Ma, "Joint a-snn: Joint training of artificial and spiking neural networks via self-distillation and weight factorization," Pattern Recognition, vol. 142, p. 109639, 2023.

[191] C. Lv, T. Li, J. Xu, C. Gu, Z. Ling, C. Zhang, X. Zheng, and X. Huang, "Spikebert: A language spikformer learned from bert with knowledge distillation," arXiv preprint arXiv:2308.15122, 2023.
[192] M. Bal and A. Sengupta, "Spikingbert: Distilling bert to train spiking language models using implicit differentiation," in Proceedings of the AAAI conference on artificial intelligence, vol. 38, no. 10, 2024, pp. 10 998–11 006.
[193] Z. Zhou, Y. Zhu, C. He, Y. Wang, S. Yan, Y. Tian, and L. Yuan, "Spikformer: When spiking neural network meets transformer," arXiv preprint arXiv:2209.15425, 2022.
[194] M. Yao, J. Hu, Z. Zhou, L. Yuan, Y. Tian, B. Xu, and G. Li, "Spike-driven transformer," Advances in neural information processing systems, vol. 36, pp. 64 043–64 058, 2023.
[195] Y. Kim, J. Chough, and P. Panda, "Beyond classification: Directly training spiking neural networks for semantic segmentation," Neuromorphic Computing and Engineering, vol. 2, no. 4, p. 044015, 2022.
[196] Q. Su, Y. Chou, Y. Hu, J. Li, S. Mei, Z. Zhang, and G. Li, "Deep directly-trained spiking neural networks for object detection," in Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 6555–6565.
[197] J. Ding, Z. Yu, J. K. Liu, and T. Huang, "Neuromorphic computing paradigms enhance robustness through spiking neural networks," Nature Communications, vol. 16, no. 1, p. 10175, 2025.
[198] J. Ding, T. Bu, Z. Yu, T. Huang, and J. Liu, "Snn-rat: Robustness-enhanced spiking neural network through regularized adversarial training," Advances in Neural Information Processing Systems, vol. 35, pp. 24 780–24 793, 2022.
[199] Y. Liu, T. Bu, J. Ding, Z. Hao, T. Huang, and Z. Yu, "Enhancing adversarial robustness in snns with sparse gradients," arXiv preprint arXiv:2405.20355, 2024.
[200] N. Perez-Nieves and D. Goodman, "Sparse spiking gradient descent," Advances in Neural Information Processing Systems, vol. 34, pp. 11 795–11 808, 2021.
[201] L. T. Nhan, L. T. Duong, P. N. Nam, and T. C. Thang, "Accuracy-robustness trade off via spiking neural network gradient sparsity trail," arXiv preprint arXiv:2509.23762, 2025.
[202] A. Mehrabi, Y. Bethi, A. Van Schaik, A. Wabnitz, and S. Afshar, "Efficient hardware implementation of a multi-layer gradient-free online-trainable spiking neural network on fpga," IEEE Access, vol. 12, pp. 170 980–170 993, 2024.
[203] D. O. Hebb, The organization of behavior: A neuropsychological theory. Psychology press, 2005.
[204] B. Siddoway, H. Hou, and H. Xia, "Molecular mechanisms of homeostatic synaptic downscaling," Neuropharmacology, vol. 78, pp. 38–44, 2014.
[205] G. Srinivasan, S. Roy, V. Raghunathan, and K. Roy, "Spike timing dependent plasticity based enhanced self-learning for efficient pattern recognition in spiking neural networks," in 2017 International Joint Conference on Neural Networks (IJCNN). IEEE, 2017, pp. 1847–1854.
[206] T. Iakymchuk, A. Rosado-Muñoz, J. F. Guerrero-Martínez, M. Bataller-Mompeán, and J. V. Francés-Víllora, "Simplified spiking neural network architecture and stdp learning algorithm applied to image classification," EURASIP Journal on Image and Video Processing, vol. 2015, no. 1, p. 4, 2015.
[207] J. C. V. Tieck, K. Secker, J. Kaiser, A. Roennau, and R. Dillmann, "Soft-grasping with an anthropomorphic robotic hand using spiking neurons," IEEE Robotics and Automation Letters, vol. 6, no. 2, pp. 2894–2901, 2020.
[208] N. Rathi and K. Roy, "Diet-snn: Direct input encoding with leakage and threshold optimization in deep spiking neural networks," arXiv preprint arXiv:2008.03658, 2020.
[209] ——, "Diet-snn: A low-latency spiking neural network with direct input encoding and leakage and threshold optimization," IEEE Transactions on Neural Networks and Learning Systems, vol. 34, no. 6, pp. 3174–3182, 2021.
[210] J. Kim, H. Kim, S. Huh, J. Lee, and K. Choi, "Deep neural networks with weighted spikes," Neurocomputing, vol. 311, pp. 373–386, 2018.
[211] V. Bojkovic, S. Anumasa, G. De Masi, B. Gu, and H. Xiong, "Data driven threshold and potential initialization for spiking neural networks," in International Conference on Artificial Intelligence and Statistics. PMLR, 2024, pp. 4771–4779.
[212] K. You, Z. Xu, C. Nie, Z. Deng, Q. Guo, X. Wang, and Z. He, "SpikeZIP-TF: Conversion is all you need for transformer-based SNN," in Proceedings of the 41st International Conference on Machine Learning, ser. Proceedings of Machine Learning Research, R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds., vol. 235. PMLR, 21–27 Jul 2024, pp. 57 367–57 383. [Online]. Available: https://proceedings.mlr.press/v235/you24b.html
[213] Z. Zhou, K. Che, W. Fang, K. Tian, Y. Zhu, S. Yan, Y. Tian, and L. Yuan, "Spikformer v2: Join the high accuracy club on imagenet with an snn ticket," arXiv preprint arXiv:2401.02020, 2024.
[214] C. Zhou, L. Yu, Z. Zhou, Z. Ma, H. Zhang, H. Zhou, and Y. Tian, "Spikingformer: Spike-driven residual learning for transformer-based spiking neural network," arXiv preprint arXiv:2304.11954, 2023.
[215] D. Patel, H. Hazan, D. J. Saunders, H. T. Siegelmann, and R. Kozma, "Improved robustness of reinforcement learning policies upon conversion to spiking neuronal network platforms applied to atari breakout game," Neural Networks, vol. 120, pp. 108–115, 2019.
[216] M. Akl, Y. Sandamirskaya, F. Walter, and A. Knoll, "Porting deep spiking q-networks to neuromorphic chip loihi," in International Conference on Neuromorphic Systems 2021, 2021, pp. 1–7.
[217] J. Pedersen et al., "Spinnaker 2: A 10 million core processor system for brain simulation and machine learning," Communicating Process Architectures 2017 & 2018: WoTUG-39 & WoTUG-40, vol. 70, p. 277, 2019.
[218] C. Frenkel and G. Indiveri, "Reckon: A 28nm sub-mm2 task-agnostic spiking recurrent neural network processor enabling on-chip learning over second-long timescales," in 2022 IEEE International Solid-State Circuits Conference (ISSCC), vol. 65. IEEE, 2022, pp. 1–3.
[219] D. Auge, J. Hille, E. Mueller, and A. Knoll, "A survey of encoding techniques for signal processing in spiking neural networks," Neural Processing Letters, vol. 53, no. 6, pp. 4693–4710, 2021.
[220] H. AliAkbarpour, A. Moori, J. Khorramdel, E. Blasch, and O. Tahri, "Emerging trends and applications of neuromorphic dynamic vision sensors: A survey," IEEE Sensors Reviews, 2024.
[221] Y. Kim, H. Park, A. Moitra, A. Bhattacharjee, Y. Venkatesha, and P. Panda, "Rate coding or direct coding: Which one is better for accurate, robust, and energy-efficient spiking neural networks?" in ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2022, pp. 71–75.
[222] E. D. Adrian, "The impulses produced by sensory nerve endings: Part i," The Journal of physiology, vol. 61, no. 1, p. 49, 1926.
[223] E. D. Adrian and Y. Zotterman, "The impulses produced by sensory nerve-endings: Part ii. the response of a single end-organ," The Journal of physiology, vol. 61, no. 2, p. 151, 1926.
[224] W. Guo, M. E. Fouda, A. M. Eltawil, and K. N. Salama, "Neural coding in spiking neural networks: A comparative study for robust neuromorphic systems," Frontiers in Neuroscience, vol. 15, p. 638474, 2021.
[225] D. Zambrano, R. Nusselder, H. S. Scholte, and S. M. Bohté, "Sparse computation in adaptive spiking neural networks," Frontiers in neuroscience, vol. 12, p. 987, 2019.
[226] M. Shafiq and Z. Gu, "Deep residual learning for image recognition: A survey," Applied sciences, vol. 12, no. 18, p. 8972, 2022.
[227] Z.-Q. Zhao, P. Zheng, S.-t. Xu, and X. Wu, "Object detection with deep learning: A review," IEEE transactions on neural networks and learning systems, vol. 30, no. 11, pp. 3212–3232, 2019.
[228] S. Minaee, Y. Boykov, F. Porikli, A. Plaza, N. Kehtarnavaz, and D. Terzopoulos, "Image segmentation using deep learning: A survey," IEEE transactions on pattern analysis and machine intelligence, vol. 44, no. 7, pp. 3523–3542, 2021.
[229] M. Elasri, O. Elharrouss, S. Al-Maadeed, and H. Tairi, "Image generation: A review," Neural Processing Letters, vol. 54, no. 5, pp. 4609–4646, 2022.
[230] W. Zhu, X. Ma, D. Ro, H. Ci, J. Zhang, J. Shi, F. Gao, Q. Tian, and Y. Wang, "Human motion generation: A survey," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 46, no. 4, pp. 2430–2449, 2023.
[231] X. Qiu, M. Yao, J. Zhang, Y. Chou, N. Qiao, S. Zhou, B. Xu, and G. Li, "Efficient 3d recognition with event-driven spike sparse convolution," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, no. 19, 2025, pp. 20 086–20 094.
[232] C. Yu, Z. Gu, D. Li, G. Wang, A. Wang, and E. Li, "Stsc-snn: Spatio-temporal synaptic connection with temporal convolution and attention for spiking neural networks," Frontiers in Neuroscience, vol. Volume 16 - 2022, 2022. [Online]. Available: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2022.1079357
[233] Y. Dan, M. Zhang, Z. Wang, and C. Sun, "Spike-tcn with depthwise-separable convolution and attention for efficient time series prediction,"

International Journal of Computational Intelligence Systems, vol. 18, no. 1, p. 312, 2025.
[234] Y. Fang, Z. Wang, L. Zhang, J. Cao, H. Chen, and R. Xu, "Spiking wavelet transformer," in European conference on computer vision. Springer, 2024, pp. 19–37.
[235] G. Srinivasan and K. Roy, "Restocnet: Residual stochastic binary convolutional spiking neural network for memory-efficient neuromorphic computing," Frontiers in neuroscience, vol. 13, p. 189, 2019.
[236] Y. Luo, M. Xu, C. Yuan, X. Cao, L. Zhang, Y. Xu, T. Wang, and Q. Feng, "Siamsnn: Siamese spiking neural networks for energy-efficient object tracking," in International conference on artificial neural networks. Springer, 2021, pp. 182–194.
[237] N. T. Luu, D. T. Luu, N. N. Pham, and T. C. Truong, "Parameter efficient hybrid spiking-quantum convolutional neural network with surrogate gradient and quantum data-reupload," PeerJ Computer Science, vol. 12, p. e3554, 2026.
[238] L. Zhang, S. Zhou, T. Zhi, Z. Du, and Y. Chen, "Tdsnn: From deep neural networks to deep spike neural networks with temporal-coding," in Proceedings of the AAAI conference on artificial intelligence, vol. 33, no. 01, 2019, pp. 1319–1326.
[239] S. Zhou, Y. Chen, X. Li, and A. Sanyal, "Deep scnn-based real-time object detection for self-driving vehicles using lidar temporal data," Ieee Access, vol. 8, pp. 76 903–76 912, 2020.
[240] A. Amir, B. Taba, D. Berg, T. Melano, J. McKinstry, C. Di Nolfo, T. Nayak, A. Andreopoulos, G. Garreau, M. Mendoza et al., "A low power, fully event-based gesture recognition system," in Proceedings of the IEEE conference on computer vision and pattern recognition, 2017, pp. 7243–7252.
[241] H. Li, H. Liu, X. Ji, G. Li, and L. Shi, "Cifar10-dvs: an event-stream dataset for object classification," Frontiers in neuroscience, vol. 11, p. 244131, 2017.
[242] G. Orchard, A. Jayawant, G. K. Cohen, and N. Thakor, "Converting static image datasets to spiking neuromorphic datasets using saccades," Frontiers in neuroscience, vol. 9, p. 437, 2015.
[243] C.-K. Lin, A. Wild, G. N. Chinya, Y. Cao, M. Davies, D. M. Lavery, and H. Wang, "Programming spiking neural networks on intel's loihi," Computer, vol. 51, no. 3, pp. 52–61, 2018.
[244] K. Buettner and A. D. George, "Heartbeat classification with spiking neural networks on the loihi neuromorphic processor," in 2021 IEEE Computer Society Annual Symposium on VLSI (ISVLSI). IEEE, 2021, pp. 138–143.
[245] P. U. Diehl, B. U. Pedroni, A. Cassidy, P. Merolla, E. Neftci, and G. Zarrella, "Truehappiness: Neuromorphic emotion recognition on truenorth," in 2016 international joint conference on neural networks (ijcnn). IEEE, 2016, pp. 4278–4285.
[246] G. Haessig, A. Cassidy, R. Alvarez, R. Benosman, and G. Orchard, "Spiking optical flow for event-based sensors using ibm's truenorth neurosynaptic system," IEEE transactions on biomedical circuits and systems, vol. 12, no. 4, pp. 860–870, 2018.
[247] M. T. L. Aung, C. Qu, L. Yang, T. Luo, R. S. M. Goh, and W.-F. Wong, "Deepfire: Acceleration of convolutional spiking neural network on modern field programmable gate arrays," in 2021 31st International Conference on Field-Programmable Logic and Applications (FPL). IEEE, 2021, pp. 28–32.
[248] M. T. L. Aung, D. Gerlinghoff, C. Qu, L. Yang, T. Huang, R. S. M. Goh, T. Luo, and W.-F. Wong, "Deepfire2: A convolutional spiking neural network accelerator on fpgas," IEEE Transactions on Computers, vol. 72, no. 10, pp. 2847–2857, 2023.
[249] J. Li, G. Shen, D. Zhao, Q. Zhang, and Y. Zeng, "Firefly v2: Advancing hardware support for high-performance spiking neural network with a spatiotemporal fpga accelerator," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 43, no. 9, pp. 2647–2660, 2024.
[250] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," Communications of the ACM, vol. 63, no. 11, pp. 139–144, 2020.
[251] M. Mirza and S. Osindero, "Conditional generative adversarial nets," arXiv preprint arXiv:1411.1784, 2014.
[252] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," arXiv preprint arXiv:1511.06434, 2015.
[253] V. Kotariya and U. Ganguly, "Spiking-gan: A spiking generative adversarial network using time-to-first-spike coding," in 2022 international joint conference on neural networks (IJCNN). IEEE, 2022, pp. 1–7.
[254] M. Liu, J. Gan, R. Wen, T. Li, Y. Chen, and H. Chen, "Spiking-diffusion: Vector quantized discrete diffusion model with spiking neural networks," in 2024 5th International Conference on Computer, Big Data and Artificial Intelligence (ICCBD+ AI). IEEE, 2024, pp. 627–631.
[255] B. Rosenfeld, O. Simeone, and B. Rajendran, "Spiking generative adversarial networks with a neural network discriminator: Local training, bayesian models, and continual meta-learning," IEEE Transactions on Computers, vol. 71, no. 11, pp. 2778–2791, 2022.
[256] L. Feng, D. Zhao, and Y. Zeng, "Spiking generative adversarial network with attention scoring decoding," Neural Networks, vol. 178, p. 106423, 2024.
[257] H. Kamata, Y. Mukuta, and T. Harada, "Fully spiking variational autoencoder," in Proceedings of the AAAI conference on artificial intelligence, vol. 36, no. 6, 2022, pp. 7059–7067.
[258] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proceedings of the IEEE conference on computer vision and pattern recognition, 2016, pp. 779–788.
[259] S. Kim, S. Park, B. Na, and S. Yoon, "Spiking-yolo: spiking neural network for energy-efficient object detection," in Proceedings of the AAAI conference on artificial intelligence, vol. 34, no. 07, 2020, pp. 11 270–11 277.
[260] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in International Conference on Medical image computing and computer-assisted intervention. Springer, 2015, pp. 234–241.
[261] K. Patel, E. Hunsberger, S. Batir, and C. Eliasmith, "A spiking neural network for image segmentation," arXiv preprint arXiv:2106.08921, 2021.
[262] Q. Su, W. He, X. Wei, B. Xu, and G. Li, "Multi-scale full spike pattern for semantic segmentation," Neural Networks, vol. 176, p. 106330, 2024.
[263] R. Zhang, L. Leng, K. Che, H. Zhang, J. Cheng, Q. Guo, J. Liao, and R. Cheng, "Accurate and efficient event-based semantic segmentation using adaptive spiking encoder–decoder network," IEEE Transactions on Neural Networks and Learning Systems, 2024.
[264] A. Ahmadi and I. Patras, "Unsupervised convolutional neural networks for motion estimation," in 2016 IEEE international conference on image processing (ICIP). IEEE, 2016, pp. 1629–1633.
[265] V. Sudevan and J. Dias, "Neurovio: Snn framework for visual-inertial pose estimation," in IROS 2025 Workshop on Edge AI for Robotics: Emerging Technologies and Applications. IEEE, 2025.
[266] Y. Liu, J. Mei, T. Feng, T. Zhang, H. Qu, and Y. Zhang, "Biomotion-snn: Spiking neural network modeling for visual motion processing," Neural Networks, p. 108239, 2025.
[267] Y. Zheng, Z. Yu, S. Wang, and T. Huang, "Spike-based motion estimation for object tracking through bio-inspired unsupervised learning," IEEE Transactions on Image Processing, vol. 32, pp. 335–349, 2022.
[268] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," IEEE transactions on neural networks and learning systems, vol. 32, no. 1, pp. 4–24, 2020.
[269] Z. Guo and H. Wang, "A deep graph neural network-based mechanism for social recommendations," IEEE Transactions on Industrial Informatics, vol. 17, no. 4, pp. 2776–2783, 2020.
[270] B. Khemani, S. Patil, K. Kotecha, and S. Tanwar, "A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions," Journal of Big Data, vol. 11, no. 1, p. 18, 2024.
[271] G. Muzio, L. O'Bray, and K. Borgwardt, "Biological network analysis with deep learning," Briefings in bioinformatics, vol. 22, no. 2, pp. 1515–1530, 2021.
[272] M. Zhang, Z. Cui, M. Neumann, and Y. Chen, "An end-to-end deep learning architecture for graph classification," in Proceedings of the AAAI conference on artificial intelligence, vol. 32, no. 1, 2018.
[273] O. Wieder, S. Kohlbacher, M. Kuenemann, A. Garon, P. Ducrot, T. Seidel, and T. Langer, "A compact review of molecular property prediction with graph neural networks," Drug Discovery Today: Technologies, vol. 37, pp. 1–12, 2020.
[274] K. Jha, S. Saha, and H. Singh, "Prediction of protein–protein interaction using graph neural networks," Scientific Reports, vol. 12, no. 1, p. 8360, 2022.
[275] S. Sharifzadeh, S. M. Baharlou, and V. Tresp, "Classification by attention: Scene graph classification with prior knowledge," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, no. 6, 2021, pp. 5025–5033.

[276] L. Cai, J. Li, J. Wang, and S. Ji, "Line graph neural networks for link prediction," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 44, no. 9, pp. 5103–5113, 2021.
[277] S. Wu, F. Sun, W. Zhang, X. Xie, and B. Cui, "Graph neural networks in recommender systems: a survey," ACM computing surveys, vol. 55, no. 5, pp. 1–37, 2022.
[278] M. Zamini, H. Reza, and M. Rabiei, "A review of knowledge graph completion," Information, vol. 13, no. 8, p. 396, 2022.
[279] K. Han, Y. Wang, J. Guo, Y. Tang, and E. Wu, "Vision gnn: An image is worth graph of nodes," Advances in neural information processing systems, vol. 35, pp. 8291–8303, 2022.
[280] Y. Lu, Y. Chen, D. Zhao, B. Liu, Z. Lai, and J. Chen, "Cnn-g: Convolutional neural network combined with graph for image segmentation with theoretical analysis," IEEE Transactions on Cognitive and Developmental Systems, vol. 13, no. 3, pp. 631–644, 2020.
[281] S. Zorzi and F. Fraundorfer, "Re:polyworld - a graph neural network for polygonal scene parsing," in Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), October 2023, pp. 16 762–16 771.
[282] R. Liao, Y. Li, Y. Song, S. Wang, W. Hamilton, D. K. Duvenaud, R. Urtasun, and R. Zemel, "Efficient graph generation with graph recurrent attention networks," Advances in neural information processing systems, vol. 32, 2019.
[283] J. Xiong, Z. Xiong, K. Chen, H. Jiang, and M. Zheng, "Graph neural networks for automated de novo drug design," Drug discovery today, vol. 26, no. 6, pp. 1382–1393, 2021.
[284] M. Ferriol-Galmés, J. Paillisse, J. Suárez-Varela, K. Rusek, S. Xiao, X. Shi, X. Cheng, P. Barlet-Ros, and A. Cabellos-Aparicio, "Routenet-fermi: Network modeling with graph neural networks," IEEE/ACM transactions on networking, vol. 31, no. 6, pp. 3080–3095, 2023.
[285] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, "Temporal graph networks for deep learning on dynamic graphs," arXiv preprint arXiv:2006.10637, 2020.
[286] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," arXiv preprint arXiv:1709.04875, 2017.
[287] S. Deng, H. Rangwala, and Y. Ning, "Dynamic knowledge graph based multi-event forecasting," in Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining, 2020, pp. 1585–1595.
[288] L. Peng, N. Wang, J. Xu, X. Zhu, and X. Li, "Gate: Graph cca for temporal self-supervised learning for label-efficient fmri analysis," IEEE Transactions on Medical Imaging, vol. 42, no. 2, pp. 391–402, 2022.
[289] S. Islam, M. N. Hasan, and P. Khanra, "A structural feature-based approach for comprehensive graph classification," Journal of Computational Science, p. 102679, 2025.
[290] M. E. Newman, S. H. Strogatz, and D. J. Watts, "Random graphs with arbitrary degree distributions and their applications," Physical review E, vol. 64, no. 2, p. 026118, 2001.
[291] E. D. Demaine, D. Emanuel, A. Fiat, and N. Immorlica, "Correlation clustering in general weighted graphs," Theoretical Computer Science, vol. 361, no. 2-3, pp. 172–187, 2006.
[292] A. V. Goldberg and C. Harrelson, "Computing the shortest path: A search meets graph theory." in SODA, vol. 5, 2005, pp. 156–165.
[293] Z. Wu and V. M. Preciado, "Laplacian spectral properties of graphs from random local samples," in Proceedings of the 2014 SIAM International Conference on Data Mining. SIAM, 2014, pp. 343–351.
[294] J. M. Klusowski and Y. Wu, "Counting motifs with graph sampling," in Conference On Learning Theory. PMLR, 2018, pp. 1966–2011.
[295] N. M. Kriege, F. D. Johansson, and C. Morris, "A survey on graph kernels," Applied Network Science, vol. 5, no. 1, p. 6, 2020.
[296] S. V. N. Vishwanathan, N. N. Schraudolph, R. Kondor, and K. M. Borgwardt, "Graph kernels," The Journal of Machine Learning Research, vol. 11, pp. 1201–1242, 2010.
[297] N. Shervashidze, P. Schweitzer, E. J. Van Leeuwen, K. Mehlhorn, and K. M. Borgwardt, "Weisfeiler-lehman graph kernels." Journal of Machine Learning Research, vol. 12, no. 9, 2011.
[298] Y. Boykov, O. Veksler, and R. Zabih, "Markov random fields with efficient approximations," in Proceedings. 1998 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (Cat. No. 98CB36231). IEEE, 1998, pp. 648–655.
[299] C. Sutton and A. McCallum, "An introduction to conditional random fields," Foundations and Trends® in Machine Learning, vol. 4, no. 4, pp. 267–373, 2012.
[300] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining, 2014, pp. 701–710.
[301] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining, 2016, pp. 855–864.
[302] Y. Alizadeh, S. Klavžar, and J. Langari, "Extremal results on k-stepwise irregular graphs," Applied Mathematics and Computation, vol. 514, p. 129818, 2026.
[303] T. Ji, Y. Wu, and M. Lan, "Graph-based dependency parsing with graph neural networks," in Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019, pp. 2475–2485.
[304] V. Garg, S. Jegelka, and T. Jaakkola, "Generalization and representational limits of graph neural networks," in International conference on machine learning. PMLR, 2020, pp. 3419–3430.
[305] D. Jin, R. Wang, M. Ge, D. He, X. Li, W. Lin, and W. Zhang, "Raw-gnn: Random walk aggregation based graph neural network," arXiv preprint arXiv:2206.13953, 2022.
[306] A. Sharma, S. Singh, and S. Ratna, "Graph neural network operators: a review," Multimedia Tools and Applications, vol. 83, no. 8, pp. 23 413–23 436, 2024.
[307] Z. Zhu, J. Peng, J. Li, L. Chen, Q. Yu, and S. Luo, "Spiking graph convolutional networks," arXiv preprint arXiv:2205.02767, 2022.
[308] Y. Sun, D. Zhu, Y. Wang, Z. Tian, N. Cao, and G. O'Hared, "Spikegraphormer: A high-performance graph transformer with spiking graph attention," arXiv preprint arXiv:2403.15480, 2024.
[309] N. Yin, M. Wang, Z. Chen, G. De Masi, H. Xiong, and B. Gu, "Dynamic spiking graph neural networks," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, no. 15, 2024, pp. 16 495–16 503.
[310] H. Zhao, X. Yang, C. Deng, and J. Yan, "Dynamic reactive spiking graph neural network," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, no. 15, 2024, pp. 16 970–16 978.
[311] N. Yin, M. Wan, L. Shen, H. L. Patel, B. Li, B. Gu, and H. Xiong, "Continuous spiking graph neural networks," arXiv preprint arXiv:2404.01897, 2024.
[312] J. Li, Z. Yu, Z. Zhu, L. Chen, Q. Yu, Z. Zheng, S. Tian, R. Wu, and C. Meng, "Scaling up dynamic graph representation learning via spiking neural networks," in Proceedings of the AAAI conference on artificial intelligence, vol. 37, no. 7, 2023, pp. 8588–8596.
[313] M. Xu, Y. Wu, L. Deng, F. Liu, G. Li, and J. Pei, "Exploiting spiking dynamics with spatial-temporal feature normalization in graph learning," arXiv preprint arXiv:2107.06865, 2021.
[314] B. Wang, B. Jiang, J. Tang, L. Bai, and B. Luo, "Graph spiking attention network: Sparsity, efficiency and robustness," IEEE Transactions on Pattern Analysis and Machine Intelligence, 2025.
[315] Z. Zhang, P. Cui, and W. Zhu, "Deep learning on graphs: A survey," IEEE Transactions on Knowledge and Data Engineering, vol. 34, no. 1, pp. 249–270, 2020.
[316] R. Sun, H. Dai, and A. W. Yu, "Does gnn pretraining help molecular representation?" Advances in Neural Information Processing Systems, vol. 35, pp. 12 096–12 109, 2022.
[317] X. Yin, W. Lin, K. Sun, C. Wei, and Y. Chen, "A 2 s 2-gnn: Rigging gnn-based social status by adversarial attacks in signed social networks," IEEE Transactions on Information Forensics and Security, vol. 18, pp. 206–220, 2022.
[318] W. Shi and R. Rajkumar, "Point-gnn: Graph neural network for 3d object detection in a point cloud," in Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2020, pp. 1711–1719.
[319] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, and P. S. Yu, "Heterogeneous graph attention network," in The world wide web conference, 2019, pp. 2022–2032.
[320] L. Sun, Z. Huang, Q. Wan, H. Peng, and P. S. Yu, "Spiking graph neural network on riemannian manifolds," Advances in Neural Information Processing Systems, vol. 37, pp. 34 025–34 055, 2024.
[321] S. Snyder, V. Clerico, G. Cong, S. Kulkarni, C. Schuman, S. Risbud, and M. Parsa, "Transductive spiking graph neural networks for loihi," in Proceedings of the Great Lakes Symposium on VLSI 2024, 2024, pp. 608–613.
[322] J. E. Pedersen, S. Abreu, M. Jobst, G. Lenz, V. Fra, F. C. Bauer, D. R. Muir, P. Zhou, B. Vogginger, K. Heckel et al., "Neuromorphic intermediate representation: A unified instruction set for interoperable brain-inspired computing," Nature Communications, vol. 15, no. 1, p. 8122, 2024.

[323] J. B. Aimone, Y. Ho, O. Parekh, C. A. Phillips, A. Pinar, W. Severa, and Y. Wang, "Provable advantages for graph algorithms in spiking neural networks," in Proceedings of the 33rd ACM Symposium on Parallelism in Algorithms and Architectures, 2021, pp. 35–47.

[324] S. Cai, R. Zhang, M. Zhang, J. Wu, and H. Li, "Eeg-based auditory attention detection with spiking graph convolutional network," IEEE Transactions on Cognitive and Developmental Systems, vol. 16, no. 5, pp. 1698–1706, 2024.

[325] P. Gong, P. Wang, Y. Zhou, and D. Zhang, "A spiking neural network with adaptive graph convolution and lstm for eeg-based brain-computer interfaces," IEEE Transactions on Neural Systems and Rehabilitation Engineering, vol. 31, pp. 1440–1450, 2023.

[326] F. Gu, W. Sng, T. Taunyazov, and H. Soh, "Tactilesgnet: A spiking graph neural network for event-based tactile object recognition," in 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2020, pp. 9876–9882.

[327] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," arXiv preprint arXiv:1301.3781, 2013.

[328] G. E. Hinton, "Learning distributed representations of concepts," in Proceedings of the Annual Meeting of the Cognitive Science Society, vol. 8, 1986.

[329] S. Minaee, N. Kalchbrenner, E. Cambria, N. Nikzad, M. Chenaghlu, and J. Gao, "Deep learning–based text classification: a comprehensive review," ACM computing surveys (CSUR), vol. 54, no. 3, pp. 1–40, 2021.

[330] D. W. Otter, J. R. Medina, and J. K. Kalita, "A survey of the usages of deep learning for natural language processing," IEEE transactions on neural networks and learning systems, vol. 32, no. 2, pp. 604–624, 2020.

[331] P. Kherwa and P. Bansal, "Topic modeling: a comprehensive review." EAI Endorsed Trans. Scalable Inf. Syst., vol. 7, no. 24, p. e2, 2020.

[332] L. Deng, G. Hinton, and B. Kingsbury, "New types of deep neural network learning for speech recognition and related applications: An overview," in 2013 IEEE international conference on acoustics, speech and signal processing. IEEE, 2013, pp. 8599–8603.

[333] J. Gehring, M. Auli, D. Grangier, D. Yarats, and Y. N. Dauphin, "Convolutional sequence to sequence learning," in International conference on machine learning. PMLR, 2017, pp. 1243–1252.

[334] R.-J. Zhu, Q. Zhao, G. Li, and J. K. Eshraghian, "Spikegpt: Generative pre-trained language model with spiking neural networks," arXiv preprint arXiv:2302.13939, 2023.

[335] Z. Wang, Y. Fang, J. Cao, Q. Zhang, Z. Wang, and R. Xu, "Masked spiking transformer," in Proceedings of the IEEE/CVF international conference on computer vision, 2023, pp. 1761–1771.

[336] K. Che, Z. Zhou, J. Niu, Z. Ma, W. Fang, Y. Chen, S. Shen, L. Yuan, and Y. Tian, "Auto-spikformer: Spikformer architecture search," Frontiers in Neuroscience, vol. 18, p. 1372257, 2024.

[337] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in Proceedings of the IEEE/CVF international conference on computer vision, 2021, pp. 10 012–10 022.

[338] L. Deng and Y. Liu, Deep learning in natural language processing. Springer, 2018.

[339] S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain, and J. Gao, "Large language models: A survey," arXiv preprint arXiv:2402.06196, 2024.

[340] R. A. Knipper, K. Mishty, M. Sadi, and S. K. K. Santu, "Snnlp: energy-efficient natural language processing using spiking neural networks," arXiv preprint arXiv:2401.17911, 2024.

[341] Q. Su, S. Mei, X. Xing, M. Yao, J. Zhang, B. Xu, and G. Li, "Snn-bert: Training-efficient spiking neural networks for energy-efficient bert," Neural Networks, vol. 180, p. 106630, 2024.

[342] Z. Zhou, Y. Zhu, C. He, Y. Wang, S. YAN, Y. Tian, and L. Yuan, "Spikformer: When spiking neural network meets transformer," in The Eleventh International Conference on Learning Representations, 2023. [Online]. Available: https://openreview.net/forum?id=frE4fUwz_h

[343] M. Białas, M. M. Mirończuk, and J. Mańdziuk, "Leveraging spiking neural networks for topic modeling," Neural Networks, vol. 178, p. 106494, 2024.

[344] M. Malik, M. K. Malik, K. Mehmood, and I. Makhdoom, "Automatic speech recognition: a survey," Multimedia Tools and Applications, vol. 80, no. 6, pp. 9411–9457, 2021.

[345] R. S. Sutton, A. G. Barto et al., Reinforcement learning: An introduction. MIT press Cambridge, 1998, vol. 1, no. 1.

[346] C. C. White and D. J. White, "Markov decision processes," European Journal of Operational Research, vol. 39, no. 1, pp. 1–16, 1989. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0377221789903482

[347] R. Tedrake, T. W. Zhang, H. S. Seung et al., "Stochastic policy gradient reinforcement learning on a simple 3d biped." in IROS. Sendai, Japan, 2004, pp. 2849–2854.

[348] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in International conference on machine learning. Pmlr, 2014, pp. 387–395.

[349] J. Boyan and A. Moore, "Generalization in reinforcement learning: Safely approximating the value function," Advances in neural information processing systems, vol. 7, 1994.

[350] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," in Proceedings of the AAAI conference on artificial intelligence, vol. 30, no. 1, 2016.

[351] B. O'Donoghue, I. Osband, R. Munos, and V. Mnih, "The uncertainty bellman equation and exploration," in International conference on machine learning, 2018, pp. 3836–3845.

[352] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," arXiv preprint arXiv:1312.5602, 2013.

[353] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," The International Journal of Robotics Research, vol. 32, no. 11, pp. 1238–1274, 2013.

[354] A. E. Sallab, M. Abdou, E. Perot, and S. Yogamani, "Deep reinforcement learning framework for autonomous driving," arXiv preprint arXiv:1704.02532, 2017.

[355] Z. Z. Saleh, M. F. Abbod, and R. Nilavalan, "Intelligent resource allocation via hybrid reinforcement learning in 5g network slicing," IEEE Access, vol. 13, pp. 47 440–47 458, 2025.

[356] M. M. Afsar, T. Crump, and B. Far, "Reinforcement learning based recommender systems: A survey," ACM Comput. Surv., vol. 55, no. 7, Dec. 2022. [Online]. Available: https://doi.org/10.1145/3543846

[357] X. Wang, S. Wang, X. Liang, D. Zhao, J. Huang, X. Xu, B. Dai, and Q. Miao, "Deep reinforcement learning: A survey," IEEE Transactions on Neural Networks and Learning Systems, vol. 35, no. 4, pp. 5064–5078, 2022.

[358] A. G. Barto, S. J. Bradtke, and S. P. Singh, "Learning to act using real-time dynamic programming," Artificial intelligence, vol. 72, no. 1-2, pp. 81–138, 1995.

[359] J. Achiam, "Spinning Up in Deep Reinforcement Learning," 2018.

[360] G. Tesauro, "Practical issues in temporal difference learning," Advances in neural information processing systems, vol. 4, 1991.

[361] C. J. Watkins and P. Dayan, "Q-learning," Machine learning, vol. 8, no. 3, pp. 279–292, 1992.

[362] S. Singh, T. Jaakkola, M. L. Littman, and C. Szepesvári, "Convergence results for single-step on-policy reinforcement-learning algorithms," Machine learning, vol. 38, no. 3, pp. 287–308, 2000.

[363] V. Konda and J. Tsitsiklis, "Actor-critic algorithms," Advances in neural information processing systems, vol. 12, 1999.

[364] J. Park and I. W. Sandberg, "Universal approximation using radial-basis-function networks," Neural computation, vol. 3, no. 2, pp. 246–257, 1991.

[365] S. Amani, C. Thrampoulidis, and L. Yang, "Safe reinforcement learning with linear function approximation," in International Conference on Machine Learning. PMLR, 2021, pp. 243–253.

[366] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," Advances in neural information processing systems, vol. 12, 1999.

[367] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017.

[368] G. Tang, N. Kumar, R. Yoo, and K. Michmizos, "Deep reinforcement learning with population-coded spiking neural network for continuous control," in Conference on robot learning. PMLR, 2021, pp. 2016–2029.

[369] B. R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A. A. Al Sallab, S. Yogamani, and P. Pérez, "Deep reinforcement learning for autonomous driving: A survey," IEEE transactions on intelligent transportation systems, vol. 23, no. 6, pp. 4909–4926, 2021.

[370] K. Shao, Z. Tang, Y. Zhu, N. Li, and D. Zhao, "A survey of deep reinforcement learning in video games," arXiv preprint arXiv:1912.10944, 2019.

[371] Z. Yang, S. Guo, Y. Fang, Z. Yu, and J. K. Liu, "Spiking variational policy gradient for brain inspired reinforcement learning," IEEE Transactions on Pattern Analysis and Machine Intelligence, 2024.
[372] T. Nakano, M. Otsuka, J. Yoshimoto, and K. Doya, "A spiking neural network model of model-free reinforcement learning with high-dimensional sensory input and perceptual ambiguity," PloS one, vol. 10, no. 3, p. e0115620, 2015.
[373] L. Zanatta, F. Barchi, S. Manoni, S. Tolu, A. Bartolini, and A. Acquaviva, "Exploring spiking neural networks for deep reinforcement learning in robotic tasks," Scientific Reports, vol. 14, no. 1, p. 30648, 2024.
[374] W. Yao, X. Shen, G. Zhang, Z. Lu, J. Wang, Y. Song, and Z. Li, "A spiking neural network based proximal policy optimization method for multi-point imaging mission scheduling of earth observation satellite," Swarm and Evolutionary Computation, vol. 94, p. 101867, 2025.
[375] K. M. Oikonomou, I. Kansizoglou, and A. Gasteratos, "A hybrid spiking neural network reinforcement learning agent for energy-efficient object manipulation," Machines, vol. 11, no. 2, p. 162, 2023.
[376] D. Chen, P. Peng, T. Huang, and Y. Tian, "Deep reinforcement learning with spiking q-learning," arXiv preprint arXiv:2201.09754, 2022.
[377] A. Kumar, L. Zhang, H. Bilal, S. Wang, A. M. Shaikh, L. Bo, A. Rohra, and A. Khalid, "Dsqn: Robust path planning of mobile robot based on deep spiking q-network," Neurocomputing, vol. 634, p. 129916, 2025.
[378] S. Feng, J. Cao, Z. Ou, G. Chen, Y. Zhong, Z. Wang, J. Yan, J. Chen, B. Wang, C. Zou et al., "Brainqn: Enhancing the robustness of deep reinforcement learning with spiking neural networks," Advanced Intelligent Systems, vol. 6, no. 9, p. 2400075, 2024.
[379] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," Machine learning, vol. 8, no. 3, pp. 229–256, 1992.
[380] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," 2019. [Online]. Available: https://arxiv.org/abs/1509.02971
[381] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in International conference on machine learning. PMLR, 2018, pp. 1587–1596.
[382] F. AlMahamid and K. Grolinger, "Autonomous unmanned aerial vehicle navigation using reinforcement learning: A systematic review," Engineering Applications of Artificial Intelligence, vol. 115, p. 105321, 2022.
[383] B. Singh, R. Kumar, and V. P. Singh, "Reinforcement learning in robotic applications: a comprehensive survey," Artificial Intelligence Review, vol. 55, no. 2, pp. 945–990, 2022.
[384] N. C. Luong, D. T. Hoang, S. Gong, D. Niyato, P. Wang, Y.-C. Liang, and D. I. Kim, "Applications of deep reinforcement learning in communications and networking: A survey," IEEE communications surveys & tutorials, vol. 21, no. 4, pp. 3133–3174, 2019.
[385] Y. Li, "Deep reinforcement learning: An overview," arXiv preprint arXiv:1701.07274, 2017.
[386] Y. Sun, Y. Zeng, and Y. Li, "Solving the spike feature information vanishing problem in spiking deep q network with potential based normalization," Frontiers in Neuroscience, vol. 16, p. 953368, 2022.
[387] G. Wu, D. Liang, S. Luan, and J. Wang, "Training spiking neural networks for reinforcement learning tasks with temporal coding method," Frontiers in Neuroscience, vol. 16, p. 877701, 2022.
[388] M. Bouvier, A. Valentian, T. Mesquida, F. Rummens, M. Reyboz, E. Vianello, and E. Beigne, "Spiking neural networks hardware implementations and challenges: A survey," ACM Journal on Emerging Technologies in Computing Systems (JETC), vol. 15, no. 2, pp. 1–35, 2019.
[389] T. Werner, E. Vianello, O. Bichler, D. Garbin, D. Cattaert, B. Yvert, B. De Salvo, and L. Perniola, "Spiking neural networks based on oxram synapses for real-time unsupervised spike sorting," Frontiers in neuroscience, vol. 10, p. 474, 2016.
[390] K. A. Boahen, "Point-to-point connectivity between neuromorphic chips using address events," IEEE Transactions on Circuits and Systems II: Analog and Digital Signal Processing, vol. 47, no. 5, pp. 416–434, 2002.
[391] S.-C. Liu, T. Delbruck, G. Indiveri, A. Whatley, and R. Douglas, Event-based neuromorphic systems. John Wiley & Sons, 2014.
[392] K. N. Dang, A. B. Ahmed, Y. Okuyama, and A. B. Abdallah, "Scalable design methodology and online algorithm for tsv-cluster defects recovery in highly reliable 3d-noc systems," IEEE Transactions on Emerging Topics in Computing, vol. 8, no. 3, pp. 577–590, 2017.
[393] A. Roy, S. Venkataramani, N. Gala, S. Sen, K. Veezhinathan, and A. Raghunathan, "A programmable event-driven architecture for evaluating spiking neural networks," in 2017 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED). IEEE, 2017, pp. 1–6.
[394] S. Yin, S. K. Venkataramanaiah, G. K. Chen, R. Krishnamurthy, Y. Cao, C. Chakrabarti, and J.-s. Seo, "Algorithm and hardware design of discrete-time spiking neural networks based on back propagation with binary activations," in 2017 IEEE biomedical circuits and systems conference (BioCAS). IEEE, 2017, pp. 1–5.
[395] L. A. Camunas-Mesa, Y. L. Dominguez-Cordero, A. Linares-Barranco, T. Serrano-Gotarredona, and B. Linares-Barranco, "A configurable event-driven convolutional node with rate saturation mechanism for modular convnet systems implementation," Frontiers in neuroscience, vol. 12, p. 63, 2018.
[396] D. Ma, J. Shen, Z. Gu, M. Zhang, X. Zhu, X. Xu, Q. Xu, Y. Shen, and G. Pan, "Darwin: A neuromorphic hardware co-processor based on spiking neural networks," Journal of systems architecture, vol. 77, pp. 43–51, 2017.
[397] B. Moons and M. Verhelst, "A 0.3–2.6 tops/w precision-scalable processor for real-time large-scale convnets," in 2016 IEEE Symposium on VLSI Circuits (VLSI-Circuits). IEEE, 2016, pp. 1–2.
[398] P. N. Whatmough, S. K. Lee, H. Lee, S. Rama, D. Brooks, and G.-Y. Wei, "14.3 a 28nm soc with a 1.2 ghz 568nj/prediction sparse deep-neural-network engine with> 0.1 timing error rate tolerance for iot applications," in 2017 IEEE International Solid-State Circuits Conference (ISSCC). IEEE, 2017, pp. 242–243.
[399] G. Urgese, F. Barchi, E. Macii, and A. Acquaviva, "Optimizing network traffic for spiking neural network simulations on densely interconnected many-core neuromorphic platforms," IEEE Transactions on Emerging Topics in Computing, vol. 6, no. 3, pp. 317–329, 2016.
[400] K. Abdelouahab, M. Pelcat, J. Serot, C. Bourrasset, and F. Berry, "Tactics to directly map cnn graphs on embedded fpgas," IEEE Embedded Systems Letters, vol. 9, no. 4, pp. 113–116, 2017.
[401] Q. Chen, C. Gao, X. Fang, and H. Luan, "Skydiver: A spiking neural network accelerator exploiting spatio-temporal workload balance," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 41, no. 12, pp. 5732–5736, 2022.
[402] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "Xnor-net: Imagenet classification using binary convolutional neural networks," in European conference on computer vision. Springer, 2016, pp. 525–542.
[403] X. Sun, S. Yin, X. Peng, R. Liu, J.-s. Seo, and S. Yu, "Xnor-rram: A scalable and parallel resistive synaptic architecture for binary neural networks," in 2018 Design, Automation & Test in Europe Conference & Exhibition (DATE). IEEE, 2018, pp. 1423–1428.
[404] S. G. Hu, G. C. Qiao, T. P. Chen, Q. Yu, Y. Liu, and L. M. Rong, "Quantized stdp-based online-learning spiking neural network," Neural Comput. Appl., vol. 33, no. 19, p. 12317–12332, Oct. 2021. [Online]. Available: https://doi.org/10.1007/s00521-021-05832-y
[405] S. A. Aamir, Y. Stradmann, P. Müller, C. Pehle, A. Hartel, A. Grübl, J. Schemmel, and K. Meier, "An accelerated lif neuronal network array for a large-scale mixed-signal neuromorphic architecture," IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 65, no. 12, pp. 4299–4312, 2018.
[406] B. V. Benjamin, P. Gao, E. McQuinn, S. Choudhary, A. R. Chandrasekaran, J.-M. Bussat, R. Alvarez-Icaza, J. V. Arthur, P. A. Merolla, and K. Boahen, "Neurogrid: A mixed-analog-digital multichip system for large-scale neural simulations," Proceedings of the IEEE, vol. 102, no. 5, pp. 699–716, 2014.
[407] A. J. Humaidi, T. M. Kadhim, S. Hasan, I. K. Ibraheem, and A. T. Azar, "A generic izhikevich-modelled fpga-realized architecture: A case study of printed english letter recognition." in ICSTCC, 2020, pp. 825–830.
[408] A. Carpegna, A. Savino, and S. Di Carlo, "Spiker: an fpga-optimized hardware accelerator for spiking neural networks," in 2022 IEEE Computer Society Annual Symposium on VLSI (ISVLSI). IEEE, 2022, pp. 14–19.
[409] Z. Zhang and T. G. Constandinou, "Adaptive spike detection and hardware optimization towards autonomous, high-channel-count bmis," Journal of Neuroscience Methods, vol. 354, p. 109103, 2021.
[410] S. Nitzsche, B. Pachideh, N. Luhn, and J. Becker, "Digital hardware implementation of optimized spiking neurons," in 2021 International Conference on Neuromorphic Computing (ICNC). IEEE, 2021, pp. 126–134.
[411] G.-S. Yi, J. Wang, H.-Y. Li, X.-L. Wei, and B. Deng, "Metabolic energy of action potentials modulated by spike frequency adaptation," Frontiers in neuroscience, vol. 10, p. 534, 2016.

[412] S. Li, C. Yan, and Y. Liu, "Energy efficiency and coding of neural network," Frontiers in Neuroscience, vol. 16, p. 1089373, 2023.
[413] N. Qiao, H. Mostafa, F. Corradi, M. Osswald, F. Stefanini, D. Sumislawska, and G. Indiveri, "A reconfigurable on-line learning spiking neuromorphic processor comprising 256 neurons and 128k synapses," Frontiers in neuroscience, vol. 9, p. 141, 2015.
[414] J. Han and M. Orshansky, "Approximate computing: An emerging paradigm for energy-efficient design," in 2013 18th IEEE European test symposium (ETS). IEEE, 2013, pp. 1–6.
[415] S. Sen, S. Venkataramani, and A. Raghunathan, "Approximate computing for spiking neural networks," in Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017. IEEE, 2017, pp. 193–198.
[416] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," The journal of machine learning research, vol. 15, no. 1, pp. 1929–1958, 2014.
[417] L. Wan, M. Zeiler, S. Zhang, Y. Le Cun, and R. Fergus, "Regularization of neural networks using dropconnect," in International conference on machine learning. PMLR, 2013, pp. 1058–1066.
[418] A. Linares-Barranco, M. Oster, D. Cascado, G. Jiménez, A. Civit, and B. Linares-Barranco, "Inter-spike-intervals analysis of aer poisson-like generator hardware," Neurocomputing, vol. 70, no. 16-18, pp. 2692–2700, 2007.
[419] P. Knag, W. Lu, and Z. Zhang, "A native stochastic computing architecture enabled by memristors," IEEE Transactions on Nanotechnology, vol. 13, no. 2, pp. 283–293, 2014.
[420] M. Jerry, A. Parihar, B. Grisafe, A. Raychowdhury, and S. Datta, "Ultra-low power probabilistic imt neurons for stochastic sampling machines," in 2017 Symposium on VLSI Technology. IEEE, 2017, pp. T186–T187.
[421] G. K. Chen, R. Kumar, H. E. Sumbul, P. C. Knag, and R. K. Krishnamurthy, "A 4096-neuron 1m-synapse 3.8-pj/sop spiking neural network with on-chip stdp learning and sparse weights in 10-nm finfet cmos," IEEE Journal of Solid-State Circuits, vol. 54, no. 4, pp. 992–1002, 2018.
[422] N. Rathi, P. Panda, and K. Roy, "Stdp-based pruning of connections and weight quantization in spiking neural networks for energy-efficient recognition," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 38, no. 4, pp. 668–677, 2018.
[423] C. Tang and J. Han, "Hardware efficient weight-binarized spiking neural networks," in 2023 Design, Automation & Test in Europe Conference & Exhibition (DATE). IEEE, 2023, pp. 1–6.
[424] A. Awang, "Receptive fields, binocular interaction and functional architecture in the cat's visual cortex," The Journal of physiology, 1962.
[425] T. Serre, L. Wolf, S. Bileschi, M. Riesenhuber, and T. Poggio, "Robust object recognition with cortex-like mechanisms," IEEE transactions on pattern analysis and machine intelligence, vol. 29, no. 3, pp. 411–426, 2007.
[426] S. AMART and M. Arbib, "Competition and cooperation in neural nets," Springer Lecture Notes in Biomathematics, vol. 45, 1977.
[427] W. Maass, "On the computational power of winner-take-all," Neural computation, vol. 12, no. 11, pp. 2519–2535, 2000.
[428] C. D. Salzman and W. T. Newsome, "Neural mechanisms for forming a perceptual decision," Science, vol. 264, no. 5156, pp. 231–237, 1994.
[429] P. V. Tymoshchuk and D. C. Wunsch, "Design of a k-winners-take-all model with a binary spike train," IEEE Transactions on Cybernetics, vol. 49, no. 8, pp. 3131–3140, 2018.
[430] H. Chu, Y. Yan, L. Gan, H. Jia, L. Qian, Y. Huan, L. Zheng, and Z. Zou, "A neuromorphic processing system with spike-driven snn processor for wearable ecg classification," IEEE Transactions on Biomedical Circuits and Systems, vol. 16, no. 4, pp. 511–523, 2022.
[431] F. Corradi, S. Pande, J. Stuijt, N. Qiao, S. Schaafsma, G. Indiveri, and F. Catthoor, "Ecg-based heartbeat classification in neuromorphic hardware," in 2019 International Joint Conference on Neural Networks (IJCNN). IEEE, 2019, pp. 1–8.
[432] Y. Yan, H. Chu, X. Chen, Y. Jin, Y. Huan, L. Zheng, and Z. Zou, "Graph-based spatio-temporal backpropagation for training spiking neural networks," in 2021 IEEE 3rd international conference on artificial intelligence circuits and systems (AICAS). IEEE, 2021, pp. 1–4.
[433] Y. Jin, Y. Liu, and P. Li, "Sso-lsm: A sparse and self-organizing architecture for liquid state machine based neural processors," in 2016 IEEE/ACM international symposium on nanoscale architectures (NANOARCH). IEEE, 2016, pp. 55–60.
[434] B. Schrauwen, M. D'Haene, D. Verstraeten, and J. Van Campenhout, "Compact hardware liquid state machines on fpga for real-time speech recognition," Neural networks, vol. 21, no. 2-3, pp. 511–523, 2008.
[435] Q. Wang, Y. Jin, and P. Li, "General-purpose lsm learning processor architecture and theoretically guided design space exploration," in 2015 IEEE Biomedical Circuits and Systems Conference (BioCAS). IEEE, 2015, pp. 1–4.
[436] Y. Paquot, F. Duport, A. Smerieri, J. Dambre, B. Schrauwen, M. Haelterman, and S. Massar, "Optoelectronic reservoir computing," Scientific reports, vol. 2, no. 1, p. 287, 2012.
[437] B. W. Ku, Y. Liu, Y. Jin, S. Samal, P. Li, and S. K. Lim, "Design and architectural co-optimization of monolithic 3d liquid state machine-based neuromorphic processor," in Proceedings of the 55th Annual Design Automation Conference, 2018, pp. 1–6.
[438] P. Lichtsteiner, C. Posch, and T. Delbruck, "A 128× 128 120 db 15 μs latency asynchronous temporal contrast vision sensor," IEEE Journal of Solid-State Circuits, vol. 43, no. 2, pp. 566–576, 2008.
[439] C. Posch, D. Matolin, and R. Wohlgenannt, "A qvga 143 db dynamic range frame-free pwm image sensor with lossless pixel-level video compression and time-domain cds," IEEE Journal of Solid-State Circuits, vol. 46, no. 1, pp. 259–275, 2011.
[440] V. Chan, S.-C. Liu, and A. van Schaik, "Aer ear: A matched silicon cochlea pair with address event representation interface," IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 54, no. 1, pp. 48–59, 2007.
[441] C. Zhao, B. T. Wysocki, C. D. Thiem, N. R. McDonald, J. Li, L. Liu, and Y. Yi, "Energy efficient spiking temporal encoder design for neuromorphic computing systems," IEEE Transactions on Multi-Scale Computing Systems, vol. 2, no. 4, pp. 265–276, 2016.
[442] A. Yousefzadeh, G. Orchard, E. Stromatias, T. Serrano-Gotarredona, and B. Linares-Barranco, "Hybrid neural network, an efficient low-power digital hardware implementation of event-based artificial neural network," in 2018 IEEE International Symposium on Circuits and Systems (ISCAS). IEEE, 2018, pp. 1–5.
[443] E. Lemaire, B. Miramond, S. Bilavarn, H. Saoud, and N. Abderrahmane, "Synaptic activity and hardware footprint of spiking neural networks in digital neuromorphic systems," ACM Transactions on Embedded Computing Systems, vol. 21, no. 6, pp. 1–26, 2022.
[444] G. W. Burr, P. Narayanan, R. M. Shelby, S. Sidler, I. Boybat, C. Di Nolfo, and Y. Leblebici, "Large-scale neural networks implemented with non-volatile memory as the synaptic weight element: Comparative performance analysis (accuracy, speed, and power)," in 2015 IEEE International Electron Devices Meeting (IEDM). IEEE, 2015, pp. 4–4.
[445] S. Kim, M. Ishii, S. Lewis, T. Perri, M. BrightSky, W. Kim, R. Jordan, G. W. Burr, N. Sosa, A. Ray et al., "Nvm neuromorphic core with 64k-cell (256-by-256) phase change memory synaptic array with on-chip neuron circuits for continuous in-situ learning," in 2015 IEEE international electron devices meeting (IEDM). IEEE, 2015, pp. 17–1.
[446] Y. Jin and P. Li, "Ap-stdp: A novel self-organizing mechanism for efficient reservoir computing," in 2016 International Joint Conference on Neural Networks (IJCNN). IEEE, 2016, pp. 1158–1165.
[447] Y. Liu, Y. Jin, and P. Li, "Online adaptation and energy minimization for hardware recurrent spiking neural networks," ACM Journal on Emerging Technologies in Computing Systems (JETC), vol. 14, no. 1, pp. 1–21, 2018.
[448] N. Zheng and P. Mazumder, "A low-power hardware architecture for on-line supervised learning in multi-layer spiking neural networks," in 2018 IEEE International Symposium on Circuits and Systems (ISCAS). IEEE, 2018, pp. 1–5.
[449] H. Asgari, B. M.-N. Maybodi, M. Payvand, and M. R. Azghadi, "Low-energy and fast spiking neural network for context-dependent learning on fpga," IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 67, no. 11, pp. 2697–2701, 2020.
[450] M. K. Bahrami and S. Nazari, "Digital design of a spatial-pow-stdp learning block with high accuracy utilizing pow cordic for large-scale image classifier spatiotemporal snn," Scientific reports, vol. 14, no. 1, p. 3388, 2024.
[451] M. Shahsavari, D. Thomas, A. Brown, and W. Luk, "Neuromorphic design using reward-based stdp learning on event-based reconfigurable cluster architecture," in International conference on neuromorphic systems 2021, 2021, pp. 1–8.
[452] J. M. Brader, W. Senn, and S. Fusi, "Learning real-world stimuli in a neural network with spike-driven synaptic dynamics," Neural computation, vol. 19, no. 11, pp. 2881–2912, 2007.
[453] S. Gomar and M. Ahmadi, "Digital realization of pstdp and tstdp learning," in 2018 International Joint Conference on Neural Networks (IJCNN). IEEE, 2018, pp. 1–5.

[454] B. Ans and S. Rousset, "Avoiding catastrophic forgetting by coupling two reverberating neural networks," Comptes Rendus de l'Académie des Sciences-Series III-Sciences de la Vie, vol. 320, no. 12, pp. 989–997, 1997.
[455] R. M. French, "Catastrophic forgetting in connectionist networks," Trends in cognitive sciences, vol. 3, no. 4, pp. 128–135, 1999.
[456] I. J. Goodfellow, M. Mirza, D. Xiao, A. Courville, and Y. Bengio, "An empirical investigation of catastrophic forgetting in gradient-based neural networks," arXiv preprint arXiv:1312.6211, 2013.
[457] A. Shaban, S. S. Bezugam, and M. Suri, "An adaptive threshold neuron for recurrent spiking neural networks with nanodevice hardware implementation," Nature Communications, vol. 12, no. 1, p. 4234, 2021.
[458] S. Yang, J. Wang, B. Deng, M. R. Azghadi, and B. Linares-Barranco, "Neuromorphic context-dependent learning framework with fault-tolerant spike routing," IEEE transactions on neural networks and learning systems, vol. 33, no. 12, pp. 7126–7140, 2021.
[459] K. Likharev, A. Mayr, I. Muckra, and Ö. Türel, "Crossnets: High-performance neuromorphic architectures for cmol circuits," Annals of the New York Academy of Sciences, vol. 1006, no. 1, pp. 146–163, 2003.
[460] J. Hsu, "Ibm's new brain [news]," IEEE spectrum, vol. 51, no. 10, pp. 17–19, 2014.
[461] S. Moradi, N. Qiao, F. Stefanini, and G. Indiveri, "A scalable multicore architecture with heterogeneous memory structures for dynamic neuromorphic asynchronous processors (dynaps)," IEEE transactions on biomedical circuits and systems, vol. 12, no. 1, pp. 106–122, 2017.
[462] A. Joubert, M. Duranton, B. Belhadj, O. Temam, and R. Héliot, "Capacitance of tsvs in 3-d stacked chips a problem? not for neuromorphic systems!" in Proceedings of the 49th Annual Design Automation Conference, 2012, pp. 1264–1265.
[463] S. Ambrogio, P. Narayanan, H. Tsai, R. M. Shelby, I. Boybat, C. Di Nolfo, S. Sidler, M. Giordano, M. Bodini, N. C. Farinha et al., "Equivalent-accuracy accelerated neural-network training using analogue memory," Nature, vol. 558, no. 7708, pp. 60–67, 2018.
[464] K. Chang, D. Kadetotad, Y. Cao, J.-s. Seo, and S. K. Lim, "Monolithic 3d ic designs for low-power deep neural networks targeting speech recognition," in 2017 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED). IEEE, 2017, pp. 1–6.
[465] B. Belhadj, A. Valentian, P. Vivet, M. Duranton, L. He, and O. Temam, "The improbable but highly appropriate marriage of 3d stacking and neuromorphic accelerators," in Proceedings of the 2014 International Conference on Compilers, Architecture and Synthesis for Embedded Systems, 2014, pp. 1–9.
[466] D. Kim, J. Kung, S. Chai, S. Yalamanchili, and S. Mukhopadhyay, "Neurocube: A programmable digital neuromorphic architecture with high-density 3d memory," ACM SIGARCH Computer Architecture News, vol. 44, no. 3, pp. 380–392, 2016.
[467] P. Batude, C. Fenouillet-Beranger, L. Pasini, V. Lu, F. Deprat, L. Brunet, B. Sklenard, F. Piegas-Luce, M. Cassé, B. Mathieu et al., "3dvlsi with coolcube process: An alternative path to scaling," in 2015 Symposium on VLSI Technology (VLSI Technology). IEEE, 2015, pp. T48–T49.
[468] K. Sakuma, 3D integration in VLSI Circuits: implementation technologies and applications. CRC Press, 2018.
[469] A. Ankit, A. Sengupta, P. Panda, and K. Roy, "Resparc: A reconfigurable and energy-efficient architecture with memristive crossbars for deep spiking neural networks," in Proceedings of the 54th Annual Design Automation Conference 2017, 2017, pp. 1–6.
[470] G. W. Burr, R. M. Shelby, A. Sebastian, S. Kim, S. Kim, S. Sidler, K. Virwani, M. Ishii, P. Narayanan, A. Fumarola et al., "Neuromorphic computing using non-volatile memory," Advances in Physics: X, vol. 2, no. 1, pp. 89–124, 2017.
[471] G. Indiveri, E. Chicca, and R. Douglas, "A vlsi array of low-power spiking neurons and bistable synapses with spike-timing dependent plasticity," IEEE transactions on neural networks, vol. 17, no. 1, pp. 211–221, 2006.
[472] A. M. Zyarah, N. Soures, L. Hays, R. B. Jacobs-Gedrim, S. Agarwal, M. Marinella, and D. Kudithipudi, "Ziksa: On-chip learning accelerator with memristor crossbars for multilevel neural networks," in 2017 IEEE International Symposium on Circuits and Systems (ISCAS). IEEE, 2017, pp. 1–4.
[473] Y. Kim, Y. Zhang, and P. Li, "A reconfigurable digital neuromorphic processor with memristive synaptic crossbar for cognitive computing," ACM Journal on Emerging Technologies in Computing Systems (JETC), vol. 11, no. 4, pp. 1–25, 2015.
[474] P. Merolla, J. Arthur, F. Akopyan, N. Imam, R. Manohar, and D. S. Modha, "A digital neurosynaptic core using embedded crossbar memory with 45pj per spike in 45nm," in 2011 IEEE custom integrated circuits conference (CICC). IEEE, 2011, pp. 1–4.
[475] G. W. Burr, R. M. Shelby, S. Sidler, C. Di Nolfo, J. Jang, I. Boybat, R. S. Shenoy, P. Narayanan, K. Virwani, E. U. Giacometti et al., "Experimental demonstration and tolerancing of a large-scale neural network (165 000 synapses) using phase-change memory as the synaptic weight element," IEEE Transactions on Electron Devices, vol. 62, no. 11, pp. 3498–3507, 2015.
[476] R. B. Jacobs-Gedrim, S. Agarwal, K. E. Knisely, J. E. Stevens, M. S. Van Heukelom, D. R. Hughart, J. Niroula, C. D. James, and M. J. Marinella, "Impact of linearity and write noise of analog resistive memory devices in a neural algorithm accelerator," in 2017 IEEE International Conference on Rebooting Computing (ICRC). IEEE, 2017, pp. 1–10.
[477] S. Kim, M. Lim, Y. Kim, H.-D. Kim, and S.-J. Choi, "Impact of synaptic device variations on pattern recognition accuracy in a hardware neural network," Scientific reports, vol. 8, no. 1, p. 2638, 2018.
[478] J. Xiang, X. Fang, J. Xiao, Y. Chen, A. He, Y. Zhao, Z. Zhao, Y. Su, M. Gu, and X. Guo, "Ghz spiking neuromorphic photonic chip with in-situ training," arXiv preprint arXiv:2506.14272, 2025.
[479] M. Rahman, S. Bose, and S. Chakrabartty, "On-device synaptic memory consolidation using fowler-nordheim quantum-tunneling," Frontiers in Neuroscience, vol. 16, p. 1050585, 2023.
[480] M. Stimberg, R. Brette, and D. F. Goodman, "Brian 2, an intuitive and efficient neural simulator," eLife, vol. 8, p. e47314, Aug. 2019.
[481] M.-O. Gewaltig and M. Diesmann, "Nest (neural simulation tool)," Scholarpedia, vol. 2, no. 4, p. 1430, 2007.
[482] T. Bekolay, J. Bergstra, E. Hunsberger, T. DeWolf, T. Stewart, D. Rasmussen, X. Choo, A. Voelker, and C. Eliasmith, "Nengo: a Python tool for building large-scale functional brain models," Frontiers in Neuroinformatics, vol. 7, no. 48, pp. 1–13, 2014.
[483] M. Mozafari, M. Ganjtabesh, A. Nowzari-Dalini, and T. Masquelier, "Spyketorch: Efficient simulation of convolutional spiking neural networks with at most one spike per neuron," Frontiers in neuroscience, vol. 13, p. 625, 2019.
[484] H. Hazan, D. J. Saunders, H. Khan, D. Patel, D. T. Sanghavi, H. T. Siegelmann, and R. Kozma, "Bindsnet: A machine learning-oriented spiking neural networks library in python," Frontiers in Neuroinformatics, vol. 12, p. 89, 2018. [Online]. Available: https://www.frontiersin.org/article/10.3389/fninf.2018.00089
[485] Lava Community, "Lava: A software framework for neuromorphic computing," https://github.com/lava-nc/lava, 2021, accessed: 2024-05-20.
[486] ——, "Lava-dl: Deep learning library for lava," https://github.com/lava-nc/lava-dl, 2021.
[487] C. Wang, T. Zhang, X. Chen, S. He, S. Li, and S. Wu, "Brainpy, a flexible, integrative, efficient, and extensible framework for general-purpose brain dynamics programming," eLife, vol. 12, p. e86365, dec 2023. [Online]. Available: https://doi.org/10.7554/eLife.86365
[488] C. Pehle and J. E. Pedersen, "Norse - A deep learning library for spiking neural networks," Jan. 2021, documentation: https://norse.ai/docs/. [Online]. Available: https://doi.org/10.5281/zenodo.4422025
[489] E. Yavuz, J. Turner, and T. Nowotny, "Genn: a code generation framework for accelerated brain simulations," Scientific reports, vol. 6, no. 1, p. 18854, 2016.
[490] B. Büller, "PySNN: Spiking neural network framework built on top of PyTorch," https://github.com/BasBuller/PySNN, 2020, accessed: 2025-12-23.
[491] M. Beyeler, K. D. Carlson, T.-S. Chou, N. Dutt, and J. L. Krichmar, "CARLsim 3: A user-friendly and highly optimized library for the creation of neurobiologically detailed spiking neural networks," in 2015 International Joint Conference on Neural Networks (IJCNN), July 2015, pp. 1–8. [Online]. Available: http://dx.doi.org/10.1109/IJCNN.2015.7280424
[492] N. K. Kasabov, "Neucube: A spiking neural network architecture for mapping, learning and understanding of spatio-temporal brain data," Neural networks, vol. 52, pp. 62–76, 2014.
[493] G. Susi, P. Garcés, E. Paracone, A. Cristini, M. Salerno, F. Maestú, and E. Pereda, "Fns allows efficient event-driven spiking neural network simulations based on a neuron model supporting spike latency," Scientific reports, vol. 11, no. 1, p. 12160, 2021.
[494] Y. Zeng, D. Zhao, F. Zhao, G. Shen, Y. Dong, E. Lu, Q. Zhang, Y. Sun, Q. Liang, Y. Zhao, Z. Zhao, H. Fang, Y. Wang, Y. Li, X. Liu, C. Du, Q. Kong, Z. Ruan, and W. Bi, "BrainCog: A spiking neural network

based, brain-inspired cognitive intelligence engine for brain-inspired AI and brain simulation," Patterns, p. 100789, Jul. 2023. [Online]. Available: https://doi.org/10.1016/j.patter.2023.100789



NHAN TRONG LUU (Member, IEEE) received the B.Sc. degree and M.Sc. degree in Computer Science from The University of Aizu, Japan, in 2024 and 2025 respectively. He is currently working as Research Assistant at College of Information and Communication Technology, Can Tho University, Vietnam. His research interests include machine learning, artificial intelligent algorithm, computational neuroscience and quantum computing.



DUONG TRUNG LUU received the B.E. degree in Informatics from Can Tho University (CTU), Vietnam, in 1997 and M.Sc. degree in Mobile and Distributed Computer Network from Leeds Beckett University, England, in 2003. He has been working at Can Tho University since 1997 to present, starting as a Network Manager and currently is the Director of Center for Digital Transformation and Communication. His research interests include large scale networking solution design, computer security and network management.



PHAM NGOC NAM (Member, IEEE) received the bachelor's degree in electronics engineering from the Hanoi University of Science and Technology (HUST), in 1997, and the M.S. degree in artificial intelligence and the Ph.D. degree in electrical engineering from KU Leuven, Belgium, in 1999 and 2004, respectively. He is currently the Vice Dean of the College of Engineering and Computer Science, VinUniversity, Vingroup. He is also a Visiting Scholar with Cornell University. He has been the PI of one key national project and three ministerial-level projects. He is the author or coauthor of 100 scientific articles, including more than 30 ISI and Scopus publications. His research interests include artificial intelligence, QoS/QoE management for multimedia applications, reconfigurable computing, and low-power embedded system design. He has been a key member of four other national projects.



TRUONG CONG THANG (Senior Member, IEEE) received the B.E. degree from Hanoi University of Technology, Vietnam, in 1997 and Ph.D. degree from KAIST, Korea, in 2006. From 1997 to 2000, he worked as an engineer in Vietnam Post & Telecom (VNPT). From 2006 to 2011, he was a Member of Research Staff at Electronics and Telecommunications Research Institute (ETRI), Korea. He was also an active member of Korean delegation to standard meetings of ISO/IEC and ITU-T from 2002 to 2011. Since 2011, he has been an Associate Professor of University of Aizu, Japan. His research interests include multimedia networking, image/video processing, content adaptation, IPTV and MPEG/ITU standards.